PROPUESTA DE CONSULTORÍA

Transmetro de Guatemala

Sistema Unificado de Gestión de Líneas, Estaciones, Flota y Recaudación del Transmetro de Guatemala

Propuesta de consultoría y memoria técnica de ingeniería. Análisis de requerimientos, estudios de factibilidad, modelo de datos, aplicación web y plan de implantación.

Autor	William Alejandro Morales Mendez
Institución	Facultad de Ingeniería en Sistemas, Universidad de Guatemala
Asignatura	Ingeniería de Requerimientos y Arquitectura de Software
Fecha	23 de octubre de 2026
Repositorio	https://github.com/william700morales-cloud/transmetro-sistema
Aplicación web	https://william700morales-cloud.github.io/transmetro-sistema/

Resumen

Este documento presenta la propuesta de consultoría y la memoria técnica del *Sistema Unificado de Gestión de Líneas, Estaciones, Flota y Recaudación del Transmetro de Guatemala*. El sistema atiende el caso planteado por las municipalidades de la capital y de los municipios aledaños: recoger la información de las líneas, sus estaciones en orden, los accesos, los buses y sus parqueos, los pilotos, los guardias de seguridad, las computadoras de cada estación y los pagos, y aplicar de forma automática las reglas de operación del servicio.

La solución se apoya en una base de datos relacional de 27 tablas, 14 vistas y 21 disparadores en PostgreSQL, que hace cumplir en el propio motor las reglas del caso: cada línea tiene entre tantos buses como estaciones y el doble, cada acceso tiene al menos un guardia, cada bus tiene siempre un parqueo, una estación genera una alerta cuando espera 150 % de la capacidad de un bus y un bus que no llena el 25 % espera cinco minutos más. Una aplicación web en React permite consultar y gestionar esa información y muestra 17 reportes con la consulta SQL que los produce. Treinta y seis pruebas automáticas verifican las reglas y todas se superan.

La inversión inicial del corredor piloto de la Línea 12 es de Q 700,500.00 y los gastos operativos anuales de Q 120,000.00. Con una tasa de descuento de 10 % a cinco años, el Valor Presente Neto es de Q 73,921.67 y la Tasa Interna de Retorno de 13.94 %. El documento incluye además el análisis de requerimientos, los estudios de factibilidad, la planificación con diagrama de Gantt, el modelo entidad-relación, los diagramas UML, el análisis de conectividad entre estaciones, las métricas de software, el plan de pruebas, los criterios de aceptación y el modelo de carta de aceptación.

Palabras clave: Transmetro, ingeniería de software, base de datos relacional, PostgreSQL, control de flota, tarjeta de acceso, Guatemala

Abstract

This document presents the consulting proposal and technical report for a unified lines, stations, fleet and fare-collection system for the Transmetro bus rapid transit network in Guatemala City and its neighbouring municipalities. The system stores lines with ordered stations, station accesses, buses and parking lots, pilots, security guards, station workstations and payments, and enforces the operating rules of the case inside the database engine: fleet size between N and 2N buses per line, at least one guard per access, no bus without a parking lot, a capacity alert at 150 % of a bus load and an extra five-minute wait for buses below 25 % occupancy.

The relational model has 27 tables, 14 views and 21 triggers on PostgreSQL, a React web application with 17 SQL-backed reports, and 36 automated rule tests, all of which pass. The pilot corridor requires an initial investment of GTQ 700,500.00; at a 10 % discount rate over five years the Net Present Value is GTQ 73,921.67 and the Internal Rate of Return is 13.94 %.

Keywords: Transmetro, software engineering, relational database, PostgreSQL, fleet management, smart card, Guatemala

Contenido

Resumen **2**

Abstract **3**

Capítulo I. Carta de presentación y resumen de la propuesta **7**

1.1 Carta de presentación personal 7
1.2 Resumen de la propuesta 8
1.3 Cumplimiento de los puntos solicitados 8

Capítulo II. Introducción **10**

2.1 Contexto y justificación 10
2.2 Planteamiento del problema 10
2.3 Objetivos 11
2.4 Alcance y delimitación 11
2.5 Limitaciones 11
2.6 Organización del documento 12

Capítulo III. Marco teórico **13**

3.1 Sistemas de transporte masivo y control operativo 13
3.2 El fenómeno del bus bunching 13
3.3 Pago electrónico sin contacto 13
3.4 Calidad del software y modelo Goal-Question-Metric 13
3.5 Bases de datos relacionales, integridad y disparadores 14
3.6 Redes privadas virtuales entre sedes 14

Capítulo IV. Antecedentes del problema **15**

4.1 El Transmetro de la Ciudad de Guatemala 15
4.2 Origen de la propuesta 15
4.3 Evolución de la necesidad de control 15

Capítulo V. Definición de la situación actual **17**

5.1 Descripción de la operación actual 17
5.2 Actores y flujo de información 17
5.3 Problemas que debe resolver el sistema 17

Capítulo VI. Deficiencias de los controles actuales **19**

Capítulo VII. Análisis de requerimientos **20**

7.1 Las dieciocho condiciones del caso 20
7.2 Interpretaciones y supuestos 21
7.3 Actores y matriz de roles y permisos 21
7.4 Requerimientos funcionales del sistema unificado 22
7.5 Requerimientos no funcionales 25

Capítulo VIII. Propuesta monetaria de la solución **27**

8.1 Presupuesto de inversión inicial y gastos operativos 27
8.2 Desglose del rubro de software 28
8.3 Módulo de estaciones: computadoras y conectividad 28
8.4 Condiciones de pago, vigencia y exclusiones 29

Capítulo IX. Estudios de factibilidad **30**

9.1 Factibilidad técnica 30
9.2 Factibilidad operativa 30
9.3 Factibilidad económica 31
9.4 Factibilidad legal y regulatoria 32
9.5 Factibilidad ambiental 33

Capítulo X. Metodología de desarrollo y planificación **34**

10.1 Enfoque de la investigación 34
10.2 Metodología elegida: Scrum adaptado 34
10.3 Fases y productos 35
10.4 Planificación: diagrama de Gantt 35
10.5 Gestión de riesgos 38
10.6 Técnicas e instrumentos 38

Capítulo XI. Modelo de datos y script de la base de datos **39**

11.1 Motor de base de datos elegido 39
11.2 Modelo entidad-relación 40
11.3 Relaciones y cardinalidades 46
11.4 Tablas y datos de demostración 46
11.5 Reglas aplicadas por el motor 47
11.6 Script de creación de las estructuras 48
11.7 Vistas de apoyo 49

Capítulo XII. Diagramas UML **50**

12.1 Diagrama de casos de uso 51
12.2 Diagrama de componentes 53
12.3 Diagramas de secuencia 54
12.4 Diagramas de actividades 56
12.5 Diagramas de clases 58
12.6 Diagrama de objetos 61

Capítulo XIII. Conectividad entre estaciones y métodos de pago **62**

13.1 El problema de conectar las estaciones 62
13.2 Opciones evaluadas 62
13.3 Matriz de decisión 62
13.4 Arquitectura de red 64
13.5 Funcionamiento ante la pérdida del enlace 65
13.6 Métodos de pago y tarjeta de acceso 65

Capítulo XIV. Aplicación web y base de datos en la nube **66**

14.1 Qué es y cómo se usa 66
14.2 Tecnologías utilizadas 67

14.3 Pantallas principales 67
14.4 Cambios de diseño respecto a la versión anterior 71
14.5 Centro de control C5: simulación de los módulos ampliados 72
14.6 Publicación en la nube 73
14.7 Repositorio y estructura de carpetas 74

Capítulo XV. Reportes de estaciones, líneas y buses 75

Capítulo XVI. Implementación de estándares de seguridad 85
16.1 Seguridad según OWASP Top 10 85
16.2 Controles de ISO/IEC 27001:2022 86
16.3 Seguridad dentro de la base de datos 86
16.4 Modelo de amenazas 87
16.5 Seguridad de la información en el sistema unificado 87

Capítulo XVII. Métricas de software 88
17.1 Modelo de calidad 88
17.2 Métricas medidas en esta versión 88
17.3 Derivación Goal-Question-Metric del nivel de red 89
17.4 Interpretación y acciones 89

Capítulo XVIII. Plan de pruebas 90
18.1 Objetivo y alcance 90
18.2 Estrategia y tipos de prueba 90
18.3 Pruebas automáticas de las reglas del caso 90
18.4 Resultados de la ejecución 92
18.5 Pruebas pendientes 93

Capítulo XIX. Criterios de aceptación 94
19.1 Criterios del nivel de red 94
19.2 Criterios del sistema unificado 95
19.3 Trazabilidad entre condiciones, pruebas y criterios 95

Capítulo XX. Carta de aceptación del proyecto 97

Capítulo XXI. Conclusiones y recomendaciones 98
21.1 Conclusiones 98
21.2 Recomendaciones 98
21.3 Líneas de trabajo futuro 98

Referencias 100

Apéndice A. Glosario de siglas y términos 102

Apéndice B. Script SQL completo de las estructuras 104

Apéndice C. Diccionario de datos 109

Apéndice D. Enlaces a los repositorios y puesta en marcha 111
D.1 Ejecutar la aplicación en un equipo 111
D.2 Publicar la aplicación en GitHub Pages 111
D.3 Crear la base de datos en un servidor PostgreSQL 111

Índice de figuras

Figura 1. Diagrama de Gantt del proyecto 37
Figura 2. Modelo entidad-relación: red de transporte (municipalidades, líneas, estaciones, accesos y tramos) 40
Figura 3. Modelo entidad-relación: flota, parqueos e historial de parqueo 41
Figura 4. Modelo entidad-relación: pilotos y su historial educativo 42
Figura 5. Modelo entidad-relación: guardias, computadora, operadores y enlaces de red de la estación 43
Figura 6. Modelo entidad-relación: recorridos, visitas, lecturas de pasajeros y alertas 44
Figura 7. Modelo entidad-relación: tarjetas de acceso, métodos de pago y pagos 45
Figura 8. Diagrama de casos de uso del sistema de control del Transmetro 51
Figura 9. Diagrama de componentes y despliegue 53
Figura 10. Diagrama de secuencia: asignar un bus a una línea con validación de la flota 54
Figura 11. Diagrama de secuencia: lectura de pasajeros, alerta y bus de refuerzo 55
Figura 12. Diagrama de actividades: registro de una visita a una estación 56
Figura 13. Diagrama de actividades: atención de una alerta de capacidad 57
Figura 14. Diagrama de clases: red de transporte 58
Figura 15. Diagrama de clases: personal, estación y comunicaciones 59
Figura 16. Diagrama de clases: operación y recaudo 60
Figura 17. Diagrama de objetos con datos del ejemplo 61
Figura 18. Arquitectura de red entre las estaciones y el centro de control 64
Figura 19. Pestaña «Líneas y flota»: estaciones en orden, distancias y flota de la Línea 12 68
Figura 20. El sistema rechaza el cambio que rompe la regla de flota 69
Figura 21. Pestaña «Estación del operador»: alerta de capacidad en El Trébol 70
Figura 22. Pestaña «Base de datos»: pruebas automáticas de las reglas 71
Figura 23. Centro de control C5: flota y terminales con detección de bus bunching 73

Capítulo I. Carta de presentación y resumen de la propuesta

1.1 Carta de presentación personal

Ciudad de Guatemala, 23 de octubre de 2026

Señores
Alcaldes y Concejos Municipales
Municipalidades de la Región Metropolitana de Guatemala
Presente

Asunto: Propuesta de consultoría para el sistema de control del Transmetro.

Estimados señores:

Me dirijo a ustedes para presentar mi propuesta de consultoría para el sistema que controlará las líneas, estaciones, buses, pilotos y accesos del Transmetro en la capital y en los municipios aledaños. Me llamo William Alejandro Morales Mendez y soy estudiante de ingeniería en sistemas en la Facultad de Ingeniería en Sistemas, Universidad de Guatemala. Esta propuesta se elabora en el marco del curso *Ingeniería de Requerimientos y Arquitectura de Software*, lo que significa que se trabaja con el rigor de un proyecto real, pero que las cifras y los datos de demostración son referenciales y deben validarse con información de cada municipalidad antes de cualquier decisión de inversión.

Conozco la situación que motiva el proyecto: las municipalidades decidieron extender el Transmetro porque el servicio no registra robos, tiene paradas establecidas y sus pilotos son mejor reclutados. Para que esa decisión funcione hace falta un solo sistema que sepa en todo momento qué línea pasa por qué estaciones y en qué orden, cuántos accesos tiene cada estación y quién los cuida, qué bus pertenece a qué línea y en qué parqueo duerme, quién lo conduce y cuándo debe llegar otro bus a una estación saturada.

La propuesta que acompaña esta carta responde a esas necesidades con cuatro compromisos concretos:

1. **Reglas claras y verificables.** Las dieciocho condiciones del caso se convierten en restricciones de la base de datos y se comprueban con pruebas automáticas; no dependen de que alguien las recuerde.
2. **Una aplicación que ya funciona.** Entrego una aplicación web con base de datos PostgreSQL, reportes de estaciones, líneas y buses, y su código abierto en un repositorio.
3. **Costos transparentes.** La propuesta monetaria detalla la inversión, los gastos anuales, el retorno esperado y los escenarios en los que el proyecto deja de ser rentable.
4. **Entrega gradual y medible.** El trabajo se divide en iteraciones de dos semanas, con criterios de aceptación que cada municipalidad puede verificar.

Quedo a su disposición para presentar la propuesta, resolver dudas y ajustar el alcance a las prioridades de cada municipalidad. Agradezco su atención y la oportunidad de participar.

Atentamente,

William Alejandro Morales Mendez
Consultor responsable
Facultad de Ingeniería en Sistemas, Universidad

de Guatemala
Correo: william700morales@gmail.com
Teléfono: 5987-1377

1.2 Resumen de la propuesta

La propuesta cubre el ciclo completo de un proyecto de software para el Transmetro: entender el problema, definir los requerimientos, estimar costos, diseñar, construir, probar y entregar. Su resultado principal es un sistema compuesto por una base de datos relacional con las reglas del servicio, una aplicación web para planificadores y operadores de estación, y un conjunto de reportes que muestran las consultas SQL utilizadas. La inversión inicial estimada para el corredor piloto de la Línea 12 es de Q 700,500.00, con gastos operativos de Q 120,000.00 al año.

> **En una frase**
>
> Un solo sistema donde cada línea, estación, acceso, bus, parqueo, piloto, guardia y pago está registrado, y donde las reglas del servicio las hace cumplir la base de datos y no la memoria de las personas.

1.3 Cumplimiento de los puntos solicitados

La Tabla 1 relaciona cada punto solicitado en el enunciado con el capítulo donde se atiende y con la evidencia que lo respalda. Los enlaces a los repositorios y a la aplicación publicada están en el Apéndice D.

Tabla 1

Puntos solicitados y dónde se atienden

N.º	Punto solicitado	Dónde se atiende	Evidencia
1	Carta de presentación personal	Capítulo I, sección 1.1	Carta firmada por el consultor
2	Propuesta monetaria de la solución	Capítulo VIII	Presupuesto de inversión, gastos operativos, condiciones de pago
3	Análisis de requerimientos	Capítulo VII	18 reglas del caso, 20 RF y 8 RNF, matriz de roles
4	Antecedentes del problema	Capítulo IV	Historia del Transmetro y origen del proyecto
5	Definición de la situación actual	Capítulo V	Descripción por área y supuestos de levantamiento
6	Deficiencias de los controles actuales	Capítulo VI	Tabla de deficiencias, riesgos y solución
7	Estudios de factibilidad	Capítulo IX	Técnica, operativa, económica, legal y ambiental
8	Planificación (diagrama de Gantt)	Capítulo X	Diagrama de Gantt de 19 semanas
9	Modelo entidad-relación	Capítulo XI	Cinco diagramas ER y diccionario de datos
10	Script de creación de las estructuras	Capítulo XI y Apéndice B	Archivos 01 a 05 de la carpeta database
11	Diagramas UML completos	Capítulo XII	Casos de uso, componentes, secuencia, actividades, clases, objetos
12	Aplicación web funcional en la nube	Capítulo XIV	Aplicación publicada en GitHub Pages
13	Base de datos a elección	Capítulo XI y XIV	PostgreSQL 16 y 17 (PGlite)
14	Metodología de desarrollo a elección	Capítulo X	Scrum adaptado con iteraciones de dos semanas

N.º	Punto solicitado	Dónde se atiende	Evidencia
15	Reportes de estaciones, líneas y buses	Capítulo XV	17 reportes con su consulta SQL
16	Carta de aceptación del proyecto	Capítulo XX	Modelo de carta con condiciones verificables
17	Métricas de software	Capítulo XVII	Métricas medidas sobre el código y la base de datos
18	Plan de pruebas	Capítulo XVIII	Estrategia, matriz de casos y 36 pruebas ejecutadas
19	Estándares de seguridad	Capítulo XVI	OWASP, ISO/IEC 27001, roles y auditoría en la base de datos
20	Criterios de aceptación	Capítulo XIX	Criterios funcionales y técnicos con trazabilidad
+	Conectividad entre estaciones y métodos de pago	Capítulo XIII	Análisis de opciones y decisión (puntos 17 y 18 del caso)

Nota. La numeración de los puntos sigue la del enunciado; el enunciado repite el número 18, por lo que la seguridad se numera como punto 19.

Capítulo II. Introducción

2.1 Contexto y justificación

El Transmetro constituye la columna vertebral de la movilidad urbana en la Región Metropolitana de Guatemala. Desde su puesta en marcha, la red ha absorbido un crecimiento sostenido en el volumen diario de pasajeros y conecta polos urbanos de alta densidad como Centra Sur, El Trébol, Plaza Barrios y Centra Norte. Ese incremento de la demanda superó, con el tiempo, la capacidad de los esquemas tradicionales de programación estática y supervisión manual, pensados para volúmenes de operación considerablemente menores.

La ausencia de un sistema tecnológico integrado produce distorsiones operativas recurrentes. El inventario de unidades en los predios no se sigue de forma automática, lo que retrasa los despachos y dificulta equilibrar el kilometraje acumulado entre los autobuses. La variabilidad del tráfico en los tramos mixtos genera el agrupamiento de unidades conocido como bus bunching. La saturación de las estaciones compromete la seguridad física de los peatones en las plataformas, y los mecanismos de validación y control en los molinetes presentan brechas que facilitan la evasión del pago. A ello se suma que las situaciones de emergencia dependen de comunicaciones poco estructuradas entre el operador, el andén y el centro de mando.

Frente a este panorama resulta indispensable contar con un sistema computarizado integral que unifique la telemetría vehicular, el cobro electrónico sin contacto, la interacción con los operadores de autobús y la gestión centralizada de la seguridad ciudadana. La justificación del trabajo es, por tanto, triple: operativa, porque permite ordenar el despacho y regular los intervalos; financiera, porque recupera ingresos hoy perdidos por evasión y reduce el consumo de combustible; y social, porque mejora la información que recibe el pasajero y acorta el tiempo de respuesta ante incidentes.

El caso que origina esta propuesta añade una razón más. Debido a la delincuencia que sufren muchos municipios de la capital, las municipalidades optaron por llevar el Transmetro a toda la capital y a los municipios aledaños, dado que el servicio no presenta robos, tiene paradas establecidas y sus pilotos son mejor reclutados. Esa expansión multiplica la información que hay que controlar y obliga a que varias municipalidades compartan un mismo modelo de datos.

2.2 Planteamiento del problema

La operación actual del sistema se apoya en la observación humana y en registros parciales. Sin un registro automático de entradas y salidas en los predios, el despachador no dispone de una visión confiable de las unidades disponibles; sin medición de intervalos entre autobuses, el agrupamiento solo se corrige cuando ya es visible en la vía; y sin conciliación entre las rotaciones físicas del molinete y los cobros validados, la evasión no deja una huella auditable. Estos problemas no son independientes. Un despacho irregular favorece el bunching, el bunching produce estaciones saturadas y largas esperas, y las estaciones saturadas incentivan el salto de molinetes.

De lo anterior se deriva la pregunta que orienta la investigación: ¿qué arquitectura de software y qué conjunto de requerimientos permiten unificar el control de flota, la recaudación y la seguridad del Transmetro, cumpliendo una latencia de cobro inferior a 300 milisegundos y una disponibilidad del 99.98 %, y resultando económicamente viable a cinco años?

Para el caso concreto de las municipalidades, la pregunta se concreta así: ¿cómo registrar de forma ordenada las líneas, estaciones, accesos, buses, parqueos, pilotos y guardias, de modo que las reglas de operación (flota por línea, vigilancia por acceso, alertas de capacidad y esperas adicionales) se cumplan

siempre y puedan auditarse?

2.3 Objetivos

Objetivo general

Diseñar, construir y documentar el sistema de control de líneas, estaciones, flota y recaudación del Transmetro, con una base de datos relacional que aplique las reglas del servicio y una aplicación web que muestre los reportes de estaciones, líneas y buses asignados.

Objetivos específicos

- Traducir las dieciocho condiciones del caso en requerimientos verificables y en restricciones de la base de datos.
- Modelar los datos con un diagrama entidad-relación y escribir el script que crea las estructuras en PostgreSQL.
- Construir una aplicación web con catálogo de líneas y flota, operación de estación, reportes con su consulta SQL y consola de la base de datos.
- Estimar la inversión, los gastos y el retorno del proyecto y examinar su factibilidad técnica, operativa, económica, legal y ambiental.
- Definir cómo se conectarán las computadoras de las estaciones y qué métodos de pago acepta el sistema.
- Establecer las métricas, el plan de pruebas y los criterios de aceptación con los que se entregará el sistema.

2.4 Alcance y delimitación

La propuesta tiene dos niveles. El **nivel de red** abarca el catálogo y las reglas del caso: municipalidades, líneas, estaciones, accesos, tramos y distancias, parqueos, buses, pilotos, guardias, computadoras y operadores de estación, enlaces de red, tarjetas, métodos de pago, recorridos, visitas, lecturas de pasajeros y alertas. Este nivel está implementado y probado sobre una base de datos real. El **nivel ampliado** es el del sistema unificado descrito en el documento original, con conteo en pórticos RFID, validación en molinetes, botón de pánico y evaluación de conducta; este nivel se conserva como especificación y como simulación en el Centro de Control C5 de la aplicación.

La propuesta económica se calcula para el corredor piloto de la Línea 12, con 15 estaciones y 40 autobuses articulados, de los cuales 27 se asignan a la línea y 13 quedan como reserva sin línea. Quedan fuera del alcance la adquisición de material rodante, la obra civil de las estaciones, la política tarifaria y la integración con sistemas bancarios más allá de la recarga.

2.5 Limitaciones

- Los datos de la base (nombres, documentos, teléfonos, distancias y pasajeros) son ficticios y reproducibles; las estaciones llevan nombres referenciales y las distancias son ilustrativas. Deben sustituirse por datos reales de cada municipalidad.
- La aplicación publicada ejecuta PostgreSQL dentro del navegador mediante PGlite (PostgreSQL compilado a WebAssembly) y guarda los datos en el propio navegador. Es una demostración funcional; para operar con varios usuarios a la vez hace falta un servidor PostgreSQL gestionado, que usa los mismos scripts.

- Los módulos del Centro de Control C5 (flota, molinete, seguridad, pasajeros y operadores) siguen siendo simulaciones en memoria y no están conectados a la base de datos.
- Las cifras económicas se basan en estimaciones de costos unitarios y en supuestos de ahorro que deben contrastarse con datos operativos reales.
- Algunas condiciones del caso admiten más de una lectura. Las interpretaciones adoptadas se listan en la sección 7.2 y pueden cambiarse modificando un parámetro.

2.6 Organización del documento

Tras esta introducción, el Capítulo III presenta el marco teórico y los Capítulos IV, V y VI describen los antecedentes, la situación actual y las deficiencias de los controles. El Capítulo VII analiza los requerimientos; el VIII contiene la propuesta monetaria y el IX los estudios de factibilidad. El Capítulo X describe la metodología y la planificación. Los Capítulos XI y XII documentan el diseño: base de datos y UML. El XIII resuelve la conectividad de las estaciones y los métodos de pago. Los Capítulos XIV y XV presentan la aplicación web y los reportes, el XVI la seguridad, el XVII las métricas, el XVIII las pruebas, el XIX los criterios de aceptación, el XX el modelo de carta de aceptación y el XXI las conclusiones.

Capítulo III. Marco teórico

3.1 Sistemas de transporte masivo y control operativo

Los sistemas de autobús de tránsito rápido operan sobre carriles segregados con estaciones de abordaje en plataforma y cobro previo a la entrada, de modo que el tiempo de permanencia en parada depende principalmente del flujo de pasajeros y no de la transacción de pago a bordo. Esta configuración hace que el control operativo se concentre en tres elementos: la regulación de los intervalos entre unidades, la gestión del flujo en estaciones y la integridad del cobro en los puntos de acceso. Un sistema de control moderno obtiene información de cada uno de ellos de manera continua y la consolida en un centro de mando donde se toman las decisiones de regulación.

3.2 El fenómeno del bus bunching

Newell y Potts (1964) describieron por primera vez la inestabilidad inherente a las rutas de autobús con intervalos cortos: una unidad que se retrasa encuentra más pasajeros esperando, lo que prolonga su tiempo de ascenso y la retrasa todavía más, mientras que la unidad que la sigue encuentra menos pasajeros, avanza más rápido y termina alcanzándola. El resultado es la formación de convoyes que reducen la frecuencia efectiva del servicio y concentran la carga de pasajeros en la unidad rezagada.

Daganzo (2009) mostró que las estrategias basadas en horarios fijos resultan frágiles cuando la frecuencia es alta y propuso controlar el intervalo entre unidades consecutivas (headway) en lugar de la hora de paso. En ese enfoque se inspira el requerimiento de regulación del sistema propuesto: se monitorea la separación entre autobuses del mismo sentido y, cuando esta cae por debajo de un umbral crítico, se instruye al operador de la unidad rezagada para que modifique su velocidad o permanezca retenido un tiempo breve. En este trabajo el umbral se fija en el 6 % de la longitud de la ruta, equivalente a aproximadamente 1.1 km sobre un recorrido de 18.4 km.

3.3 Pago electrónico sin contacto

Las tarjetas de acceso empleadas en el transporte público se basan en el estándar ISO/IEC 14443, que define el funcionamiento de las tarjetas de proximidad a 13.56 MHz y constituye la base de la tecnología NFC (International Organization for Standardization [ISO], 2018). La seguridad de la transacción no reside únicamente en la lectura del identificador de la tarjeta, sino en la verificación criptográfica de su contenido mediante un módulo de acceso seguro (SAM, por sus siglas en inglés) alojado en el lector. Este módulo custodia las llaves de autenticación y evita que el saldo almacenado o el perfil tarifario puedan ser duplicados o alterados fuera del sistema.

Para que el pago no interrumpa el flujo de pasajeros, la validación debe completarse en una fracción de segundo. El sistema propuesto adopta como meta una latencia total inferior a 300 milisegundos entre la presentación de la tarjeta y la liberación del molinete, un valor que obliga a resolver las consultas de saldo y perfil con almacenamiento en memoria de baja latencia y a mantener una copia local de datos en el lector para operar cuando el enlace con el servidor central no esté disponible.

3.4 Calidad del software y modelo Goal-Question-Metric

La norma ISO/IEC 25010 define un modelo de calidad del producto de software compuesto por ocho características: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad (ISO, 2011). El modelo ofrece un vocabulario común para formular los criterios de aceptación y para seleccionar las métricas que permiten evaluarlos de forma objetiva.

El método Goal-Question-Metric (GQM) complementa esa clasificación al exigir que toda métrica responda a una pregunta y que toda pregunta se derive de un objetivo explícito (Basili et al., 1994). De este modo se evita acumular indicadores sin propósito. La complejidad ciclomática de McCabe (1976), que cuenta los caminos linealmente independientes de un método, es una de las métricas de mantenibilidad más difundidas, y su umbral recomendado de 10 por unidad de código se adopta en este proyecto. Las prácticas de ingeniería de requerimientos, diseño y pruebas siguen los lineamientos de Sommerville (2016) y de Pressman y Maxim (2021).

3.5 Bases de datos relacionales, integridad y disparadores

El modelo relacional organiza los datos en tablas relacionadas por llaves y permite declarar reglas de integridad en el propio esquema: llaves primarias, llaves foráneas, valores únicos, restricciones de verificación y obligatoriedad (Elmasri y Navathe, 2016). Cuando una regla no puede expresarse con una restricción simple, por ejemplo «una línea activa necesita entre N y 2N buses», se utilizan disparadores, que son funciones que el motor ejecuta automáticamente ante una inserción, modificación o borrado (PostgreSQL Global Development Group, 2024). PostgreSQL permite además que esos disparadores se evalúen al final de la transacción, lo que deja mover varios buses a la vez y rechazar el conjunto si al terminar la regla no se cumple.

Aplicar las reglas en la base de datos, y no solo en la interfaz, tiene una ventaja práctica: cualquier programa que escriba en ella, ahora o en el futuro, queda sujeto a las mismas condiciones. Es el enfoque que sigue este proyecto.

3.6 Redes privadas virtuales entre sedes

Una red privada virtual (VPN) crea un túnel cifrado sobre una red pública, de modo que las computadoras de varias sedes se comunican como si estuvieran en la misma red local. IPsec es el conjunto de protocolos estándar para ello (Kent y Seo, 2005). WireGuard es una alternativa más reciente, de configuración sencilla y código reducido, basada en criptografía moderna (Donenfeld, 2017). Ambas opciones se evalúan en el Capítulo XIII para conectar las computadoras de las estaciones con el centro de control, cuyo método aún no se había definido.

Capítulo IV. Antecedentes del problema

4.1 El Transmetro de la Ciudad de Guatemala

El Transmetro es el sistema de autobuses de tránsito rápido de la Municipalidad de Guatemala. Su primera línea, el Eje Sur, comenzó a operar el 3 de febrero de 2007 (Wikipedia, s. f.). Al cumplir quince años de servicio, en febrero de 2022, la Municipalidad informó que el sistema funcionaba en la Ciudad de Guatemala y en el área norte de Villa Nueva y que había acumulado más de 1,294 millones de viajes con 207 unidades y 109 estaciones (Emisoras Unidas, 2022a).

Desde 2021 el sistema incorporó la Tarjeta Ciudadana, una tarjeta prepagada que se recarga en puntos de la red de estaciones (Emisoras Unidas, 2021). Las autoridades han mantenido una tarifa de Q 1.00 por viaje, según declaraciones de marzo de 2022 (Emisoras Unidas, 2022b). La tarifa y los descuentos por perfil (estudiante, adulto mayor y persona con discapacidad) se manejan en este documento como parámetros configurables, pues son decisiones de política pública.

4.2 Origen de la propuesta

El caso que motiva esta propuesta parte de una decisión de las municipalidades: introducir el Transmetro en toda la capital y en los municipios aledaños como medida alternativa frente a la delincuencia que sufren muchos de ellos. Los argumentos que se dieron para escoger este servicio son que no hay un índice de robos, que las paradas están establecidas y que los pilotos son mejor reclutados. Con la expansión, cada municipalidad pasa a ser titular de líneas y estaciones, y el sistema necesita saber a quién pertenece cada una.

4.3 Evolución de la necesidad de control

Tabla 2

Etapas de crecimiento del Transmetro y necesidades de información

Etapa	Qué ocurre	Qué información hace falta
Inicio (2007)	Una primera línea con pocas estaciones y una flota reducida.	Hojas de cálculo y registros por estación son suficientes.
Crecimiento (2010 en adelante)	Se añaden líneas y estaciones de intercambio; la misma estación sirve a varias líneas.	Hay que saber el orden de cada línea y qué estaciones comparte.
Pago electrónico (2021)	La Tarjeta Ciudadana prepagada y las recargas en estaciones.	Saldos, tipos de tarjeta, métodos de pago y conciliación.
Expansión metropolitana (caso actual)	Varias municipalidades son titulares de líneas y estaciones.	Quién es titular, cuántos buses por línea, quién vigila cada acceso y cómo se comunican las estaciones.

Nota. Elaboración propia con base en Wikipedia (s. f.), Emisoras Unidas (2021, 2022a) y el enunciado del caso.

Cada etapa añadió información que los controles anteriores no contemplaban. La Tabla 2 muestra que la necesidad actual no es solo de más datos, sino de datos relacionados entre sí: una estación pertenece a varias líneas, una línea necesita una flota proporcional a sus estaciones y un acceso necesita vigilancia. Esas

relaciones son las que una hoja de cálculo no puede hacer cumplir.

Capítulo V. Definición de la situación actual

> **Cómo se obtuvo esta descripción.** No se realizó una auditoría en sitio. La situación actual se describe a partir del enunciado, de la documentación pública y de la operación típica de un sistema de este tipo. Cada afirmación marcada como supuesto se confirmará en la fase de levantamiento con las municipalidades (Capítulo X).

5.1 Descripción de la operación actual

Hoy la información de cada línea se administra en la municipalidad que la opera. Para cada área de trabajo se describe cómo se maneja la información y qué consecuencia tiene, como muestra la Tabla 3.

Tabla 3

Situación actual por área de trabajo

Área	Cómo se maneja hoy (supuesto de levantamiento)	Consecuencia
Líneas y estaciones	Listas separadas por municipalidad, en hojas de cálculo o documentos. El orden de las estaciones y las distancias están en planos y archivos distintos.	Cuesta saber qué estaciones comparten dos líneas y cuánto mide cada línea.
Accesos y seguridad	Los guardias se asignan por estación y no siempre por acceso. Los turnos se anotan en papel o mensajes.	Un acceso puede quedar sin vigilancia sin que nadie lo note.
Flota	El inventario de buses y su asignación a líneas se lleva en archivos. El parqueo de cada bus se registra de forma informal.	No hay forma automática de saber si una línea tiene los buses que necesita.
Pilotos	Expedientes en papel con estudios, residencia y teléfonos.	Consultar un dato implica buscar un archivo; los datos de contacto se desactualizan.
Estaciones y computadoras	Cada estación decide cómo reporta. No hay una PC con acceso común ni un método de conexión definido.	La información llega tarde y con formatos distintos.
Aforo y alertas	El personal de la estación avisa por radio o teléfono cuando hay demasiada gente.	No queda registro ni se mide cuánto tarda en llegar el refuerzo.
Pagos	Tarjeta Ciudadana prepagada y efectivo en taquilla.	La conciliación entre recargas, cobros y métodos de pago se hace manualmente.

Nota. Los contenidos de la segunda columna son supuestos de levantamiento que se validarán con cada municipalidad.

5.2 Actores y flujo de información

Intervienen siete tipos de actores: las municipalidades, que son titulares de líneas y estaciones; los planificadores, que definen líneas y flota; los operadores de estación, que atienden la PC de cada estación; los guardias de seguridad; los pilotos; los cajeros de taquilla y los pasajeros. La información fluye hoy en sentido ascendente y de forma manual: los operadores informan a los planificadores, que informan a las municipalidades.

5.3 Problemas que debe resolver el sistema

- Conocer el orden de las estaciones de cada línea y las visitas que realiza cada unidad durante su recorrido.

- Saber a qué línea pertenece cada bus, en qué parqueo está y qué piloto lo conduce.
- Garantizar que cada acceso tenga vigilancia y que cada línea tenga la flota que necesita.
- Detectar a tiempo las estaciones saturadas y los buses que circulan casi vacíos.
- Conectar las computadoras de las estaciones con un centro de control y registrar los pagos.

Capítulo VI. Deficiencias de los controles actuales

Un control es una medida que evita o detecta un error. La Tabla 4 compara los controles que se supone existen hoy con las deficiencias que presentan, el riesgo que generan y la solución que propone el sistema. Cada deficiencia recibe un código para poder rastrearla hasta los requerimientos y las pruebas.

Tabla 4

Deficiencias de los controles actuales y solución propuesta

Cód.	Control actual	Deficiencia	Riesgo	Solución del sistema
D-01	Orden de estaciones en planos o listas	No está ligado a las visitas de cada bus	Itinerarios inconsistentes	Tabla de orden por línea y visitas generadas desde ella
D-02	Lista de estaciones por municipalidad	Una estación compartida se duplica o se olvida	Datos contradictorios entre municipalidades	Estación única con pertenencia a varias líneas (regla 2)
D-03	Registro de guardias por estación	No se verifica un guardia por acceso	Accesos sin vigilancia	Restricción que exige al menos un guardia vigente por acceso (regla 10)
D-04	Inventario de buses en archivos	No se valida la flota mínima ni máxima por línea	Líneas sin buses suficientes o con exceso	Validación de entre N y 2N buses al confirmar cambios (regla 5)
D-05	Registro informal de parqueos	Un bus puede quedar sin parqueo o en uno lleno	Pérdida de control de la unidad	Parqueo obligatorio, capacidad validada e historial de cambios (reglas 6 y 7)
D-06	Expedientes de pilotos en papel	Datos dispersos y desactualizados	Contacto imposible en una emergencia	Ficha digital con estudios, residencia y comunicación (regla 9)
D-07	Aviso por radio o teléfono	No hay umbral ni registro de la saturación	Estaciones saturadas sin refuerzo a tiempo	Alerta automática al llegar al 150 % de la capacidad de un bus (regla 14)
D-08	Salida del bus a criterio del piloto	Buses casi vacíos recorren la línea sin espera	Frecuencia mal aprovechada	Espera adicional de 5 minutos si la ocupación es menor al 25 % (regla 15)
D-09	Distancias en documentos separados	No se puede sumar la distancia de una línea	Planificación sin base	Tabla de tramos y distancias calculadas por consulta (reglas 12 y 13)
D-10	Conexión de cada estación por su cuenta	No hay método común ni enlace de respaldo	Información tardía; estaciones aisladas	VPN con enlace principal y respaldo móvil (Capítulo XIII)
D-11	Conciliación manual de pagos	No se separan recargas, cobros ni métodos de pago	Pérdida de ingresos y errores de saldo	Tabla de pagos con validación de saldo automática (regla 18)
D-12	Cambios sin trazabilidad	No se sabe quién cambió qué	Imposible auditar	Bitácora que no se puede modificar ni borrar (Capítulo XVI)

Nota. Las reglas se numeran según el enunciado del caso. Los controles actuales son supuestos de levantamiento.

De las doce deficiencias, nueve se corrigen con reglas que aplica la base de datos, dos con el diseño de red y una con la bitácora de auditoría. Esa proporción refleja una idea central del diseño: lo que es una regla del servicio debe vivir en el lugar donde ninguna persona pueda saltársela por descuido.

Capítulo VII. Análisis de requerimientos

El análisis de requerimientos se hizo en dos capas. La primera traduce las dieciocho condiciones del caso a elementos del modelo de datos y a reglas verificables. La segunda conserva el catálogo ampliado del sistema unificado (veinte requerimientos funcionales y ocho no funcionales) que se redactó en la versión anterior de este documento y que describe el sistema completo de control operativo, recaudación y seguridad.

7.1 Las dieciocho condiciones del caso

La Tabla 5 indica, para cada condición del enunciado, el elemento del sistema que la hace cumplir. Esa correspondencia es la base de las pruebas del Capítulo XVIII.

Tabla 5

Condiciones del caso (RN) y su implementación en el sistema

RN	Condición del enunciado (resumida)	Cómo la hace cumplir el sistema
1	Una línea es una serie de estaciones en orden; interesa el orden de la ruta y las visitas de cada unidad.	`linea_estacion.orden` con orden único por línea; `viaje` y `visita`; función `fn_generar_visitas` que crea las visitas en el orden de la línea (ida) o inverso (vuelta).
2	Toda estación pertenece al menos a una línea y puede pertenecer a varias.	Relación muchos a muchos en `linea_estacion`; disparador diferido `ct_estacion_con_linea` que rechaza una estación sin línea.
3	Una estación tiene varios accesos; un acceso pertenece a una sola estación.	`acceso.id_estacion` obligatorio; numeración única por estación.
4	Un bus no puede estar en más de una línea, pero puede no estar en ninguna.	Una sola columna opcional `bus.id_linea`; el viaje exige que el bus sea de la línea.
5	Cada línea tiene como mínimo tantos buses como estaciones y como máximo el doble.	Función `fn_validar_flota_linea` y disparadores diferidos sobre `bus`, `linea` y `linea_estacion`. Mínimo exigido a las líneas activas.
6	Algunas estaciones tienen parqueos y cada bus tiene un parqueo.	`parqueo.id_estacion` opcional; `bus.id_parqueo` obligatorio; capacidad del parqueo validada.
7	Un bus puede cambiar de parqueo, pero no quedar sin él.	Restricción `NOT NULL` y tabla `bus_parqueo_historial` llenada por disparador.
8	Interesa conocer todos los accesos de cada línea.	Vista `vw_accesos_por_linea` y reporte R06.
9	De los pilotos: historial educativo, residencia y comunicación.	Tablas `piloto` y `piloto_estudio`; reportes R10 y R11.
10	Cada acceso tiene por lo menos un guardia de seguridad.	`asignacion_guardia` y disparador diferido `ct_acceso_con_guardia` (también al retirar al último guardia).
11	Cada línea y cada estación pertenecen a una municipalidad.	`id_municipalidad` obligatorio en `linea` y `estacion`.
12	Se necesita la distancia de una estación a otra.	Tabla `tramo` con una distancia por par de estaciones contiguas; funciones `fn_distancia_tramo` y `fn_distancia_en_linea`.
13	Se necesita la distancia de una línea.	Función `fn_distancia_linea` (suma de tramos) y reportes R01 y R02.

RN	Condición del enunciado (resumida)	Cómo la hace cumplir el sistema
14	Si una estación tiene al menos 50 % más de la capacidad, se genera una alerta para que llegue otro bus.	`lectura_estacion` y disparador que crea una `alerta` al llegar al 150 % de la capacidad del bus; función `fn_atender_alerta`.
15	Si un bus no llena el 25 % de su capacidad espera 5 minutos más en cada estación.	Disparador sobre `visita` que calcula la ocupación y la espera adicional; reporte R14.
16	Una PC de trabajo por estación con un operador que la controle.	`computadora` con estación única y `operador` obligatorio por computadora; reporte R15.
17	Aún no se decide cómo conectar las máquinas.	Análisis de opciones del Capítulo XIII; `enlace_red` registra el enlace principal y el de respaldo de cada estación.
18	Métodos de pago y tarjeta de acceso.	`tipo_tarjeta`, `tarjeta_acceso`, `metodo_pago` y `pago`, con validación de saldo por disparador.

Nota. RN = condición del enunciado del caso. El enunciado recoge las condiciones numeradas del 1 al 18.

7.2 Interpretaciones y supuestos

Algunas condiciones admiten más de una lectura. Las que siguen son las que se adoptaron; todas pueden cambiarse sin modificar la estructura, porque se guardan como parámetros o están aisladas en una función.

- **Regla 14.** «Una estación tiene por lo menos un 50 % más de su capacidad» se interpreta como: esperan al menos 150 % de la capacidad del bus que atiende la línea (160 pasajeros en un articulado, 240 en la estación; 80 en un bus convencional, 120). Si el cliente prefiere medir contra la capacidad del andén, basta cambiar la consulta del disparador. El porcentaje es el parámetro `ALERTA_AFORO_PCT`.
- **Regla 15.** «No llena el 25 %» significa estrictamente menos del 25 %: con 40 de 160 pasajeros (exactamente 25 %) el bus no espera más. La parada normal es de 1 minuto (`ESPERA_BASE_MIN`) y el tiempo extra de 5 minutos (`ESPERA_EXTRA_MIN`).
- **Regla 5.** El máximo se exige siempre; el mínimo, solo a las líneas activas, porque una línea en planificación necesita ir recibiendo buses antes de abrir. La verificación se hace al confirmar la transacción para poder mover varios buses a la vez.
- **Regla 11.** Una línea puede atravesar varios municipios; se registra la municipalidad titular, que es la que la administra.
- **Regla 12.** Se guarda la distancia entre estaciones contiguas. La distancia entre dos estaciones de una misma línea es la suma de los tramos intermedios; entre líneas distintas se compone a partir de ellos.
- **Regla 16.** Se asume una PC por estación y uno o más operadores por PC según el turno. Los operadores y guardias se tratan como personal municipal existente; su salario no entra en el flujo de caja del proyecto.
- **Regla 18.** Se aceptan efectivo en taquilla, tarjeta de acceso con saldo prepagado, tarjeta de débito o crédito, billetera móvil y transferencia bancaria. Las tarifas por perfil son parámetros de política pública (Q 1.00, Q 0.50, Q 0.25 y Q 0.00).

7.3 Actores y matriz de roles y permisos

El sistema reconoce cinco roles con ámbitos de interacción y permisos diferenciados, de acuerdo con el principio de mínimo privilegio. La Tabla 6 presenta la matriz de control de acceso basado en roles (RBAC) que se implementa en la plataforma.

Tabla 6

Matriz de roles y permisos del sistema Transmetro

Rol	Ámbito de interacción	Permisos otorgados
Pasajero	Aplicación móvil y monitores de estación	Consulta de saldo, recarga digital y visualización del tiempo estimado de llegada y la ocupación de los autobuses.
Operador (piloto)	Consola a bordo (CAB/HMI)	Inicio de turno, recepción de instrucciones de regulación y activación del botón de pánico.
Despachador de patio	Consola de despacho y predios	Asignación de rutas, autorización de salidas de los predios y monitoreo de pórticos RFID.
Supervisor de seguridad	Centro de Control C5	Gestión de alertas SOS, monitoreo de cámaras y administración de listas negras.
Auditor financiero	Portal de fiscalización y cobro	Generación de reportes de conciliación tarifaria, auditoría de evasión e ingresos.

Nota. Los niveles de acceso se definen según las funciones operativas de cada actor.

Para el nivel de red se añaden cuatro roles de la aplicación, que corresponden a tres roles de la base de datos con privilegios mínimos (ver Capítulo XVI).

Tabla 7

Roles del sistema de red y rol de base de datos asociado

Rol de la aplicación	Qué hace	Rol en PostgreSQL
Planificador municipal	Crea y activa líneas, asigna buses, cambia parqueos y administra pilotos y guardias.	`tm_admin`
Operador de estación	Registra pasajeros en espera, atiende alertas, registra visitas y consulta reportes.	`tm_operador`
Cajero de taquilla	Recarga tarjetas y cobra pasajes.	`tm_operador`
Auditor	Consulta reportes y la bitácora; no ve el DPI de las personas.	`tm_consulta`

Nota. En esta versión la aplicación no pide inicio de sesión (RF-20 queda pendiente); los roles existen en la base de datos y se prueban con las pruebas T31 a T33.

7.4 Requerimientos funcionales del sistema unificado

Los requerimientos funcionales RF-01 a RF-20 describen el sistema completo: control de flota en patios, pasajeros y estaciones, recaudación, operadores y seguridad. El catálogo se conserva de la versión anterior del documento. Su cobertura actual en la aplicación se detalla en el Capítulo XIV.

Módulo de conteo, organización y control de flota

Tabla 8

Requerimientos funcionales del módulo de flota

ID	Nombre	Descripción
RF-01	Conteo en pórticos de terminales	El sistema registrará de forma automática la entrada, la salida y el tiempo de permanencia de los autobuses articulados en los predios y terminales mediante lectores RFID y cámaras de reconocimiento óptico de caracteres (OCR).
RF-02	Secuenciación dinámica de despachos	El sistema calculará la cola ordenada de salida de las unidades, priorizando el equilibrio del kilometraje acumulado, el historial de mantenimiento preventivo y la demanda proyectada por franja horaria.
RF-03	Regulación de agrupamiento (bus bunching)	El sistema monitoreará la separación entre autobuses del mismo sentido. Si esta disminuye por debajo del umbral crítico (6 % de la ruta, cercano a 1.1 km), enviará una instrucción de regulación de velocidad a la consola del piloto.

Nota. RF = requerimiento funcional.

Módulo de pasajeros, aforo y estaciones

Tabla 9

Requerimientos funcionales del módulo de pasajeros

ID	Nombre	Descripción
RF-04	Conteo de aforo interno por sensores ópticos	Cada autobús contará con sensores ópticos en las puertas para contabilizar las subidas y bajadas de usuarios y mantener la cifra exacta de ocupación a bordo en tiempo real.
RF-05	Medición de densidad en andenes	El sistema analizará, mediante visión computacional en el borde, el nivel de saturación de pasajeros en las plataformas de abordaje y emitirá alertas cuando la densidad supere el 80 % de la capacidad de la estación.
RF-06	Información en tiempo real al pasajero	El sistema calculará y desplegará en los monitores de estación y en la aplicación móvil los minutos estimados de llegada (ETA) del próximo autobús y su nivel de ocupación.
RF-07	Monitoreo de accesibilidad universal	El sistema registrará el despliegue de rampas para sillas de ruedas e integrará alertas sonoras y visuales automáticas para personas con discapacidad auditiva o visual.

Nota. ETA = tiempo estimado de llegada.

Módulo de recaudación y tarjeta de acceso

Tabla 10

Requerimientos funcionales del módulo de recaudación

ID	Nombre	Descripción
RF-08	Validación transaccional sin contacto	El validador del molinete leerá la tarjeta de acceso (NFC o Tarjeta Ciudadana), verificará el saldo, debitará la tarifa correspondiente y emitirá la señal de apertura del torniquete electromecánico en un tiempo no mayor a 300 milisegundos.
RF-09	Gestión de perfiles y tarifas preferenciales	El sistema aplicará descuentos o tarifa cero según el perfil grabado criptográficamente en la tarjeta: regular, Q 1.00; estudiante, Q 0.50; adulto mayor, Q 0.25; persona con discapacidad, Q 0.00.
RF-10	Recarga presencial y digital omnicanal	El sistema gestionará la acreditación de saldo en la tarjeta mediante taquillas automáticas en las estaciones, bancarización en línea y billeteras digitales.
RF-11	Conciliación y auditoría de evasión tarifaria	El sistema cruzará en tiempo real las rotaciones físicas del brazo del molinete contra los cobros validados. Ante giros no autorizados registrará un evento de evasión y notificará al personal de seguridad.
RF-12	Lista negra de tarjetas y control antifraude	El sistema sincronizará en menos de 60 segundos la lista negra de tarjetas reportadas por robo, clonación o patrones de uso anómalos.

Nota. Las tarifas son parámetros configurables del sistema.

Módulo de operadores y gestión de conductores

Tabla 11

Requerimientos funcionales del módulo de operadores

ID	Nombre	Descripción
RF-13	Inicio de turno con clave biométrica o RFID	El piloto deberá autenticarse en la consola a bordo mediante su credencial RFID o su huella dactilar antes de iniciar el desplazamiento de la unidad.
RF-14	Control de jornadas y descansos reglamentarios	El sistema contabilizará las horas continuas de conducción y bloqueará la asignación de nuevos trayectos si el operador excede las 4 horas consecutivas de servicio sin receso.
RF-15	Consola de navegación y telemetría HMI	La pantalla táctil de la cabina mostrará al conductor la desviación de horario, el límite de velocidad en el carril exclusivo, las alertas viales y los avisos del centro de mando.
RF-16	Evaluación de conducta al volante	El sistema registrará eventos de frenado brusco, aceleración violenta, giros cerrados y exceso de velocidad, y calculará una puntuación de desempeño (Driver Score).

Nota. HMI = interfaz hombre-máquina.

Módulo de seguridad e integridad operativa

Tabla 12

Requerimientos funcionales del módulo de seguridad

ID	Nombre	Descripción
RF-17	Botonera de pánico silenciosa en unidad y andén	La cabina del autobús y las garitas de supervisión dispondrán de botones de emergencia que transmitirán señales prioritarias con coordenadas GPS al Centro de Monitoreo C5 y a las fuerzas de seguridad pública.
RF-18	Integración de cámaras CCTV con detección de eventos	El sistema procesará los flujos de video internos y de estación para identificar bultos desatendidos, invasión del carril exclusivo y altercados físicos.
RF-19	Seguridad de datos e integridad transaccional	Toda la comunicación entre los nodos de borde y la central se cifrará mediante TLS 1.3 con certificados X.509, y el almacenamiento se protegerá con AES-256. El seguimiento de incidentes se registrará en bitácora.
RF-20	Control de acceso basado en roles y doble factor	La plataforma restringirá las funciones administrativas según los perfiles de usuario y exigirá autenticación de doble factor para las modificaciones sensibles.

Nota. CCTV = circuito cerrado de televisión.

7.5 Requerimientos no funcionales

Los requerimientos no funcionales de la Tabla 13 establecen los umbrales cuantitativos que caracterizan el comportamiento del sistema y que, posteriormente, se convierten en criterios de aceptación verificables.

Tabla 13

Requerimientos no funcionales del sistema

ID	Nombre	Descripción
RNF-01	Latencia de cobro	El procesamiento de la tarjeta sin contacto, desde el impacto NFC hasta la liberación del molinete, debe ser inferior a 300 milisegundos.
RNF-02	Frecuencia de actualización GPS	La posición geográfica de cada autobús debe actualizarse en la central cada 1,000 milisegundos.
RNF-03	Disponibilidad del sistema central	El núcleo de servicios en la nube debe garantizar un tiempo de actividad del 99.98 % anual.
RNF-04	Operación fuera de línea	Los validadores de estación y las computadoras a bordo deben almacenar localmente hasta 50,000 transacciones o 24 horas de eventos durante caídas de red.
RNF-05	Capacidad transaccional concurrente	La arquitectura de servidor debe procesar al menos 2,500 validaciones de cobro por segundo durante las horas pico.
RNF-06	Protección criptográfica en tarjetas	Las tarjetas de acceso deben utilizar módulos de seguridad SAM y llaves criptográficas que impidan la duplicación o la alteración del saldo.
RNF-07	Protección de datos personales	Los algoritmos de visión computacional para aforo no almacenarán imágenes ni rasgos biométricos faciales de los usuarios.
RNF-08	Auditoría inalterable	La bitácora del sistema mantendrá registros firmados digitalmente, sin opción de modificación ni eliminación.

Nota. RNF = requerimiento no funcional.

El nivel de red añade cinco requerimientos no funcionales, que se verifican en los capítulos XVII y XVIII.

Tabla 14

Requerimientos no funcionales del nivel de red

ID	Nombre	Descripción	Verificación
RNF-09	Integridad en la base de datos	Las reglas del caso se aplican en el motor y no pueden saltarse desde la aplicación ni desde otra herramienta.	36 pruebas automáticas (T01 a T36)
RNF-10	Rendimiento de reportes	Cada reporte responde en menos de 2 segundos con los datos de demostración.	Mayor tiempo medido: 12.9 ms
RNF-11	Portabilidad	Los scripts SQL funcionan en PostgreSQL 16 y 17 sin cambios.	Ejecución en PostgreSQL 16 (Docker) y 17 (PGlite)
RNF-12	Accesibilidad	La aplicación se puede usar con teclado, con contraste suficiente y sin depender solo del color.	Revisión manual de la interfaz
RNF-13	Reproducibilidad	La base se reconstruye completa con cinco scripts y datos con semilla fija.	Carga completa en 3.4 s

Capítulo VIII. Propuesta monetaria de la solución

Esta propuesta cubre el corredor piloto de la Línea 12 (15 estaciones y 40 autobuses articulados, de los cuales 27 se asignan a la línea y 13 quedan como reserva sin línea) y deja preparada la ampliación al resto de la red. Los montos están en quetzales y son estimaciones que deben confirmarse con cotizaciones.

> **Cifras principales**
>
> Inversión inicial: Q 700,500.00 · Gastos operativos: Q 120,000.00 al año · Beneficio directo estimado: Q 310,000.00 al año · VPN a 5 años (10 %): Q 73,921.67 · TIR: 13.94 % · Recuperación: 3 años y 6 meses.

8.1 Presupuesto de inversión inicial y gastos operativos

La inversión inicial (CAPEX) del corredor piloto asciende a Q 700,500.00, distribuida entre equipamiento de estaciones, telemetría de las unidades, infraestructura central, desarrollo de software e instalación, como se detalla en la Tabla 15.

Tabla 15

Presupuesto de inversión inicial (CAPEX)

Rubro / categoría	Cantidad	Costo unitario (GTQ)	Subtotal (GTQ)
Kits de sensores de aforo y cámaras de visión para estaciones	15 estaciones	12,500.00	187,500.00
Nodos de procesamiento edge e interfaz para molinetes	15 estaciones	8,000.00	120,000.00
Unidades de telemetría GPS y OBD2 para autobuses	40 buses	3,200.00	128,000.00
Servidor central de mando y almacenamiento en rack	2 unidades	35,000.00	70,000.00
Desarrollo de software, algoritmo de despacho y pruebas	1 proyecto	150,000.00	150,000.00
Instalación física, cableado estructurado y mano de obra	1 global	45,000.00	45,000.00
Total inversión inicial (CAPEX)			700,500.00

Nota. Costos calculados para la implementación del corredor piloto de la Línea 12.

Los gastos operativos anuales (OPEX) ascienden a Q 120,000.00 y comprenden el mantenimiento del hardware en campo, las licencias y el soporte técnico especializado, y la conectividad celular redundante. La Tabla 16 desglosa estos conceptos.

Tabla 16

Gastos operativos anuales (OPEX)

Concepto de gasto operativo	Costo mensual (GTQ)	Costo anual (GTQ)
Mantenimiento preventivo y correctivo de hardware en campo	4,000.00	48,000.00
Licencias de software y soporte técnico especializado	3,500.00	42,000.00
Plan de datos y conectividad celular redundante (M2M)	2,500.00	30,000.00
Total costo operativo anual (OPEX)	10,000.00	120,000.00

Nota. Estimación de gastos recurrentes de operación y mantenimiento.

8.2 Desglose del rubro de software

El rubro de software y pruebas de Q 150,000.00 corresponde al trabajo de consultoría, diseño, construcción y pruebas descrito en este documento. La Tabla 17 lo desglosa por actividad.

Tabla 17

Desglose del rubro de desarrollo de software

Actividad	Qué incluye	Porcentaje	Monto (GTQ)
Análisis y diseño	Levantamiento, requerimientos, modelo ER, UML, arquitectura	20 %	30,000.00
Base de datos y reglas	Script SQL, disparadores, vistas, datos de carga	18 %	27,000.00
Aplicación web y reportes	Interfaz, catálogo, operación de estación, 17 reportes	28 %	42,000.00
Pruebas y calidad	Plan de pruebas, casos automáticos, métricas	16 %	24,000.00
Seguridad y auditoría	Roles, privilegios mínimos, bitácora, revisión OWASP	8 %	12,000.00
Documentación y capacitación	Manuales, material y entrenamiento de 16 horas	10 %	15,000.00
Total		**100 %**	**150,000.00**

8.3 Módulo de estaciones: computadoras y conectividad

El caso exige una computadora de trabajo en cada estación y una conexión entre ellas (condiciones 16 y 17). Su costo se estima por separado en la Tabla 18, porque se trata de equipamiento ordinario de las estaciones que puede ya existir o ser financiado por cada municipalidad. Por prudencia, la sección 9.3 muestra qué ocurre con la rentabilidad si el proyecto tuviera que asumirlo.

Tabla 18

Costo referencial del módulo de computadoras y conectividad de 15 estaciones

Concepto	Cantidad	Costo unitario (GTQ)	Subtotal (GTQ)
PC de trabajo con monitor y UPS	15	4,000.00	60,000.00
Router con soporte de VPN	15	900.00	13,500.00
Configuración e instalación	1	10,000.00	10,000.00
Inversión inicial del módulo			**83,500.00**
Internet de banda ancha empresarial (por mes y estación)	15 × 12	300.00	54,000.00 al año
Mantenimiento de las PC (por mes)	12	1,000.00	12,000.00 al año
Gasto anual del módulo			**66,000.00 al año**

Nota. El enlace de respaldo móvil ya está incluido en la partida de datos celulares de la Tabla 16. Los salarios de operadores y guardias no se incluyen: se asumen como personal municipal existente.

8.4 Condiciones de pago, vigencia y exclusiones

Tabla 19

Plan de pagos propuesto por hitos de entrega

Hito	Entregable que lo activa	Porcentaje	Monto (GTQ)
Anticipo	Firma de la carta de aceptación del proyecto y arranque	30 %	210,150.00
Diseño y base de datos	Modelo ER, script SQL y diagramas UML aprobados	30 %	210,150.00
Aplicación y pruebas	Aplicación publicada, reportes y 36 pruebas superadas	30 %	210,150.00
Aceptación final	Piloto de la Línea 12 y carta de aceptación firmada	10 %	70,050.00
Total		**100 %**	**700,500.00**

Nota. Los porcentajes corresponden a la inversión inicial de la Tabla 15. Los gastos operativos se pagan por mes, desde el inicio de la operación.

- **Vigencia de la oferta:** 90 días desde la fecha de este documento.
- **Precios:** en quetzales, sin impuestos; deben actualizarse si las cotizaciones de equipo cambian más de 10 %.
- **Exclusiones:** compra de buses, obra civil, política tarifaria, salarios del personal operativo y la integración con bancos más allá de la recarga.
- **Garantía:** corrección de defectos durante 90 días después de la aceptación, sin costo.

Capítulo IX. Estudios de factibilidad

El estudio de factibilidad examina cinco dimensiones: técnica, operativa, económica, legal y ambiental. Las conclusiones de cada una se resumen en la Tabla 20.

Tabla 20
Resumen de la evaluación de factibilidad

Dimensión	Conclusión	Condición para que se mantenga
Técnica	Viable	Compatibilidad con los molinetes y equipo existentes; conectividad en las 15 estaciones.
Operativa	Viable	Capacitación de 16 horas y participación de los operadores de estación.
Económica	Viable pero ajustada	Que los ahorros reales no sean más de 10 % menores a lo estimado.
Legal	Viable	Contratación conforme a la ley y tratamiento de datos personales con medidas de protección.
Ambiental	Favorable	Reducción del consumo de combustible con menos recorridos en vacío.

9.1 Factibilidad técnica

La solución es técnicamente viable por la madurez de los componentes de hardware disponibles en el mercado local y por la existencia de infraestructura previa en los corredores. Los controladores de borde se integran con los molinetes mecánicos existentes mediante protocolos industriales estándar como RS-485 y Modbus, lo que evita reemplazar el equipamiento actual. El equipo de estación posee certificación IP66, adecuada para tolerar intemperie, polvo y vibración vehicular.

La parte nueva del sistema se apoya en tecnología de código abierto y de amplio uso: PostgreSQL como motor de datos, React para la interfaz y WireGuard o IPsec para las conexiones. La base de datos se construyó y probó con éxito en dos versiones del motor (16 y 17), lo que reduce el riesgo de portabilidad.

9.2 Factibilidad operativa

El sistema se integra sin alterar el flujo habitual de abordaje de los usuarios. Para el personal operativo se prevé un programa de capacitación de 16 horas, enfocado en el manejo de la consola HMI por parte de los pilotos y del panel de despacho por parte de los operadores de patio. La dosificación del ingreso de pasajeros en los molinetes contribuye, además, a la seguridad en las plataformas de abordaje.

La interfaz de la estación concentra en una sola pantalla lo que el operador necesita: la línea o líneas que sirven a la estación, los accesos y sus guardias, los buses que llegan y las alertas pendientes. Con 35 estaciones en los datos de demostración, el modelo de una PC por estación y un operador por turno requiere 44 operadores en total, de los cuales 35 cubren el turno de la mañana.

9.3 Factibilidad económica

Los beneficios del proyecto provienen de la reducción en el consumo de diésel (Q 210,000.00 anuales), de la disminución de las horas extraordinarias derivadas de un despacho ineficiente y de la recuperación de ingresos por evasión tarifaria (Q 100,000.00 anuales), lo que genera un beneficio directo estimado de Q 310,000.00 por año. La Tabla 21 proyecta el flujo de caja a cinco años con un incremento promedio anual del 4 % tanto en los costos operativos como en los ahorros.

Tabla 21

Flujo de caja proyectado a cinco años (en quetzales)

Año	Inversión (CAPEX)	Operación (OPEX)	Ahorro generado	Flujo neto
0	700,500.00	0.00	0.00	-700,500.00
1	0.00	120,000.00	310,000.00	190,000.00
2	0.00	124,800.00	322,400.00	197,600.00
3	0.00	129,792.00	335,296.00	205,504.00
4	0.00	134,983.00	348,707.00	213,724.00
5	0.00	140,382.00	362,655.00	222,273.00

Nota. Proyección con incremento promedio anual del 4 % en OPEX y ahorros. Los flujos netos de los años 4 y 5 se calcularon con las cifras sin redondear.

A partir de estos flujos se calcularon los indicadores de la Tabla 22. El período de recuperación se determinó acumulando los flujos netos: al cierre del tercer año se han recuperado Q 593,104.00 y restan Q 107,396.00, que se cubren con poco más de la mitad del flujo del cuarto año, de modo que la inversión se recupera a los tres años y seis meses de operación continua. Con una tasa de descuento del 10 % anual, el proyecto reporta un Valor Presente Neto positivo de Q 73,921.67 y una Tasa Interna de Retorno de 13.94 %, superior a la tasa de oportunidad asumida para inversiones públicas municipales. La relación beneficio-costo, calculada con los flujos descontados de ahorros frente a la inversión y los costos operativos, es de 1.06.

Tabla 22

Indicadores de evaluación económica

Indicador	Resultado	Criterio de decisión
Valor presente neto (tasa del 10 %)	Q 73,921.67	Aceptar si es mayor que cero.
Tasa interna de retorno	13.94 %	Aceptar si supera la tasa de descuento (10 %).
Período de recuperación	3 años y 6 meses	Menor a la vida útil estimada de cinco años.
Relación beneficio-costo	1.06	Aceptar si es mayor que 1.

Nota. Cálculos propios con base en el flujo de la tabla anterior.

El margen económico del proyecto es estrecho, por lo que conviene examinar su comportamiento ante desviaciones en los supuestos. La Tabla 23 muestra que el resultado depende sobre todo del ahorro efectivamente logrado. Una reducción del 10 % en los beneficios vuelve negativo el VPN (-Q 52,431) y reduce

la TIR a 7.10 %, mientras que un aumento equivalente la eleva a 20.34 %. Un incremento del 15 % en los costos operativos deja el VPN prácticamente en cero. Estos resultados indican que el éxito financiero está condicionado a la recuperación efectiva de los ingresos por evasión y al ahorro de combustible, y justifican que el despliegue comience con el corredor piloto antes de escalar.

Tabla 23

Sensibilidad del VPN y la TIR ante variaciones en los supuestos

Escenario	VPN al 10 % (GTQ)	TIR (%)
Escenario base	73,922	13.94
Ahorros 10 % menores	-52,431	7.10
Ahorros 20 % menores	-178,784	-0.34
Ahorros 10 % mayores	200,275	20.34
Costos operativos 15 % mayores	556	10.03
Ahorros 10 % menores y costos 15 % mayores	-125,797	2.87

Nota. Cálculos propios sobre el flujo base con horizonte de cinco años y sin valor residual.

Efecto del módulo de estaciones

Si el proyecto asumiera también el costo del módulo de computadoras y conectividad (Q 83,500.00 de inversión y Q 66,000.00 al año), el flujo neto anual bajaría de Q 190,000.00 a Q 124,000.00 en el primer año, el Valor Presente Neto sería de −Q 278,587.84 y la Tasa Interna de Retorno de −4.83 %. Para recuperar la inversión con ese módulo incluido, los ahorros tendrían que ser de Q 378,350.00 al año, es decir, 22 % más que los Q 310,000.00 estimados. Por eso esta propuesta trata las PC y los enlaces como equipamiento de las estaciones a cargo de cada municipalidad y recomienda medir en el piloto si los ahorros alcanzan ese nivel.

Tabla 24

Indicadores con y sin el módulo de estaciones

Escenario	Inversión (GTQ)	Gasto anual año 1 (GTQ)	VPN al 10 % (GTQ)	TIR
Proyecto base (sin módulo de estaciones)	700,500.00	120,000.00	73,921.67	13.94 %
Con el módulo de estaciones incluido	784,000.00	186,000.00	−278,587.84	−4.83 %

Nota. Cálculo propio con la misma metodología de la Tabla 21: crecimiento de 4 % anual en gastos y ahorros, cinco años y sin valor residual.

9.4 Factibilidad legal y regulatoria

El proyecto se enmarca en la Ley de Contrataciones del Estado de Guatemala (Congreso de la República de Guatemala, 1992), que regula las adquisiciones de las entidades públicas. En materia de privacidad, el procesamiento de aforo mediante cámaras anonimiza el video en el origen: las imágenes se transforman en vectores numéricos sin almacenar archivos de rostro, lo que respeta los derechos de los ciudadanos y es coherente con el requerimiento RNF-07.

El sistema almacena datos personales de pilotos, guardias y operadores: número de documento de identificación, dirección de residencia, teléfonos y estudios. Guatemala no cuenta con una ley específica de protección de datos personales; la norma que hoy los regula en los archivos del Estado es la Ley de Acceso a la Información Pública, Decreto 57-2008 (Congreso de la República de Guatemala, 2008; EDP Guatemala, 2019). Como buena práctica, el diseño aplica la minimización y la protección de esos datos: el DPI se muestra enmascarado, los roles de consulta no pueden leerlo y toda modificación queda en una bitácora inalterable (Capítulo XVI).

9.5 Factibilidad ambiental

La reducción del agrupamiento de autobuses y la eliminación de recorridos en vacío optimizan la marcha de los motores diésel. Se estima un ahorro del 8 % en el consumo anual de combustible, con la consiguiente disminución de gases de efecto invernadero en el corredor urbano.

Al registrar la ocupación de cada visita, el sistema permite reducir los recorridos casi vacíos y enviar buses de refuerzo solo donde hacen falta, lo que mejora el uso del combustible. La aplicación, que se ejecuta en el navegador y se publica como sitio estático, tiene además un consumo de energía de servidor muy bajo.

Capítulo X. Metodología de desarrollo y planificación

10.1 Enfoque de la investigación

El trabajo es una investigación aplicada de carácter descriptivo y propositivo. Es descriptiva porque caracteriza la problemática operativa del Transmetro a partir de fuentes documentales, y propositiva porque formula, modela y valida una solución tecnológica. La validación se realiza mediante un prototipo funcional y mediante un plan de pruebas con criterios cuantitativos de aceptación.

10.2 Metodología elegida: Scrum adaptado

Para el desarrollo se eligió **Scrum adaptado** (Schwaber y Sutherland, 2020), con iteraciones de dos semanas. Las razones son tres: las municipalidades aún no han confirmado todas las reglas y es probable que cambien; el valor se puede entregar por partes (primero líneas y estaciones, después flota, después alertas); y el cliente debe ver resultados cada dos semanas para ajustar el rumbo. La Tabla 25 compara tres alternativas.

Tabla 25

Comparación de metodologías de desarrollo

Criterio	Cascada	Scrum adaptado	Programación extrema
Requisitos que pueden cambiar	Bajo	**Alto**	Alto
Entregas parciales cada pocas semanas	No	**Sí**	Sí
Participación del cliente	Al inicio y al final	**En cada iteración**	Continua en el equipo
Documentación exigida por el curso	**Alta**	Media (se complementa)	Baja
Equipo de una sola persona	Adecuado	**Adecuado con ajustes**	Poco adecuado (requiere parejas)
Puntuación (de 5 criterios)	2	**5**	3

Nota. La puntuación suma un punto por criterio en que la metodología es adecuada. Elaboración propia.

Adaptaciones al contexto

- **Roles.** El representante designado por las municipalidades actúa como dueño del producto; el consultor asume los roles de desarrollo y de facilitación. Las decisiones de prioridad son del dueño del producto.
- **Iteraciones.** Cuatro iteraciones de desarrollo de dos semanas (Capítulo X, sección 10.4). Cada una termina con una demostración y una revisión de lo aprendido.
- **Pila del producto.** Las dieciocho condiciones del caso son las historias principales; cada una tiene su prueba automática.
- **Definición de hecho.** Una historia está terminada cuando su regla está en la base de datos, tiene prueba automática superada, aparece en la interfaz o en un reporte y está documentada.
- **Control de versiones.** Todo el código y los documentos se guardan en un repositorio Git; la publicación de la aplicación se hace de forma automática.

10.3 Fases y productos

Se adoptó un proceso iterativo e incremental, apropiado para sistemas de gran alcance cuyos módulos pueden entregarse de forma gradual (Sommerville, 2016). Cada iteración recorre las actividades de análisis, modelado, construcción del prototipo y verificación. La Tabla 26 resume las fases del trabajo y los productos que se obtuvieron en cada una.

Tabla 26
Fases metodológicas del proyecto

Fase	Actividades principales	Productos
Diagnóstico	Análisis documental del problema operativo y revisión de literatura sobre control de flota y recaudación.	Planteamiento del problema y marco teórico.
Especificación	Identificación de actores, módulos y reglas de negocio; redacción de requerimientos.	Catálogo de 20 RF y 8 RNF; matriz de roles.
Factibilidad	Estimación de costos, ahorros y flujo de caja a cinco años.	Presupuesto, VPN, TIR y análisis de sensibilidad.
Diseño	Definición de capas, selección de tecnologías y modelado UML y relacional.	Arquitectura, pila tecnológica y cuatro diagramas.
Construcción	Desarrollo del prototipo web con simulación de sensores y eventos.	Aplicación de cinco módulos y consola de eventos.
Verificación	Diseño del plan de pruebas, criterios de aceptación y métricas.	Matriz de casos, criterios A-FUN y A-TEC, carta de aceptación.

Nota. Elaboración propia a partir del proceso iterativo descrito en el texto.

10.4 Planificación: diagrama de Gantt

El proyecto dura 19 semanas, del 2 de noviembre de 2026 al 12 de marzo de 2027. La Figura 1 muestra el diagrama de Gantt y la Tabla 27 lista las fechas de cada actividad.

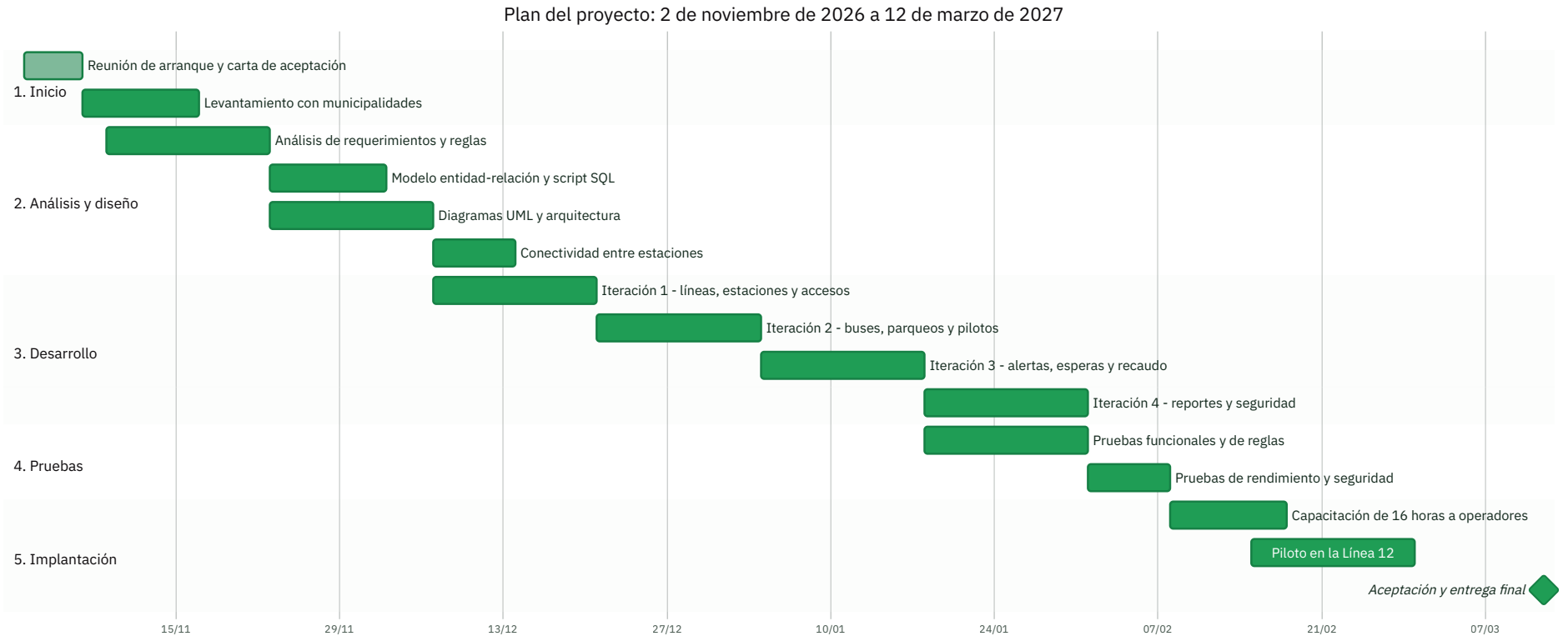
Tabla 27
Cronograma de actividades

Fase	Actividad	Inicio	Fin	Días
Inicio	Reunión de arranque y carta de aceptación	02/11/2026	06/11/2026	5
Inicio	Levantamiento con municipalidades	07/11/2026	16/11/2026	10
Análisis y diseño	Análisis de requerimientos y reglas	09/11/2026	22/11/2026	14
Análisis y diseño	Modelo entidad-relación y script SQL	23/11/2026	02/12/2026	10
Análisis y diseño	Diagramas UML y arquitectura	23/11/2026	06/12/2026	14
Análisis y diseño	Conectividad entre estaciones	07/12/2026	13/12/2026	7
Desarrollo	Iteración 1: líneas, estaciones y accesos	07/12/2026	20/12/2026	14
Desarrollo	Iteración 2: buses, parqueos y pilotos	21/12/2026	03/01/2027	14

Fase	Actividad	Inicio	Fin	Días
Desarrollo	Iteración 3: alertas, esperas y recaudo	04/01/2027	17/01/2027	14
Desarrollo	Iteración 4: reportes y seguridad	18/01/2027	31/01/2027	14
Pruebas	Pruebas funcionales y de reglas	18/01/2027	31/01/2027	14
Pruebas	Pruebas de rendimiento y seguridad	01/02/2027	07/02/2027	7
Implantación	Capacitación de 16 horas a operadores	08/02/2027	17/02/2027	10
Implantación	Piloto en la Línea 12	15/02/2027	28/02/2027	14
Implantación	Aceptación y entrega final	12/03/2027	12/03/2027	hito

Figura 1

Diagrama de Gantt del proyecto



Nota. Duraciones en días calendario. Las iteraciones de desarrollo son de dos semanas y las pruebas funcionales se solapan con la última iteración.

10.5 Gestión de riesgos

Tabla 28

Riesgos del proyecto y medidas de mitigación

Cód.	Riesgo	Probabilidad	Impacto	Mitigación
RI-01	Datos municipales incompletos o inconsistentes	Alta	Alto	Plantillas de carga y validaciones en la base; revisión conjunta en el levantamiento.
RI-02	Las reglas del caso se interpretan de otra forma	Media	Alto	Parámetros configurables y aprobación escrita de las interpretaciones (sección 7.2).
RI-03	Conectividad deficiente en algunas estaciones	Media	Alto	Enlace móvil de respaldo y operación local con sincronización posterior.
RI-04	Ahorros reales menores a los estimados	Media	Alto	Medición en el piloto antes de ampliar; plan por etapas.
RI-05	Resistencia de los operadores al nuevo sistema	Media	Medio	Capacitación de 16 horas y operadores participantes en las demostraciones.
RI-06	Filtración de datos personales	Baja	Alto	Roles mínimos, DPI enmascarado, bitácora y revisión de seguridad.
RI-07	Demora en las decisiones municipales	Media	Medio	Reuniones quincenales con fecha límite y escalamiento al dueño del producto.
RI-08	Rotación del personal capacitado	Media	Medio	Manuales cortos y videos de uso; capacitación repetible.

10.6 Técnicas e instrumentos

Para la obtención de la información se empleó la revisión documental de comunicados y noticias de la Municipalidad de Guatemala (Emisoras Unidas, 2022a), normas técnicas y bibliografía especializada. El modelado se realizó con diagramas UML de casos de uso, clases y secuencia, y con un diagrama entidad-relación para la base de datos. La construcción del prototipo utilizó React y Tailwind CSS sobre Vite, y la estimación económica se apoyó en los criterios de evaluación de proyectos de inversión descritos por Sapag y Sapag (2014): flujo de caja descontado, valor presente neto, tasa interna de retorno y período de recuperación.

Capítulo XI. Modelo de datos y script de la base de datos

11.1 Motor de base de datos elegido

Se eligió **PostgreSQL** por cuatro razones: es de código abierto y no tiene costo de licencia, permite disparadores que se evalúan al final de la transacción (necesarios para reglas como «entre N y 2N buses»), incluye funciones de ventana para calcular distancias acumuladas y puede ejecutarse tanto en un servidor como, mediante PGlite, dentro de un navegador. La Tabla 29 resume la comparación con otras opciones.

Tabla 29

Comparación de motores de base de datos

Criterio	PostgreSQL	MySQL	SQL Server	MongoDB
Costo de licencia	Ninguno	Ninguno (edición comunitaria)	Con licencia	Ninguno (edición comunitaria)
Restricciones diferidas (al confirmar la transacción)	**Sí**	No	No	No
Disparadores con reglas entre varias filas	**Sí**	Sí	Sí	Limitado
Modelo relacional con integridad referencial	**Sí**	Sí	Sí	No
Se ejecuta también en el navegador	**Sí (PGlite)**	No	No	No
Servicio gestionado en la nube	Sí	Sí	Sí	Sí

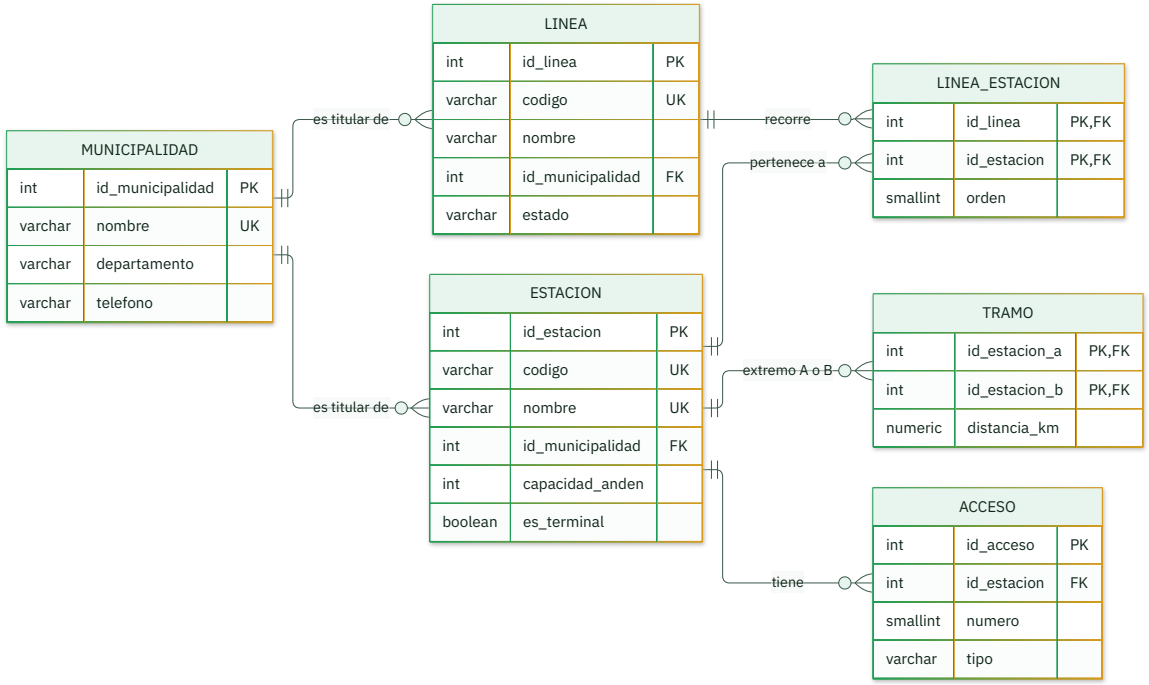
Nota. Elaboración propia. «Limitado» indica que la regla debe programarse fuera del motor.

11.2 Modelo entidad-relación

El modelo tiene 27 tablas, 36 llaves foráneas y 48 restricciones de verificación. Para que sea legible se presenta en seis diagramas temáticos: red de transporte (Figura 2), flota y parqueos, personal, estación y comunicaciones, operación y recaudo. Todas las tablas se nombran en español, con llave primaria numérica y llaves foráneas con el prefijo `id_`. Las dos tablas que no aparecen en los diagramas son `parametro_sistema` (valores configurables de las reglas) y `bitacora` (auditoría), que no tienen relaciones.

Figura 2

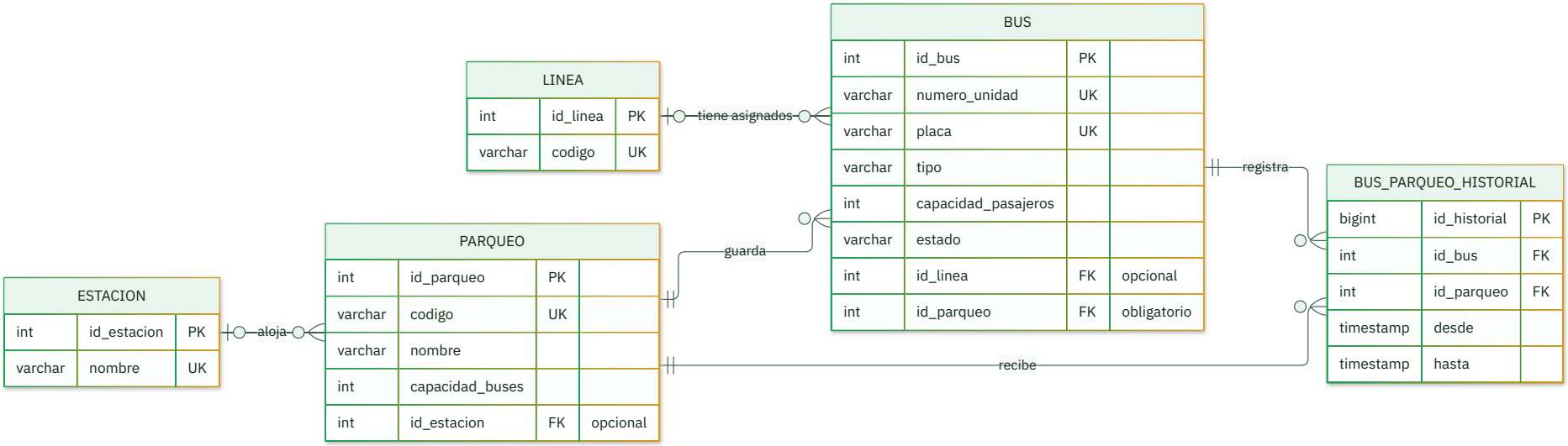
Modelo entidad-relación: red de transporte (municipalidades, líneas, estaciones, accesos y tramos)



Nota. PK = llave primaria, FK = llave foránea, UK = valor único. Una estación puede estar en varias líneas por medio de LINEA_ESTACION.

Figura 3

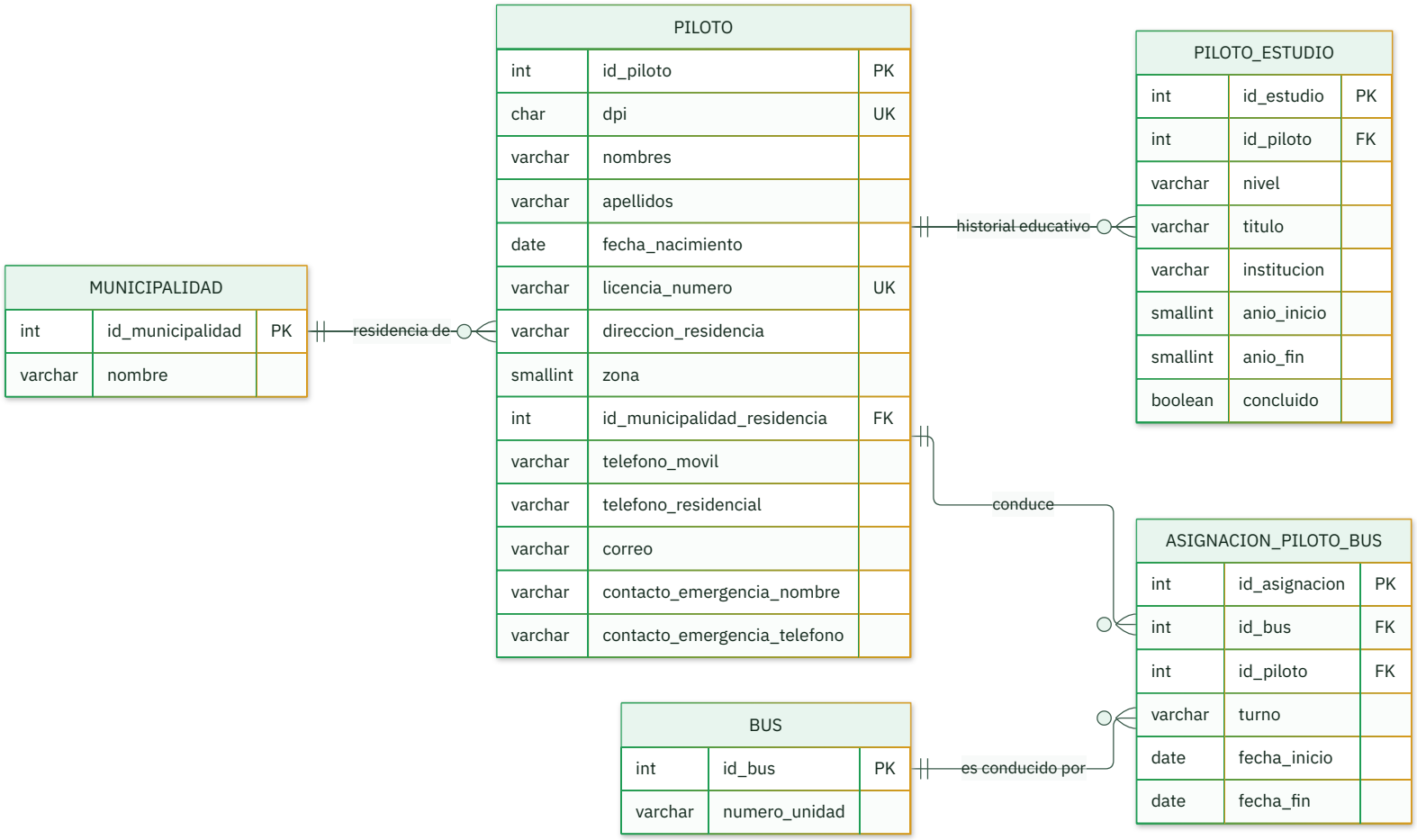
Modelo entidad-relación: flota, parqueos e historial de parqueo



Nota. El parqueo del bus es obligatorio; la línea es opcional. BUS_PARQUEO_HISTORIAL conserva cada cambio de parqueo.

Figura 4

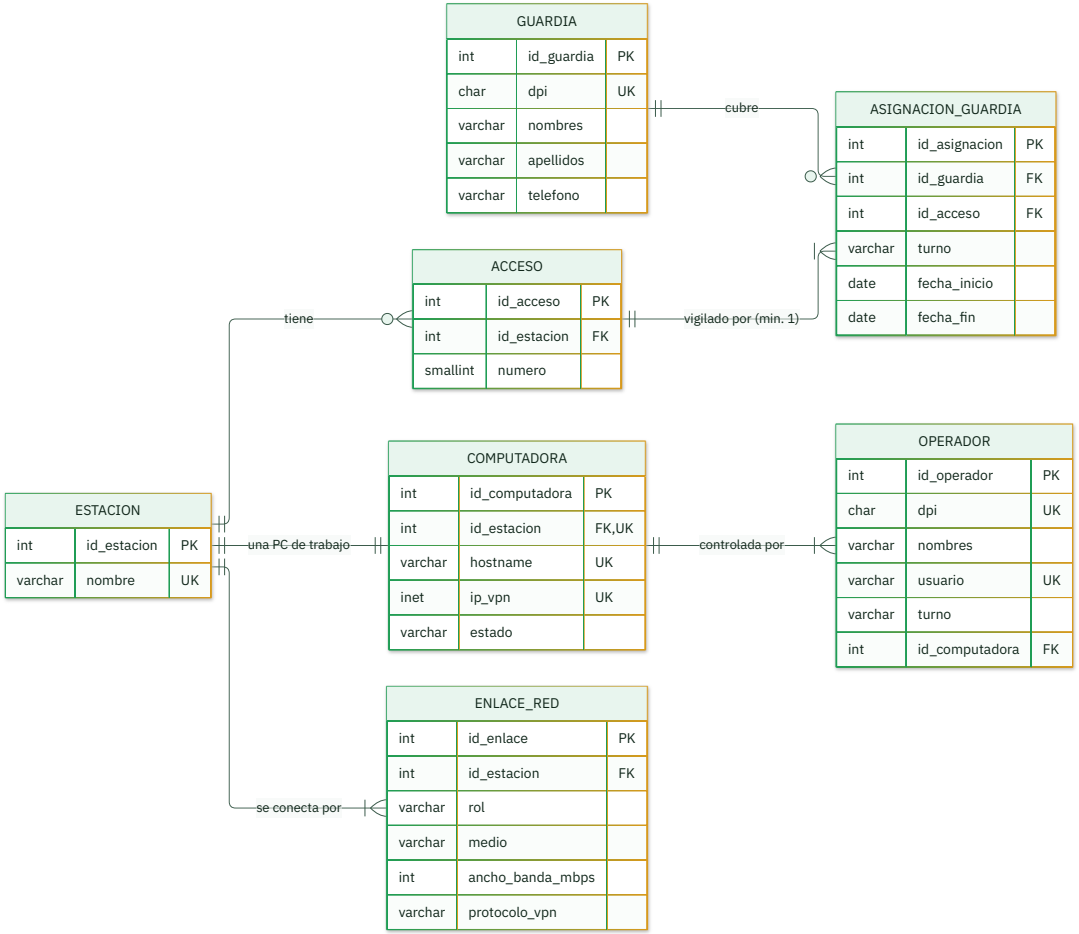
Modelo entidad-relación: pilotos y su historial educativo



Nota. Los datos de residencia y comunicación del piloto están en la misma tabla, y los estudios en una tabla de detalle.

Figura 5

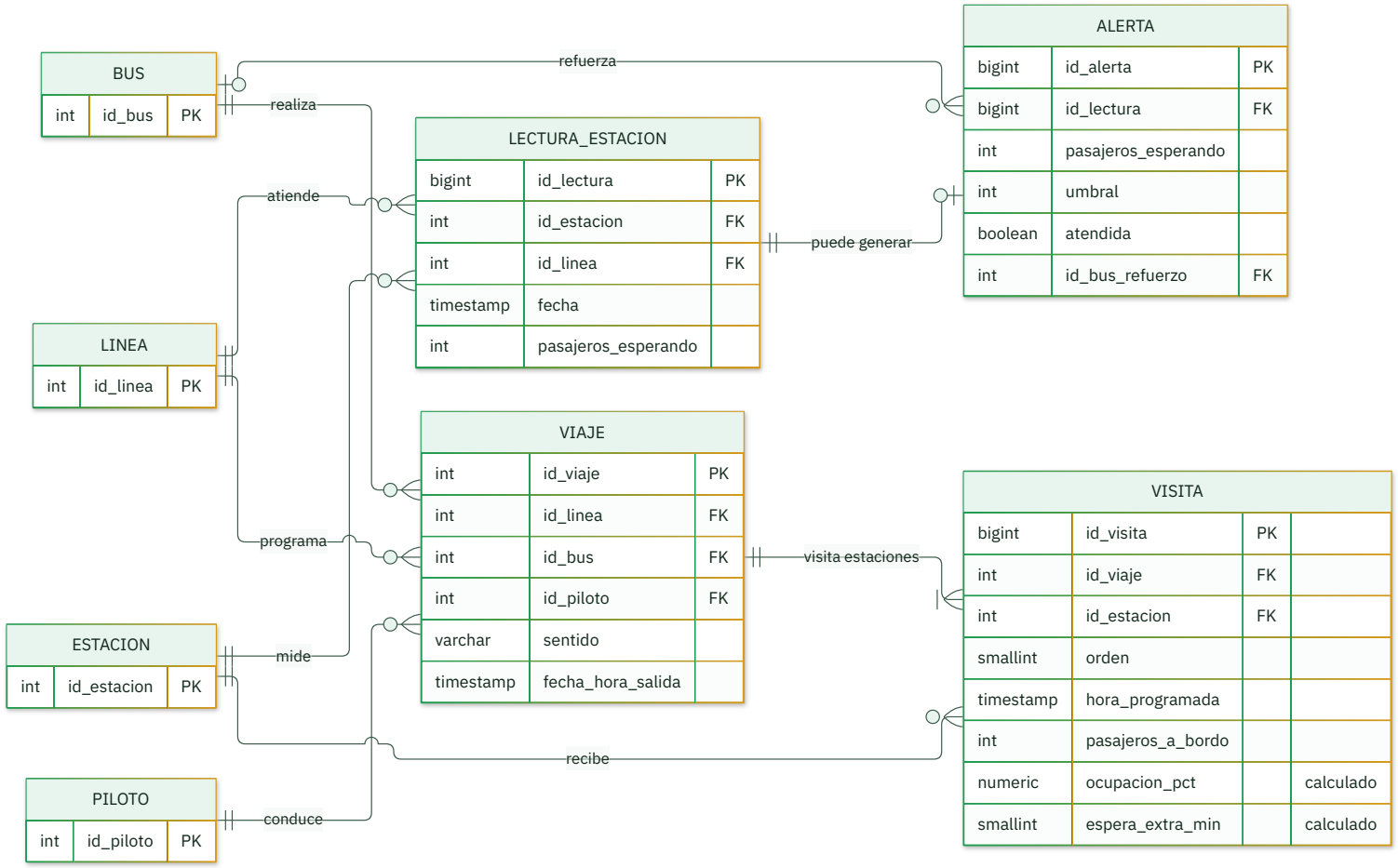
Modelo entidad-relación: guardias, computadora, operadores y enlaces de red de la estación



Nota. Cada acceso necesita al menos un guardia vigente; cada estación tiene una sola computadora de trabajo.

Figura 6

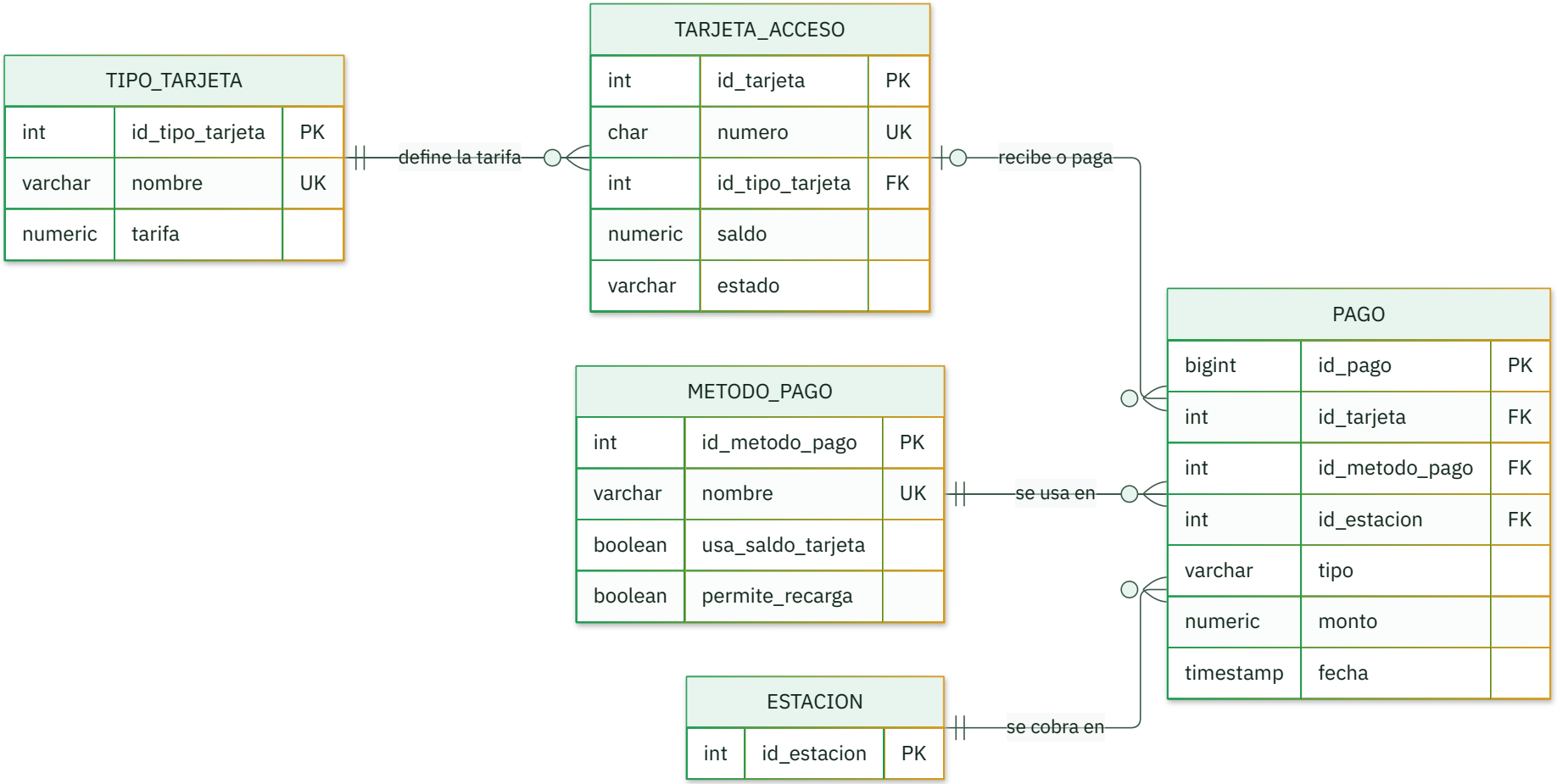
Modelo entidad-relación: recorridos, visitas, lecturas de pasajeros y alertas



Nota. ocupacion_pct y espera_extra_min las calcula la base de datos al registrar los pasajeros a bordo.

Figura 7

Modelo entidad-relación: tarjetas de acceso, métodos de pago y pagos



Nota. El saldo de la tarjeta se actualiza por disparador al registrar una recarga o un cobro.

11.3 Relaciones y cardinalidades

Tabla 30

Relaciones principales del modelo y reglas que expresan

Relación	Cardinalidad	Regla del caso
Municipalidad – Línea	1 a muchos	Cada línea pertenece a una municipalidad (11)
Municipalidad – Estación	1 a muchos	Cada estación pertenece a una municipalidad (11)
Línea – Estación (LINEA_ESTACION)	muchos a muchos	Una estación en varias líneas, con orden (1, 2)
Estación – Estación (TRAMO)	pares de estaciones contiguas	Distancia entre estaciones (12)
Estación – Acceso	1 a muchos (acceso obligatorio en una sola estación)	Varios accesos por estación (3)
Acceso – Guardia (ASIGNACION_GUARDIA)	1 a 1 o más guardias vigentes	Un guardia por acceso, como mínimo (10)
Estación – Parqueo	0 o 1 a muchos	Algunas estaciones tienen parqueo (6)
Parqueo – Bus	1 a muchos (obligatorio)	Cada bus tiene parqueo (6, 7)
Línea – Bus	0 o 1 a muchos	Un bus en una línea o en ninguna (4); entre N y 2N (5)
Piloto – Estudio	1 a muchos	Historial educativo (9)
Piloto – Bus (ASIGNACION_PILOTO_BUS)	muchos a muchos con turno	Quién conduce cada bus
Estación – Computadora	1 a 1	Una PC de trabajo por estación (16)
Computadora – Operador	1 a 1 o más	Un operador que controle la PC (16)
Viaje – Visita	1 a 1 o más	Visitas de cada unidad en su recorrido (1)
Lectura de estación – Alerta	1 a 0 o 1	Alerta de capacidad (14)
Tarjeta – Pago – Método de pago	0 o 1 a muchos	Pagos con tarjeta y otros métodos (18)

11.4 Tablas y datos de demostración

La Tabla 31 lista las 27 tablas con su propósito y la cantidad de filas que traen los datos de demostración. Todos los datos son ficticios: los nombres, documentos y teléfonos se generan con una semilla fija para que cualquier persona pueda reconstruir exactamente la misma base.

Tabla 31

Tablas de la base de datos y filas de demostración

Tabla	Propósito	Filas
`acceso`	Accesos de cada estación	72

Tabla	Propósito	Filas
`alerta`	Alertas de capacidad y su atención	26
`asignacion_guardia`	Guardias asignados a accesos por turno	102
`asignacion_piloto_bus`	Pilotos asignados a buses por turno	85
`bitacora`	Auditoría de cambios (solo inserciones)	0
`bus`	Buses, su línea y su parqueo	71
`bus_parqueo_historial`	Cambios de parqueo de cada bus	72
`computadora`	PC de trabajo de cada estación	35
`enlace_red`	Enlaces principal y de respaldo de cada estación	70
`estacion`	Estaciones con ubicación y capacidad del andén	35
`guardia`	Guardias de seguridad	108
`lectura_estacion`	Pasajeros esperando en cada estación	170
`linea`	Líneas del Transmetro y su estado	5
`linea_estacion`	Estaciones de cada línea y su orden	41
`metodo_pago`	Métodos de pago aceptados	5
`municipalidad`	Municipalidades titulares de líneas y estaciones	8
`operador`	Operadores que controlan cada PC	44
`pago`	Recargas y cobros de pasajes	697
`parametro_sistema`	Valores que cambian las reglas (porcentajes, minutos)	7
`parqueo`	Parqueos y patios de buses	7
`piloto`	Datos personales, de residencia y de contacto de los pilotos	90
`piloto_estudio`	Historial educativo de los pilotos	349
`tarjeta_acceso`	Tarjetas con saldo prepagado	60
`tipo_tarjeta`	Perfiles de tarjeta y sus tarifas	4
`tramo`	Distancia entre estaciones contiguas	35
`viaje`	Recorridos programados de cada bus	13
`visita`	Visitas planificadas y reales a cada estación	140

Nota. La bitácora se vacía al terminar la carga inicial para que solo contenga actividad posterior.

11.5 Reglas aplicadas por el motor

Las reglas más importantes del caso no se dejan a la aplicación: las hacen cumplir funciones y disparadores de PostgreSQL. La base de datos contiene 20 funciones, 21 disparadores, 14 vistas y 59 índices. La Tabla 32 describe los disparadores principales.

Tabla 32

Funciones y disparadores que aplican las reglas del caso

Elemento	Tabla	Cuándo actúa	Qué hace
`ct_bus_flota`, `ct_linea_estado`, `ct_linea_estacion_flota`	bus, linea, linea_estacion	Al confirmar la transacción	Exige entre N y 2N buses por línea (el mínimo solo si está activa) (RN-05)
`ct_estacion_con_linea`, `ct_linea_estacion_borrado`	estacion, linea_estacion	Al confirmar	Rechaza una estación sin línea (RN-02)
`ct_acceso_con_guardia`, `ct_guardia_retiro`	acceso, asignacion_guardia	Al confirmar	Exige un guardia vigente por acceso, incluso al retirar al último (RN-10)
`tg_bus_parqueo_capacidad`	bus	Antes de guardar	Rechaza un parqueo lleno (RN-06)
`tg_bus_parqueo_historial`	bus	Después de guardar	Registra cada cambio de parqueo (RN-07)
`tg_viaje_valida`	viaje	Antes de guardar	El bus debe ser de la línea del viaje (RN-04)
`fn_generar_visitas`	viaje, visita	Al llamarla	Crea las visitas con horas estimadas por distancia (RN-01)
`tg_visita_ocupacion`	visita	Antes de guardar	Calcula ocupación y espera extra de 5 min bajo 25 % (RN-15)
`tg_lectura_alerta`	lectura_estacion	Después de guardar	Crea la alerta al llegar a 150 % de la capacidad del bus (RN-14)
`fn_atender_alerta`	alerta	Al llamarla	Marca la alerta atendida con el bus de refuerzo
`tg_pago_aplica`	pago	Antes de guardar	Valida método, estado y saldo; suma o descuenta el saldo (RN-18)
`aud_*`, `tg_bitacora_inmutable`	varias, bitacora	Después de cambios	Auditan los cambios y bloquean modificar o borrar la bitácora
`fn_distancia_tramo`, `fn_distancia_en_linea`, `fn_distancia_linea`	tramo, linea_estacion	Al consultar	Calculan distancias entre estaciones y de la línea (RN-12, RN-13)

11.6 Script de creación de las estructuras

El script se divide en cinco archivos que se ejecutan en orden y dos más para reportes y pruebas, todos en la carpeta `database` del repositorio. El texto completo de los archivos 01 a 04 está en el Apéndice B.

Tabla 33
Archivos del script de base de datos

Archivo	Contenido	Líneas
`01_schema.sql`	Esquema, tablas, llaves, restricciones e índices	344
`02_funciones_triggers.sql`	Funciones y disparadores que aplican las reglas	390
`03_vistas.sql`	Vistas de apoyo a los reportes	161
`04_seguridad.sql`	Roles, privilegios mínimos y datos sensibles	35
`05_seed.sql`	Datos de demostración (generado con `generar_seed.mjs`)	2,307
`06_reportes.sql`	Las 17 consultas de los reportes	176
`07_pruebas.sql`	36 pruebas automáticas de las reglas	240

Para crear la base en un servidor PostgreSQL basta con ejecutar los archivos en orden:

```
psql -U postgres -f database/01_schema.sql
psql -U postgres -f database/02_funciones_triggers.sql
psql -U postgres -f database/03_vistas.sql
psql -U postgres -f database/04_seguridad.sql
psql -U postgres -f database/05_seed.sql
psql -U postgres -f database/07_pruebas.sql    # opcional: comprueba las reglas
```

Como ejemplo de lo que contiene el script, la siguiente listado muestra la definición de la tabla de buses con las restricciones de las reglas 4, 6 y 7. Su texto completo y el de todas las demás tablas están en el Apéndice B.

Fragmento de 01_schema.sql: tabla bus

```
CREATE TABLE bus (
    id_bus              INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    numero_unidad       VARCHAR(10) NOT NULL UNIQUE,
    placa               VARCHAR(10) NOT NULL UNIQUE,
    tipo                VARCHAR(12) NOT NULL DEFAULT 'ARTICULADO'
                        CHECK (tipo IN ('ARTICULADO','CONVENCIONAL')),
    capacidad_pasajeros INT         NOT NULL CHECK (capacidad_pasajeros > 0),
    anio                SMALLINT    NOT NULL CHECK (anio BETWEEN 2000 AND 2100),
    estado              VARCHAR(13) NOT NULL DEFAULT 'OPERATIVO'
                        CHECK (estado IN ('OPERATIVO','MANTENIMIENTO')),
    id_linea            INT         REFERENCES linea(id_linea),          -- NULL = sin línea (regla 4)
    id_parqueo          INT         NOT NULL REFERENCES parqueo(id_parqueo) -- nunca sin parqueo (regla 7)
);
```

11.7 Vistas de apoyo

Las vistas encapsulan las consultas más repetidas y mantienen las consultas de los reportes cortas. Son: `vw_linea_estaciones` (orden y distancia acumulada), `vw_linea_resumen` y `vw_linea_cumplimiento` (flota frente al rango permitido), `vw_estacion_resumen`, `vw_accesos_por_linea`, `vw_buses_por_linea`, `vw_buses_sin_linea`, `vw_pilotos_ficha` (con el DPI enmascarado), `vw_alertas_capacidad`, `vw_visitas_espera`, `vw_operadores_estacion`, `vw_recaudo_metodo_pago` y las dos vistas de control `vw_accesos_sin_guardia` y `vw_tramos_faltantes`.

Capítulo XII. Diagramas UML

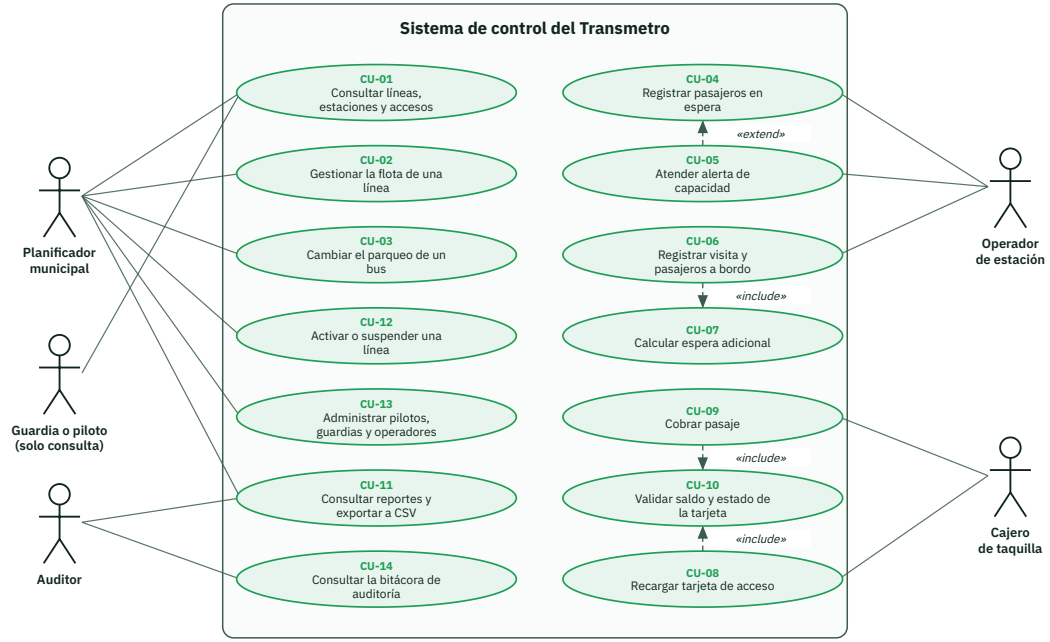
Este capítulo presenta los seis tipos de diagramas UML solicitados: casos de uso, componentes, secuencia, actividades, clases y objetos, además de un diagrama de estados de la línea. Los diagramas se dibujaron con Mermaid o como SVG y sus fuentes están en la carpeta `docs/diagramas` del repositorio, de modo que pueden editarse y regenerarse.

12.1 Diagrama de casos de uso

El sistema tiene catorce casos de uso y cinco actores. La Figura 8 los muestra y la Tabla 34 los relaciona con las condiciones del caso. El caso CU-05 extiende a CU-04, porque la alerta solo se atiende cuando la lectura supera el umbral; CU-07 y CU-10 son comportamientos que otros casos incluyen siempre.

Figura 8

Diagrama de casos de uso del sistema de control del Transmetro



Nota. Las líneas continuas unen actores y casos de uso; las discontinuas son relaciones de inclusión y extensión.

Tabla 34

Casos de uso y trazabilidad con las condiciones del caso

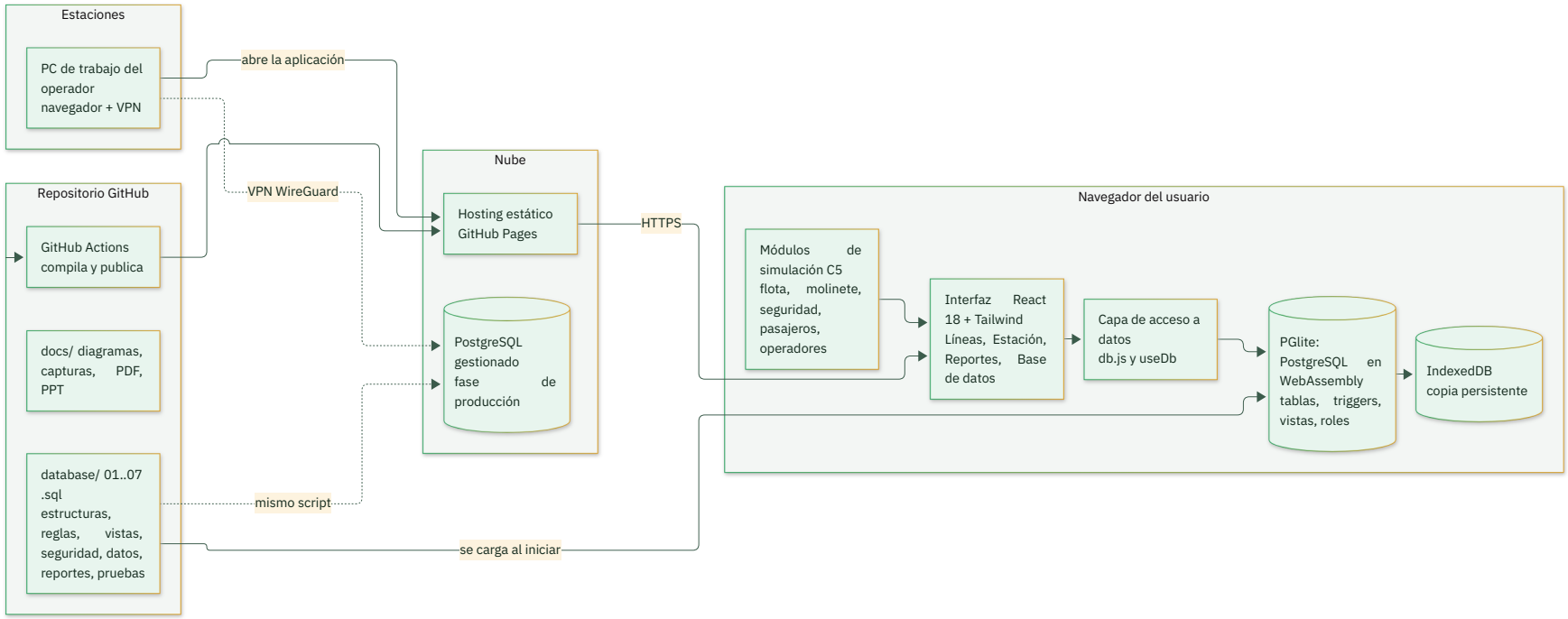
ID	Caso de uso	Actor principal	Condiciones	Prueba
CU-01	Consultar líneas, estaciones y accesos	Planificador, operador	RN-01, 02, 03, 08	T10, T14, T24
CU-02	Gestionar la flota de una línea	Planificador	RN-04, 05	T01 a T05, T23
CU-03	Cambiar el parqueo de un bus	Planificador	RN-06, 07	T06 a T08
CU-04	Registrar pasajeros en espera	Operador de estación	RN-14	T17, T18
CU-05	Atender alerta de capacidad	Operador de estación	RN-14	T19
CU-06	Registrar visita y pasajeros a bordo	Operador de estación	RN-01, 15	T20 a T22, T24
CU-07	Calcular espera adicional	(incluido en CU-06)	RN-15	T20, T21
CU-08	Recargar tarjeta de acceso	Cajero de taquilla	RN-18	T26, T28
CU-09	Cobrar pasaje	Cajero de taquilla	RN-18	T25, T27
CU-10	Validar saldo y estado de la tarjeta	(incluido en CU-08 y CU-09)	RN-18	T25
CU-11	Consultar reportes y exportar a CSV	Planificador, auditor	RN-08, 09, 12, 13, 16	T14 a T16
CU-12	Activar o suspender una línea	Planificador	RN-05	T03, T04
CU-13	Administrar pilotos, guardias y operadores	Planificador	RN-09, 10, 16	T11, T12, T34, T35
CU-14	Consultar la bitácora de auditoría	Auditor	(seguridad)	T29, T30

12.2 Diagrama de componentes

La Figura 9 muestra los componentes del sistema y dónde se ejecutan. La interfaz React y los módulos de simulación corren en el navegador; la capa de acceso a datos conecta la interfaz con PGlite, que ejecuta PostgreSQL en WebAssembly y guarda una copia en IndexedDB. Los scripts SQL del repositorio se cargan al iniciar y son los mismos que se usarían en un PostgreSQL gestionado en la fase de producción.

Figura 9

Diagrama de componentes y despliegue



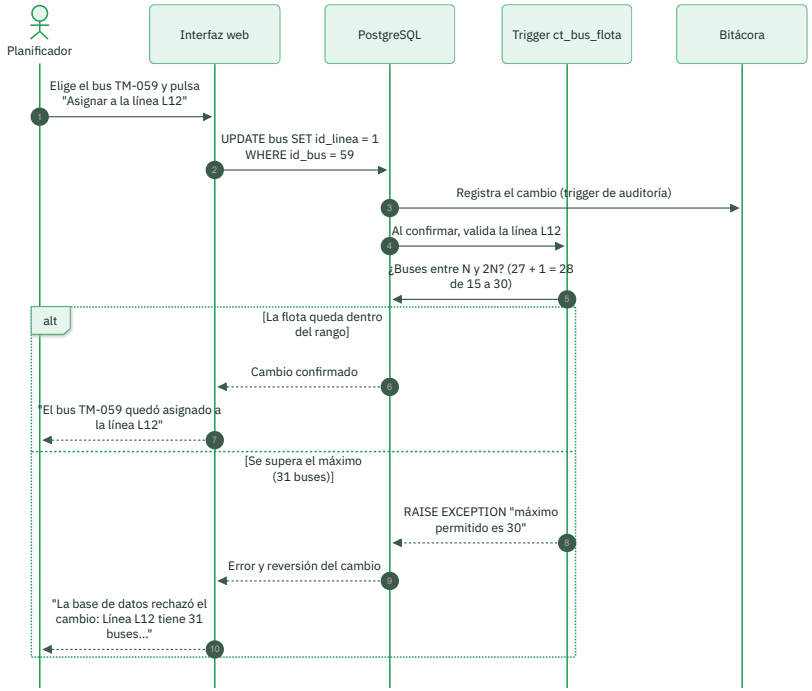
Nota. Las líneas discontinuas indican elementos de la fase de producción que usan los mismos scripts.

12.3 Diagramas de secuencia

La Figura 10 muestra qué ocurre cuando un planificador asigna un bus a una línea y la regla 5 lo rechaza, y la Figura 11 el camino desde que el operador registra los pasajeros que esperan hasta que se envía un bus de refuerzo. En ambos casos la decisión la toma la base de datos y la interfaz solo muestra el mensaje.

Figura 10

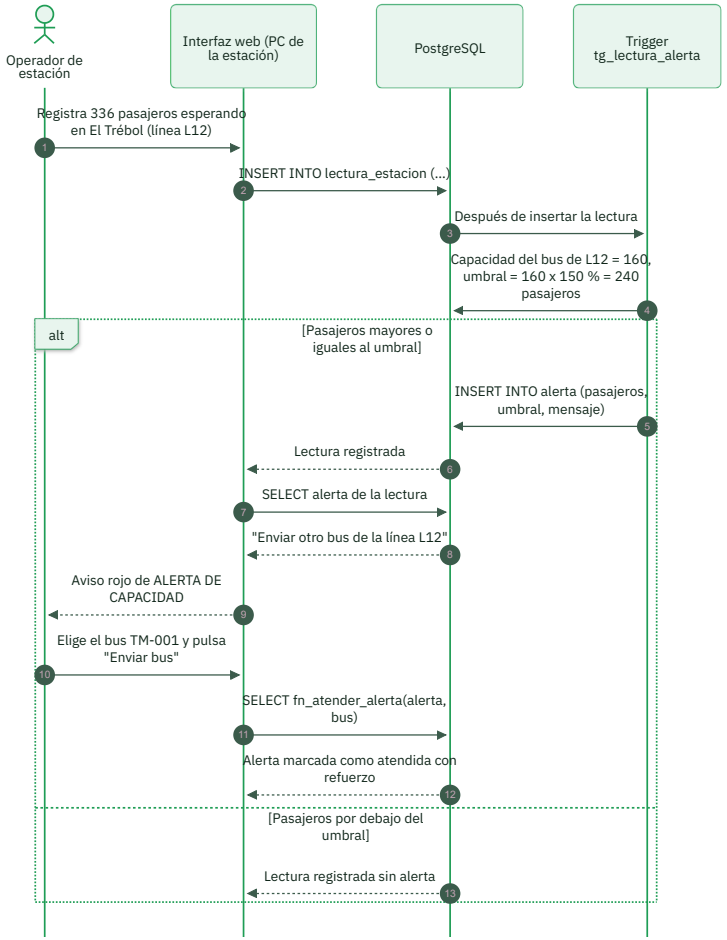
Diagrama de secuencia: asignar un bus a una línea con validación de la flota



Nota. El disparador diferido valida la flota al confirmar el cambio; si falla, PostgreSQL revierte la operación.

Figura 11

Diagrama de secuencia: lectura de pasajeros, alerta y bus de refuerzo



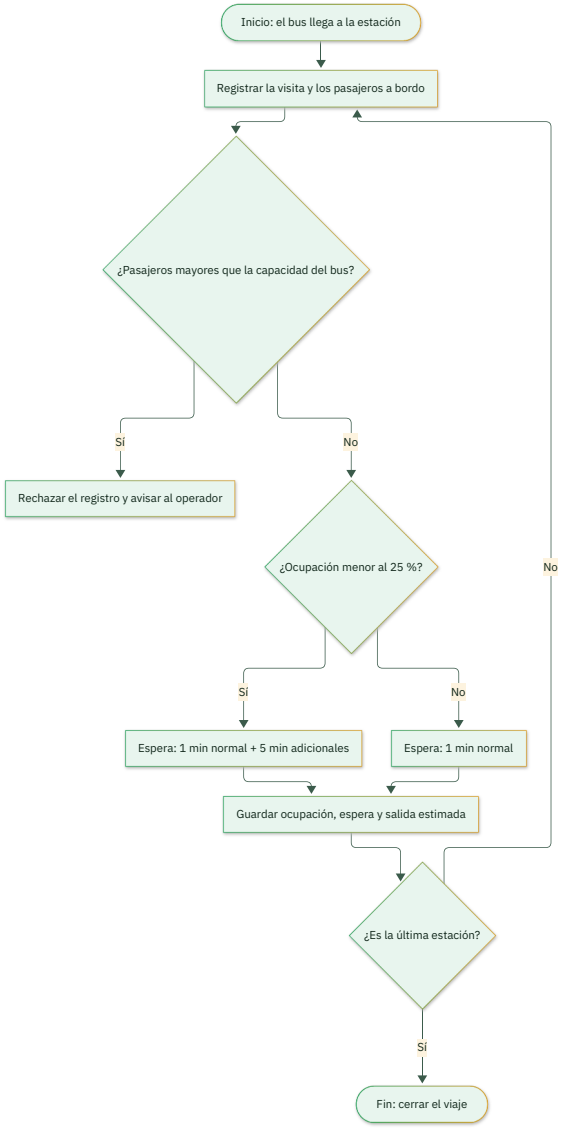
Nota. El umbral es 150 % de la capacidad del bus (240 pasajeros para un bus articulado de 160).

12.4 Diagramas de actividades

Las actividades de mayor valor son el registro de una visita, que aplica la regla de espera por baja ocupación (Figura 12), y la atención de la alerta de capacidad (Figura 13).

Figura 12

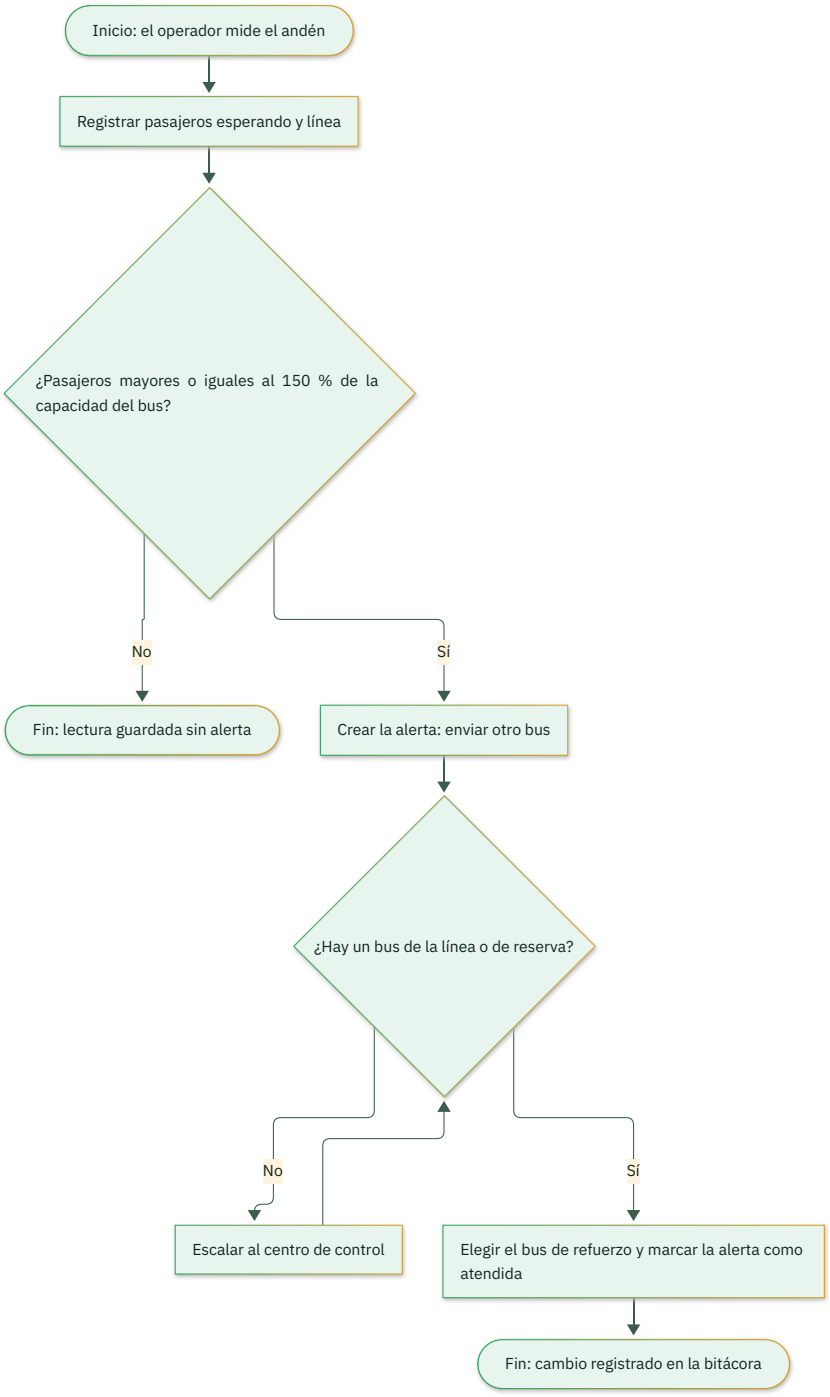
Diagrama de actividades: registro de una visita a una estación



Nota. Se repite para cada estación del recorrido. Con menos del 25 % de ocupación la espera es de 1 + 5 minutos.

Figura 13

Diagrama de actividades: atención de una alerta de capacidad



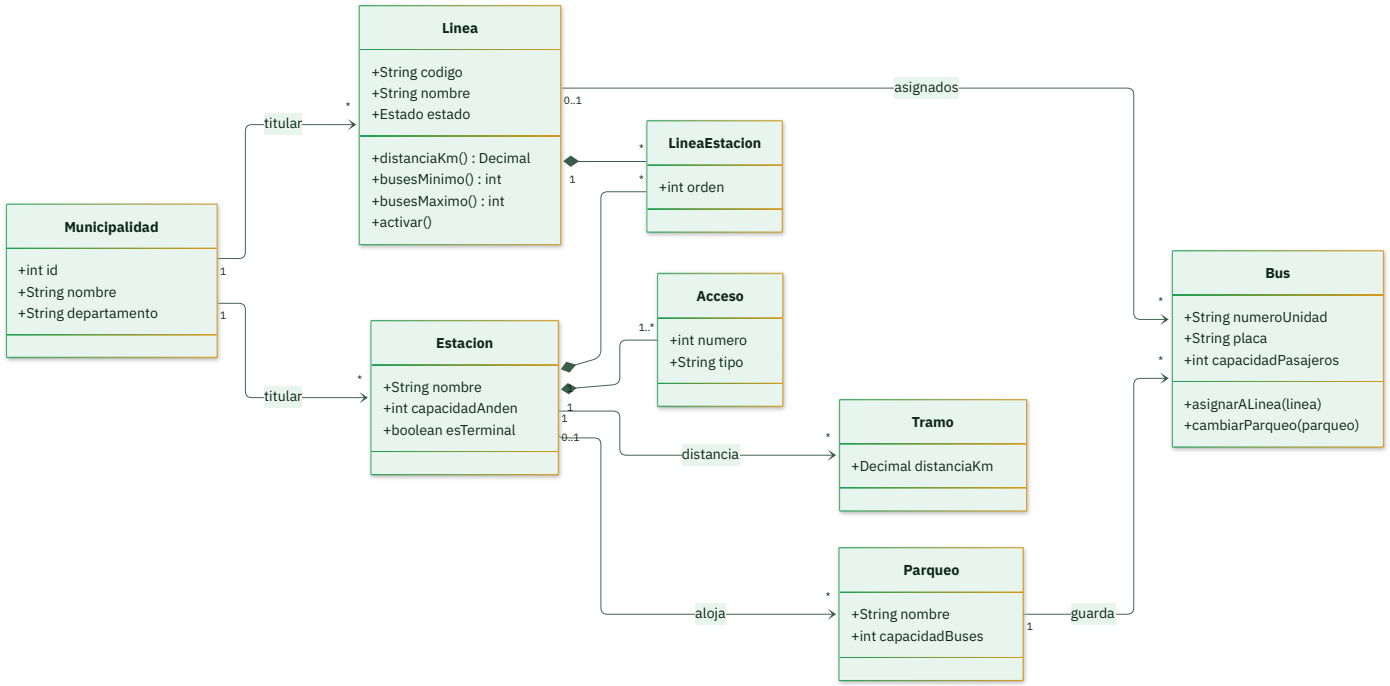
Nota. Si no hay bus disponible se escala al centro de control hasta que lo haya.

12.5 Diagramas de clases

El modelo de clases se presenta en tres partes: la red de transporte (Figura 14), el personal y la estación (Figura 15) y la operación y el recaudo (Figura 16). Las clases se corresponden con las tablas de la base de datos; los métodos indicados son las operaciones que la base de datos ofrece como funciones o que la interfaz invoca.

Figura 14

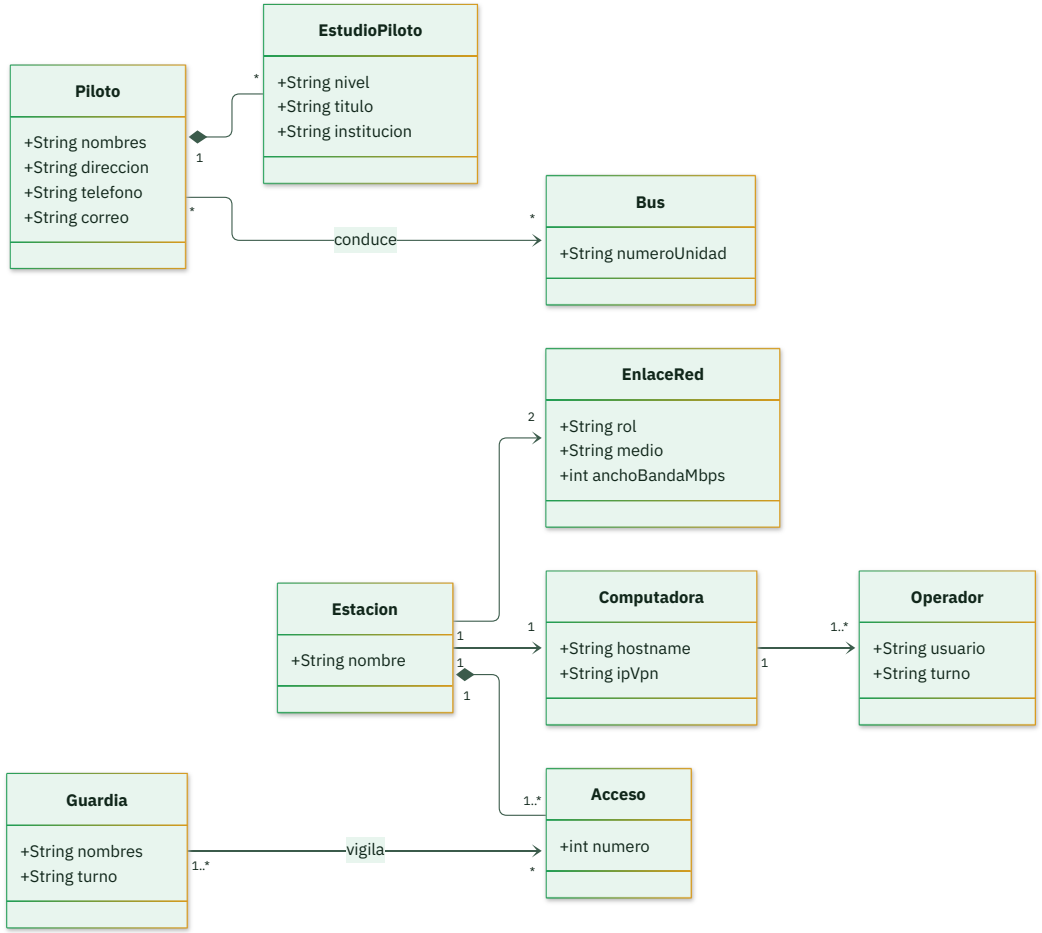
Diagrama de clases: red de transporte



Nota. Las clases LineaEstacion y Tramo materializan las relaciones muchos a muchos con orden y con distancia.

Figura 15

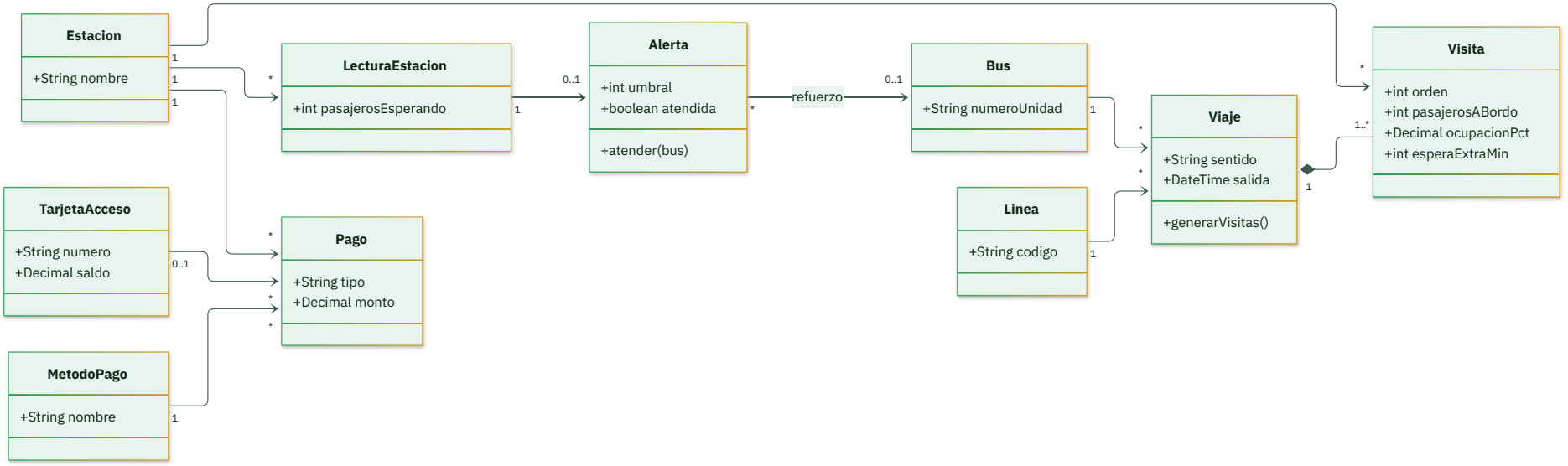
Diagrama de clases: personal, estación y comunicaciones



Nota. Bus, Estacion y Acceso se repiten solo como referencia; sus atributos están en la primera figura.

Figura 16

Diagrama de clases: operación y recaudo



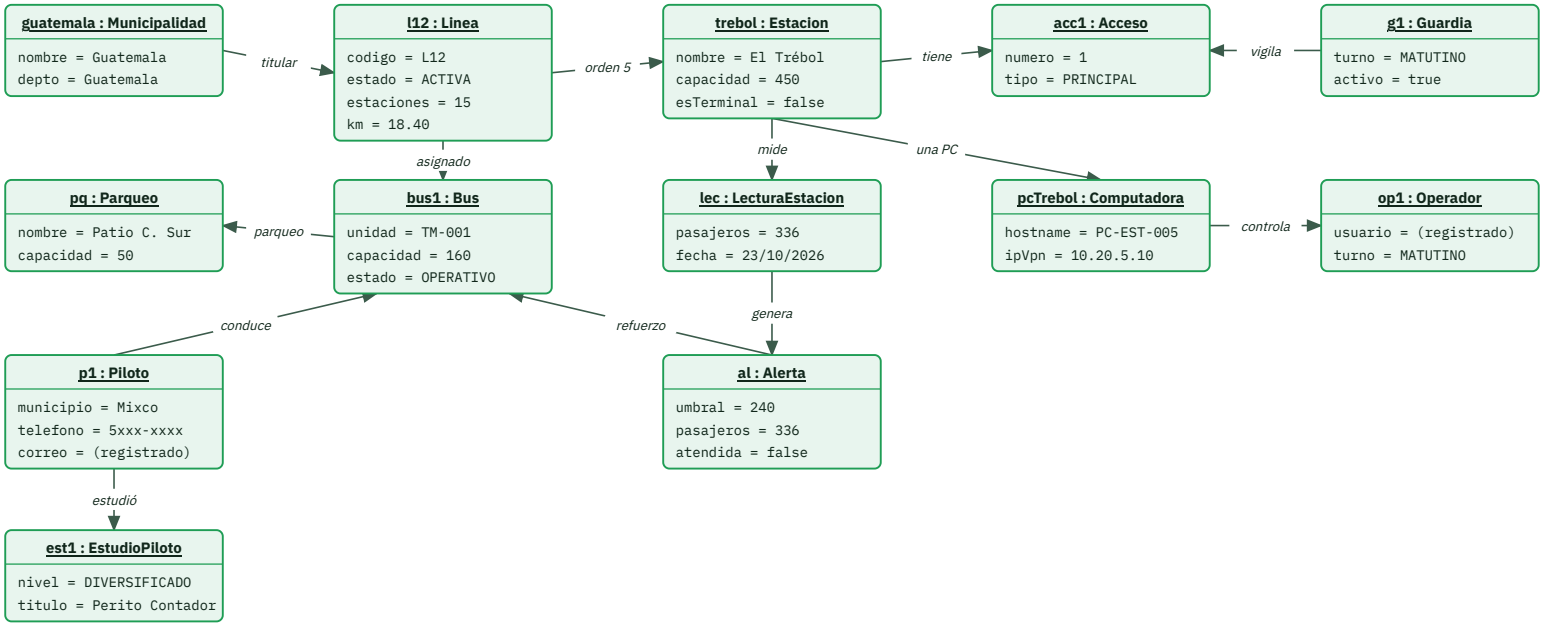
Nota. Bus, Linea y Estacion se repiten solo como referencia.

12.6 Diagrama de objetos

La Figura 17 muestra instancias reales de los datos de demostración: la Línea 12 de la municipalidad de Guatemala, la estación El Trébol con su acceso, su guardia, su computadora y su operador, un bus asignado a la línea con su parqueo, un piloto con su estudio y una lectura de 336 pasajeros que genera una alerta.

Figura 17

Diagrama de objetos con datos del ejemplo



Nota. Los valores corresponden a los datos de demostración; los datos personales aparecen enmascarados.

Capítulo XIII. Conectividad entre estaciones y métodos de pago

13.1 El problema de conectar las estaciones

El enunciado indica que cada estación tendrá una computadora de trabajo con un operador a cargo, y que aún no se ha determinado la mejor forma de conectar las máquinas de un lugar con otro. Esta sección analiza las opciones y toma una decisión. Los requisitos son: transportar datos de operación (lecturas, alertas, pagos) con poco ancho de banda, proteger datos personales, seguir funcionando cuando un enlace falla y costar poco por estación.

13.2 Opciones evaluadas

Tabla 35

Opciones para conectar las computadoras de las estaciones

Opción	Descripción	Ventajas	Desventajas
A. Enlaces privados dedicados	Fibra o MPLS contratados punto a punto entre cada estación y el centro.	Alta calidad y aislamiento.	Costo alto y plazos de instalación largos en 35 estaciones.
B. VPN sobre internet de banda ancha	Cada estación sale a internet por un enlace común y crea un túnel cifrado (WireGuard o IPsec) hacia el centro.	Costo bajo, instalación rápida, cifrado robusto.	Depende del proveedor de internet; necesita respaldo.
C. SD-WAN gestionada	Un proveedor administra los enlaces y elige la mejor ruta.	Gestión centralizada y conmutación automática.	Cuota mensual por sitio y dependencia del proveedor.
D. Solo datos móviles con VPN	Un módem LTE por estación con túnel cifrado.	No requiere instalación fija.	Ancho de banda y cobertura variables; costo por dato.
E. Internet público sin VPN (solo HTTPS)	Cada PC abre la aplicación por HTTPS sin túnel.	El más sencillo.	La base de datos quedaría expuesta; no cumple la protección de datos.

13.3 Matriz de decisión

La Tabla 36 puntúa cada opción de 1 (peor) a 5 (mejor) con pesos que reflejan las prioridades del caso. La opción B, la VPN sobre banda ancha, obtiene el mayor puntaje; para cubrir su principal desventaja, que depende del proveedor de internet, se le añade un enlace móvil de respaldo.

Tabla 36

Matriz de decisión ponderada para la conectividad

Criterio	Peso	A	B	C	D	E
Costo por estación	30 %	1	4	3	3	5
Seguridad	25 %	5	5	4	4	2
Disponibilidad	20 %	5	3	5	3	3
Facilidad de instalación	15 %	1	4	4	5	5
Escalabilidad a nuevas estaciones	10 %	2	5	4	4	5
Puntaje ponderado	100 %	2.90	**4.15**	3.90	3.65	3.85

Nota. Puntuación de 1 a 5, donde 5 es lo mejor. La opción E se descarta además por incumplir la protección de datos.

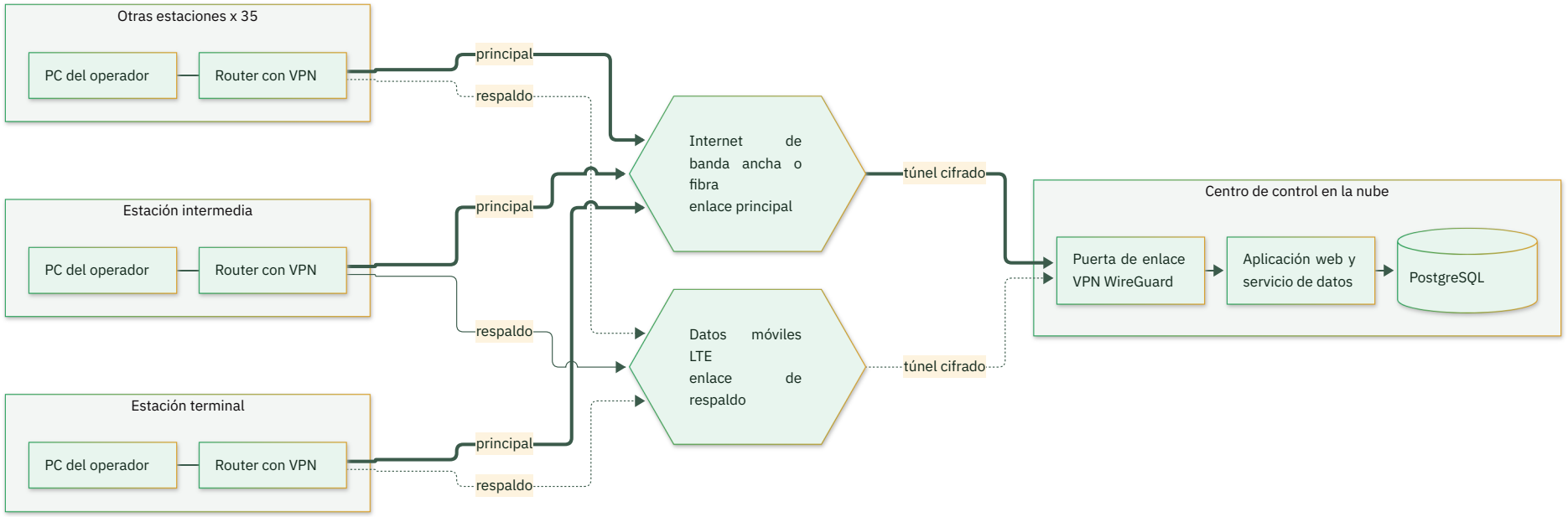
Decisión

Conectar cada estación con una **VPN WireGuard** hacia el centro de control, con **internet de banda ancha como enlace principal** (fibra de 200 Mbps en terminales e intercambios, 50 Mbps en las demás) y **datos móviles LTE de 40 Mbps como respaldo**. Cada PC recibe una dirección fija en la red privada 10.20.0.0/16 (10.20.<n.º de estación>.10).

13.4 Arquitectura de red

Figura 18

Arquitectura de red entre las estaciones y el centro de control



Nota. Línea gruesa: enlace principal. Línea discontinua: enlace de respaldo móvil. Todo el tráfico entre sedes viaja dentro del túnel cifrado.

Los enlaces de cada estación se guardan en la tabla `enlace_red`, lo que permite consultar la conectividad con el reporte R15 y, en la fase de producción, comparar el estado real del túnel con lo registrado. En los datos de demostración hay 70 enlaces (dos por estación) y 35 computadoras, una por estación.

13.5 Funcionamiento ante la pérdida del enlace

Cuando falla el enlace principal, el router pasa al enlace móvil en pocos segundos. Si fallan ambos, la PC de la estación debe seguir trabajando: la aplicación puede registrar lecturas y visitas en una cola local y enviarlas al restablecerse la conexión, de forma ordenada y sin duplicados, con un esquema de almacenamiento y reenvío. Esa cola local es parte de la fase de producción; la demostración actual guarda los datos en el navegador de la propia PC.

13.6 Métodos de pago y tarjeta de acceso

El enunciado menciona los métodos de pago y la tarjeta de acceso. El sistema acepta cinco métodos y cuatro perfiles de tarjeta, que se muestran en la Tabla 37 y la Tabla 38. Las tarifas son parámetros: se guardan en la tabla `tipo_tarjeta` y pueden cambiarse por decisión municipal sin tocar el código.

Tabla 37

Métodos de pago aceptados

Método	Descuenta saldo de la tarjeta	Sirve para recargar	Uso típico
Efectivo en taquilla	No	Sí	Recarga y compra de pasaje
Tarjeta de acceso (saldo prepagado)	Sí	No	Pago del pasaje en el molinete
Tarjeta de débito o crédito	No	Sí	Recarga en taquilla
Billetera móvil	No	Sí	Recarga desde el teléfono
Transferencia bancaria	No	Sí	Recarga de montos mayores

Tabla 38

Perfiles de tarjeta de acceso y tarifa por viaje

Perfil	Tarifa por viaje (GTQ)	Observación
Regular	1.00	Tarifa general
Estudiante	0.50	Descuento por perfil
Adulto mayor	0.25	Descuento por perfil
Persona con discapacidad	0.00	Exento; el viaje se registra en cero

Nota. Valores del estudio original; se deben validar con la autoridad competente.

El disparador `tg_pago_aplica` mantiene el saldo de cada tarjeta: suma las recargas, descuenta la tarifa del perfil en cada cobro con tarjeta y rechaza el cobro cuando el saldo no alcanza o la tarjeta no está activa. Con los datos de demostración, las recargas suman Q 1,480.00, los cobros con tarjeta Q 335.50 y los saldos vigentes Q 1,144.50, que es exactamente la diferencia: la conciliación cuadra.

Capítulo XIV. Aplicación web y base de datos en la nube

14.1 Qué es y cómo se usa

La aplicación es una página web que combina dos espacios de trabajo. El espacio **Red y base de datos** es el sistema del caso: permite ver las líneas y su flota, operar una estación, consultar reportes y revisar la base de datos. El espacio **Centro de control C5 (simulación)** conserva los cinco módulos de simulación del estudio original. Una consola de eventos, visible en todo momento, registra cada acción con su código de regla.

Tabla 39

Pestañas de la aplicación

Espacio	Pestaña	Para quién	Qué permite	Reglas
Red	Líneas y flota	Planificador	Ver líneas, estaciones en orden, km acumulados; asignar o quitar buses; cambiar parqueo; activar o suspender una línea	2, 4–7, 11, 13
Red	Estación del operador	Operador, cajero	Pasajeros en espera y alertas; visitas y espera extra; accesos y guardias; PC, operadores y enlaces; recargas y cobros	3, 10, 14–18
Red	Reportes SQL	Todos	17 reportes con su consulta, exportación a CSV e impresión	8, 9, 12, 13
Red	Base de datos	Administrador	Estado del motor, tablas, pruebas automáticas, consola SQL y restablecimiento	Todas
C5	Flota y terminales	Despachador	Pórticos RFID, despacho y bus bunching (simulación)	RF-01 a 03
C5	Tarjeta y molinete	Pasajero, auditor	Validación y evasión (simulación)	RF-08, 11
C5	Seguridad y alertas	Supervisor	Botón de pánico y seguimiento (simulación)	RF-17, 19
C5	Pasajeros	Operador	Aforo por unidad y ETA (simulación)	RF-04, 06
C5	Operadores	Despachador	Turnos y conducta (simulación)	RF-13, 16

14.2 Tecnologías utilizadas

Tabla 40

Tecnologías de la aplicación web

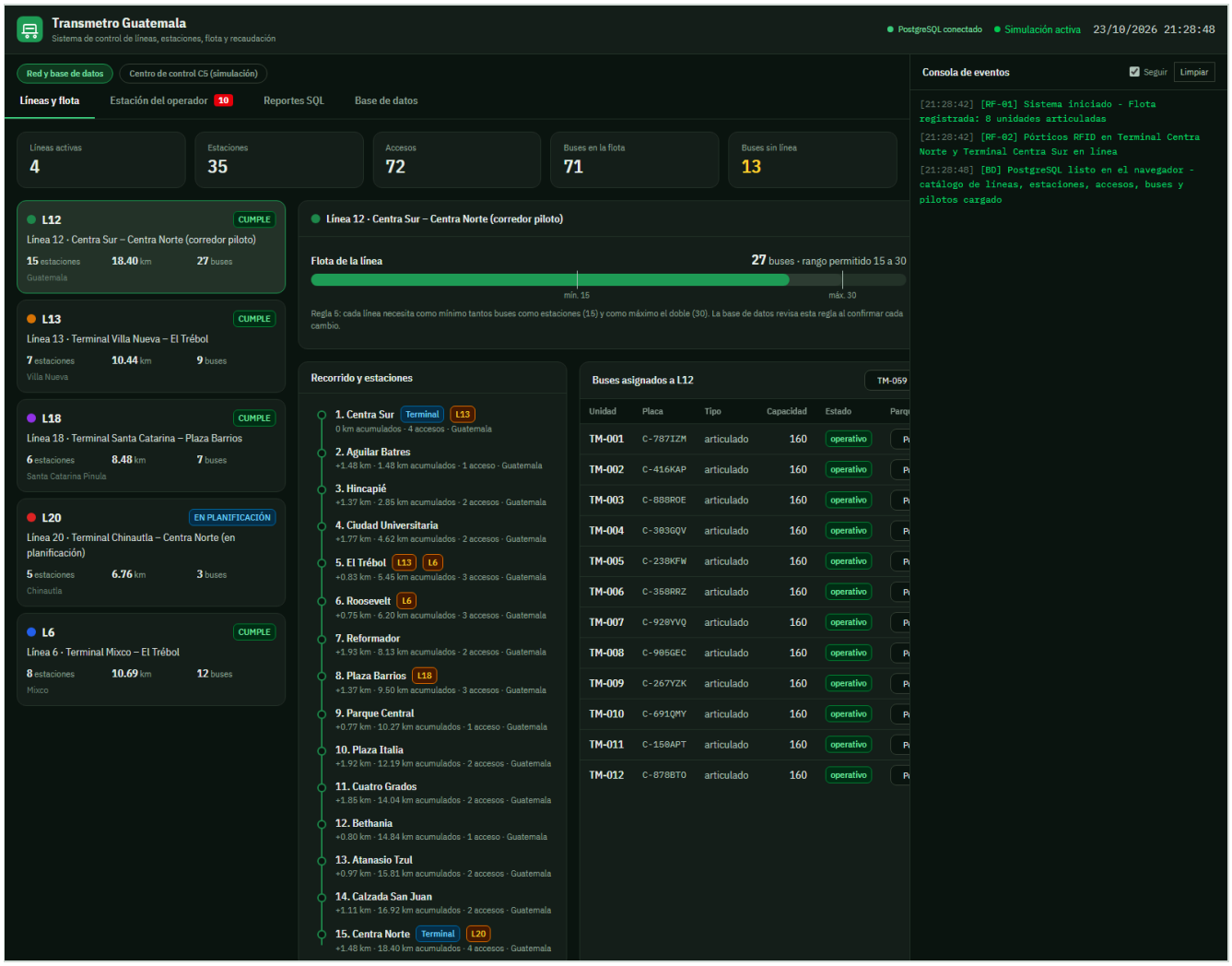
Capa	Tecnología	Función
Interfaz	React 18, Tailwind CSS 4 y Vite 6	Pantallas, estilos y compilación; sin otras dependencias de interfaz
Base de datos	PostgreSQL 17 en WebAssembly (PGlite)	Ejecuta los mismos scripts SQL que un servidor PostgreSQL; guarda los datos en IndexedDB
Tipografía	IBM Plex Sans Condensed y Plex Mono (autohospedadas)	Se incluyen en la aplicación; no se descargan de terceros
Publicación	GitHub Actions y GitHub Pages	Compilan y publican el sitio de forma automática en cada cambio
Documentación	Mermaid, Puppeteer y pptxgenjs	Generan los diagramas, el PDF y las presentaciones

14.3 Pantallas principales

Las figuras siguientes son capturas de la aplicación en funcionamiento, tomadas con los datos de demostración.

Figura 19

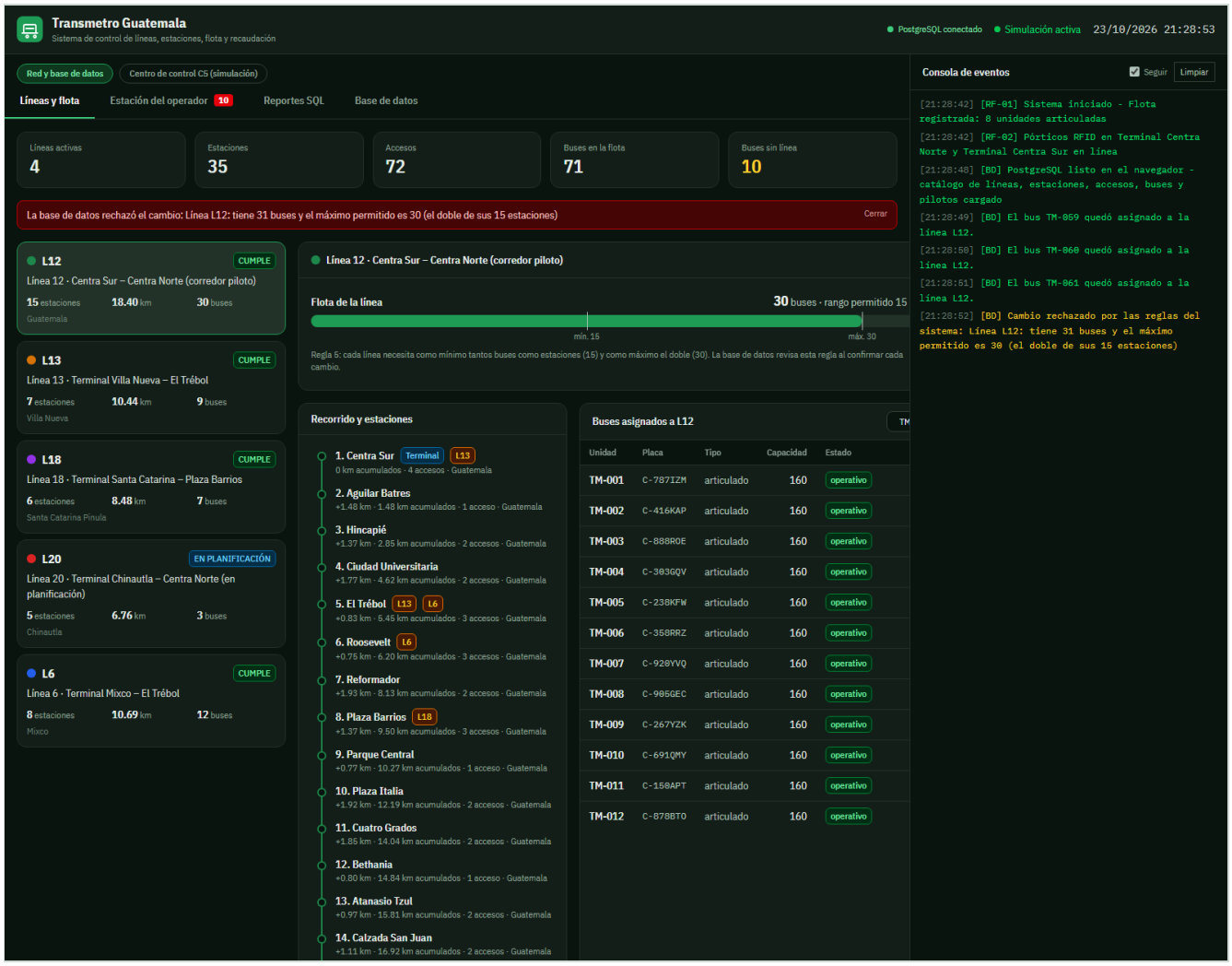
Pestaña «Líneas y flota»: estaciones en orden, distancias y flota de la Línea 12



Nota. El medidor muestra 27 buses dentro del rango permitido de 15 a 30. Las insignias naranja indican estaciones compartidas con otras líneas.

Figura 20

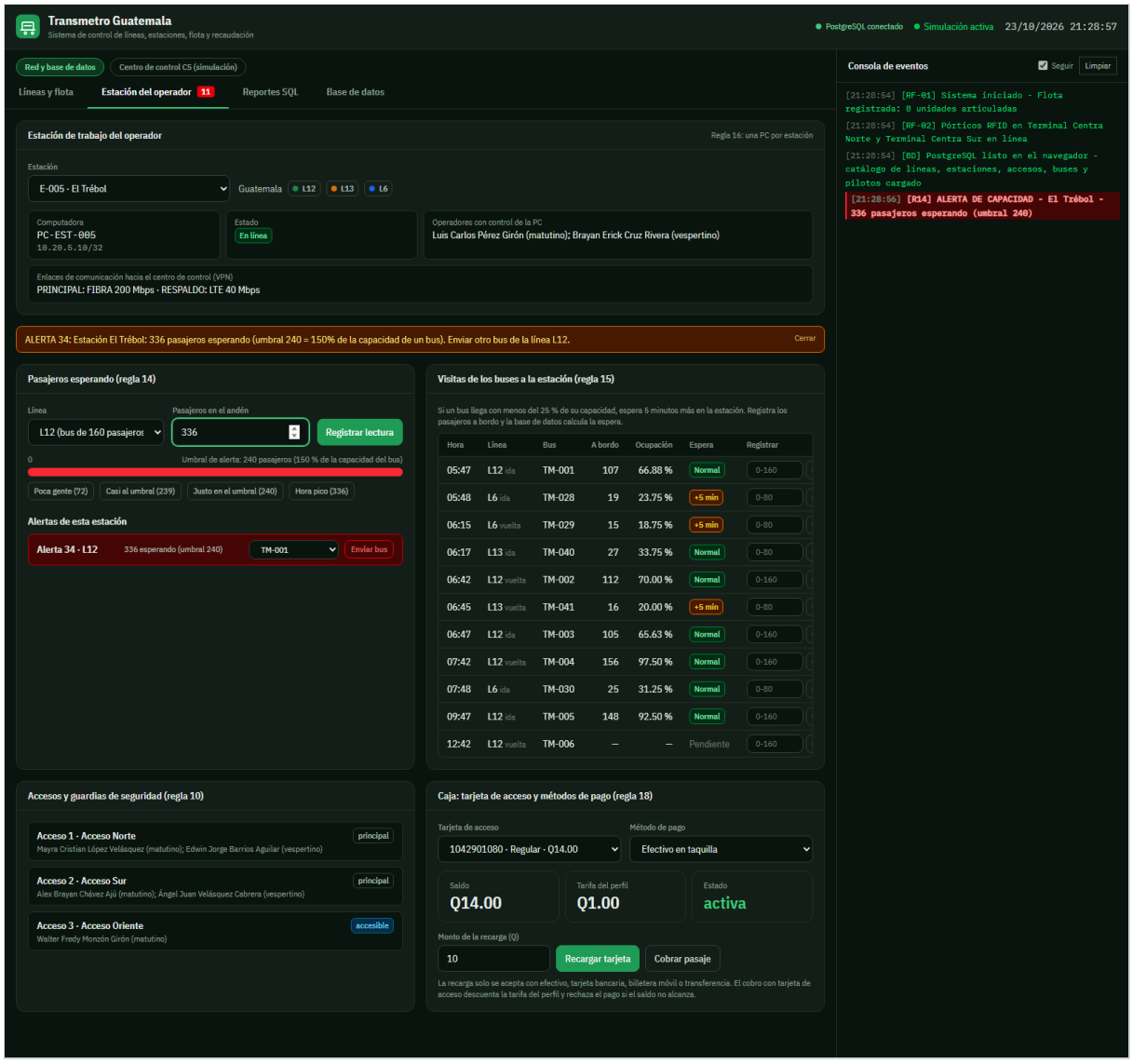
El sistema rechaza el cambio que rompe la regla de flota



Nota. Tras asignar tres buses más, el cuarto supera el máximo de 30 y PostgreSQL revierte el cambio. El mensaje lo produce la base de datos.

Figura 21

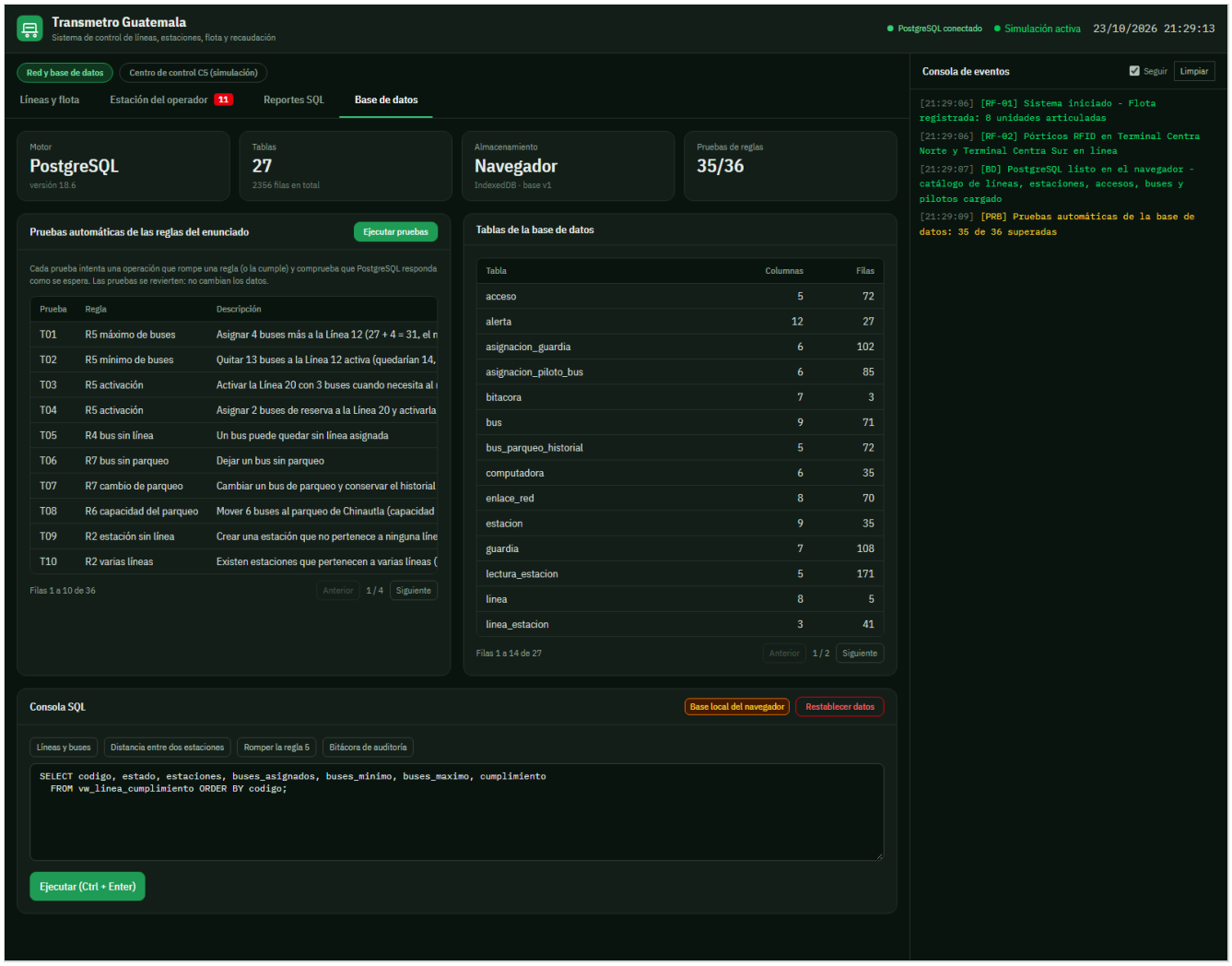
Pestaña «Estación del operador»: alerta de capacidad en El Trébol



Nota. Con 336 pasajeros esperando se supera el umbral de 240 y se genera la alerta 34, que el operador atiende enviando un bus de refuerzo.

Figura 22

Pestaña «Base de datos»: pruebas automáticas de las reglas



Nota. Las pruebas se ejecutan en el navegador sobre el mismo motor. Cada una se revierte y no altera los datos.

14.4 Cambios de diseño respecto a la versión anterior

Además de añadir las pestañas del caso, se rediseñó la apariencia de la aplicación para que sea más clara y consistente. La Tabla 41 resume los cambios.

Tabla 41

Cambios de diseño de la interfaz

Aspecto	Antes	Ahora
Paleta	Grises neutros con blanco como acento	Grises con tinte verde y un verde de marca para acciones y selección
Navegación	Cinco pestañas en una sola barra	Dos espacios de trabajo con sus propias pestañas; la dirección `#reportes/R04` abre una pantalla concreta
Formas	Esquinas rectas	Paneles, botones y campos con esquinas redondeadas
Estado del sistema	Solo la simulación	Indicador de conexión a la base de datos y alertas pendientes en la pestaña de la estación
Tipografía	Fuentes descargadas de internet	IBM Plex incluida en la aplicación: funciona sin conexión
Identidad	Solo texto	Marca gráfica y subtítulo con el alcance del sistema
Tablas de datos	Tablas sencillas en cada pantalla	Componente común con paginación, alineación de cifras y formato de fechas
Mensajes	Solo consola de eventos	Avisos de éxito, advertencia y error junto a la acción, además de la consola

14.5 Centro de control C5: simulación de los módulos ampliados

El segundo espacio de trabajo conserva el prototipo del sistema unificado. Es una simulación que se ejecuta en el navegador sin comunicarse con la base de datos; sirve para validar la lógica y la ergonomía de los módulos que requieren hardware real.

El módulo de flota presenta el inventario de unidades por estado (en patio, en ruta y en mantenimiento), simula el paso por los pórticos RFID de ambas terminales y calcula en cada ciclo la separación entre unidades del mismo sentido. Cuando esta cae por debajo del umbral del 6 % de la ruta, se muestra una alerta de agrupamiento y se ofrece al despachador la retención de la unidad rezagada por diez segundos. El módulo permite además forzar un convoy para demostrar la detección y enviar unidades a mantenimiento, lo que bloquea su despacho.

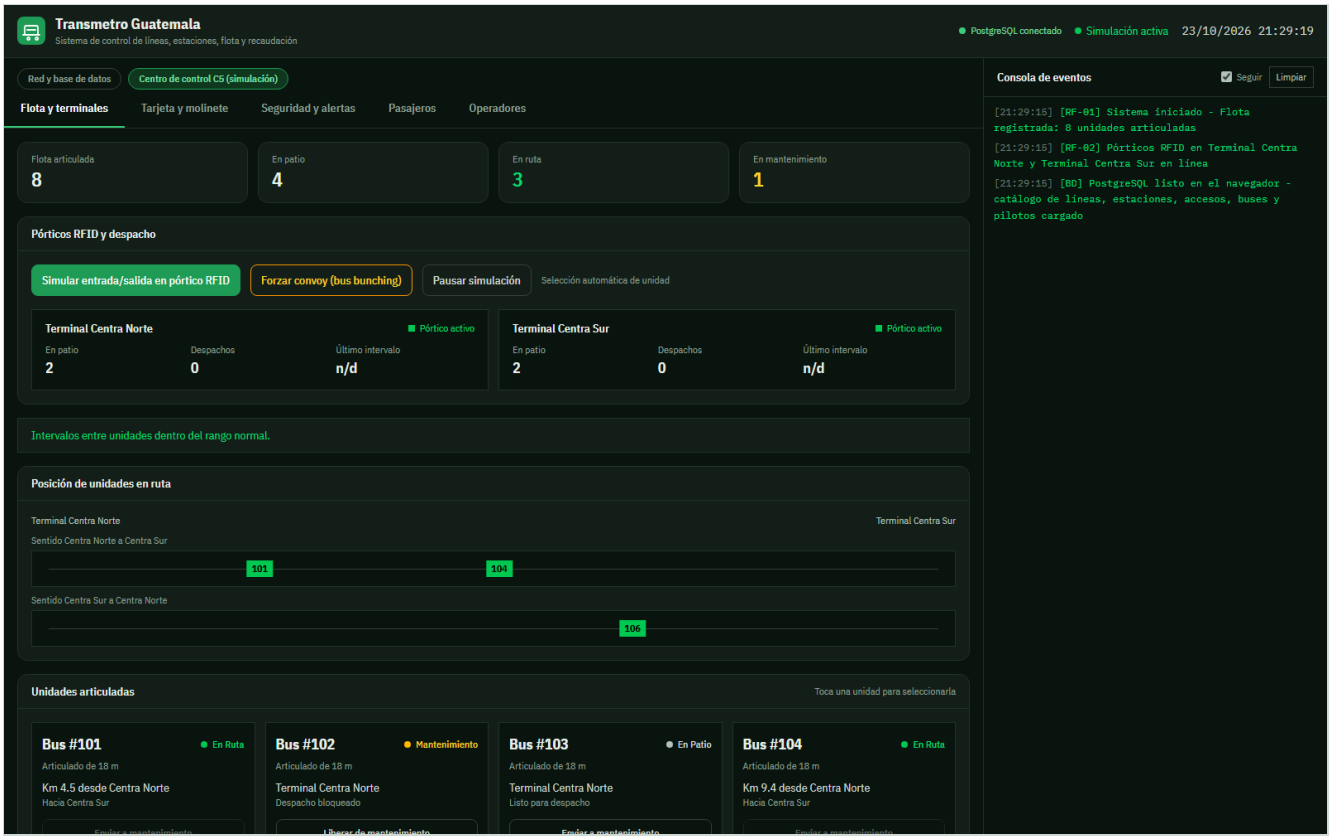
El módulo de tarjeta y molinete simula el ciclo completo de una validación. Se pueden seleccionar los perfiles regular, estudiante y adulto mayor, acercar la tarjeta, recargar saldo y validar el acceso con saldo suficiente o con saldo insuficiente. Cada validación recibe una latencia simulada entre 110 y 280 ms, que se compara con el objetivo de 300 ms, y la animación del molinete refleja el resultado. Una acción independiente simula el salto del molinete, que genera un evento de evasión en el panel de auditoría y una alerta en la consola.

El módulo de seguridad implementa el botón de pánico desde la unidad o desde el andén, con coordenadas GPS que en el caso de un autobús se actualizan en vivo. Mientras la alerta está activa, la interfaz entera cambia de aspecto (borde rojo, banda de emergencia y destello) y el incidente avanza por los estados activa, reconocida, apoyo despachado y cerrada. Una tabla de monitoreo muestra la posición, velocidad y estado de señal de cada unidad, y un historial conserva los incidentes de la sesión.

El módulo de pasajeros simula los sensores ópticos de puerta con un conteo que varía cada segundo sobre una capacidad de 160 pasajeros por unidad, emite una alerta en la consola cuando la ocupación alcanza el 90 % y permite reproducir una hora pico. Además calcula el tiempo estimado de llegada del próximo autobús en cada sentido para las tres estaciones, a partir de la distancia que le falta recorrer y de su velocidad, y lo presenta como lo haría un monitor de andén. El módulo de operadores gestiona el inicio de turno mediante un PIN de cuatro dígitos y la asignación de una unidad disponible, bloquea al piloto tras tres intentos fallidos hasta que un despachador lo habilite y registra infracciones de conducta (frenada brusca, aceleración brusca, exceso de velocidad y apertura de puertas en movimiento) que reducen una puntuación inicial de 100 puntos.

Figura 23

Centro de control C5: flota y terminales con detección de bus bunching



Nota. Simulación en memoria de 8 autobuses articulados sobre una ruta de 18.4 km.

14.6 Publicación en la nube

La aplicación se compila a un sitio estático (HTML, JavaScript y los archivos del motor de base de datos) y se publica en GitHub Pages. Cada vez que se sube un cambio al repositorio, una acción automática compila la aplicación y actualiza el sitio. No hay servidor propio que mantener, el costo es cero y el acceso es por HTTPS.

Tabla 42

Dirección de la aplicación publicada y del repositorio

Elemento	Dirección
Repositorio (código, base de datos y documentos)	https://github.com/william700morales-cloud/transmetro-sistema
Aplicación web publicada	https://william700morales-cloud.github.io/transmetro-sistema/

Nota. Las direcciones se activan cuando el repositorio se crea en GitHub con ese nombre de usuario y se habilita GitHub Pages (Apéndice D).

Dónde está la base de datos

En esta versión la base de datos PostgreSQL se ejecuta **dentro del navegador** de cada persona que abre la aplicación (PGlite) y se carga con los mismos scripts del repositorio. Esto hace que la demostración funcione sin servidor, pero significa que los cambios que haga un usuario no los ven los demás. Para una operación real con varias estaciones, la fase de producción cambia solo la capa de acceso a datos para hablar con un PostgreSQL gestionado en la nube (por ejemplo, un servicio administrado de cualquier proveedor), al que se llega por la VPN descrita en el Capítulo XIII. Los scripts 01 a 05 se ejecutan sin cambios en ese servidor, como se comprobó con PostgreSQL 16.

14.7 Repositorio y estructura de carpetas

```
transmetro-sistema/
├── database/            Scripts SQL 01 a 07 y generador de datos (generar_seed.mjs)
├── src/                 Aplicación React (pestañas, capa de datos, componentes)
├── docs/
│   ├── diagramas/       Fuentes .mmd y SVG/PNG de los diagramas
│   ├── capturas/        Capturas de la aplicación
│   └── entregables/     PDF, presentaciones y guion del video
├── .github/workflows/   Publicación automática en GitHub Pages
├── package.json         Dependencias y comandos (npm run dev, npm run build)
└── README.md            Cómo ejecutar, probar y publicar
```

Capítulo XV. Reportes de estaciones, líneas y buses

El enunciado pide que los reportes de estaciones, líneas y buses asignados se reflejen en la aplicación web y que las consultas queden documentadas. Se construyeron 17 reportes. Cada uno se ve en la pestaña **Reportes SQL** con su consulta, su resultado, la posibilidad de exportarlo a CSV y de imprimirlo. El archivo `database/06_reportes.sql` es la única fuente: la aplicación lee ese mismo archivo, de modo que lo que se muestra en pantalla es exactamente lo que se documenta aquí.

Tabla 43
Reportes del sistema

ID	Reporte	Reglas	Filas	Tiempo (ms)
R01	Líneas del sistema	5, 11, 13	5	12.9
R02	Estaciones de cada línea en orden	1, 12, 13	41	4.3
R03	Estaciones de la red	2, 3, 6, 10, 11, 16	35	7.3
R04	Buses asignados a cada línea	4, 6, 7	58	4.2
R05	Buses sin línea asignada	4, 7	13	1.2
R06	Accesos de cada línea	3, 8, 10	92	4.4
R07	Guardias de seguridad por acceso	10	102	3.5
R08	Distancia entre estaciones contiguas	12	35	5.5
R09	Estaciones de intercambio	2	5	1.9
R10	Pilotos: residencia y comunicación	9	90	11.7
R11	Historial educativo de los pilotos	9	349	7.9
R12	Parqueos y ocupación	6, 7	7	1.6
R13	Alertas de capacidad de estaciones	14	26	2.9
R14	Visitas con espera adicional	1, 15	18	3.7
R15	PC, operador y enlaces de red por estación	16, 17	44	3.2
R16	Recaudo por método de pago	18	8	2
R17	Controles de integridad	2, 5, 10, 12	5	7.7

Nota. Tiempos medidos al ejecutar cada consulta sobre los datos de demostración con PGlite en Node.js.

R01. Líneas del sistema

Cada línea con su municipalidad, número de estaciones, distancia total y buses asignados frente al mínimo y máximo permitidos. *Reglas del enunciado: 5, 11, 13.*

Consulta R01

```
SELECT l.codigo                          AS "Línea",
       l.nombre                          AS "Nombre",
       m.nombre                          AS "Municipalidad",
```

```
       l.estado                              AS "Estado",
       COUNT(DISTINCT le.id_estacion)        AS "Estaciones",
       fn_distancia_linea(l.id_linea)        AS "Distancia (km)",
       COUNT(DISTINCT b.id_bus)              AS "Buses asignados",
       COUNT(DISTINCT le.id_estacion)        AS "Mínimo de buses",
       2 * COUNT(DISTINCT le.id_estacion)    AS "Máximo de buses"
  FROM linea l
  JOIN municipalidad m        ON m.id_municipalidad = l.id_municipalidad
  LEFT JOIN linea_estacion le ON le.id_linea = l.id_linea
  LEFT JOIN bus b             ON b.id_linea = l.id_linea
 GROUP BY l.id_linea, l.codigo, l.nombre, m.nombre, l.estado
 ORDER BY l.codigo;
```

Tabla 44

Resultado del reporte R01 (primeras 3 de 5 filas)

Línea	Nombre	Municipalidad	Estado	Estaciones	Distancia (km)	Buses asignados	Mínimo de buses	Máximo de buses
L12	Línea 12 · Cent…	Guatemala	ACTIVA	15	18.40	27	15	30
L13	Línea 13 · Term…	Villa Nueva	ACTIVA	7	10.44	9	7	14
L18	Línea 18 · Term…	Santa Catarina …	ACTIVA	6	8.48	7	6	12

Nota. 5 filas en 12.9 ms.

R02. Estaciones de cada línea en orden

Recorrido de cada línea: orden de las estaciones, distancia desde la anterior y distancia acumulada. *Reglas del enunciado: 1, 12, 13.*

Consulta R02

```
SELECT linea AS "Línea", orden AS "Orden", estacion AS "Estación",
       km_desde_anterior AS "Km desde la anterior", km_acumulado AS "Km acumulados"
  FROM vw_linea_estaciones
 ORDER BY linea, orden;
```

Tabla 45

Resultado del reporte R02 (primeras 3 de 41 filas)

Línea	Orden	Estación	Km desde la anterior	Km acumulados
L12	1	Centra Sur	0	0
L12	2	Aguilar Batres	1.48	1.48
L12	3	Hincapié	1.37	2.85

Nota. 41 filas en 4.3 ms.

R03. Estaciones de la red

Cada estación con su municipalidad, líneas a las que pertenece, accesos, guardias, parqueos, PC de trabajo y operadores. *Reglas del enunciado: 2, 3, 6, 10, 11, 16.*

Consulta R03

```
SELECT e.codigo AS "Código", e.nombre AS "Estación", m.nombre AS "Municipalidad",
       (SELECT string_agg(l.codigo, ', ' ORDER BY l.codigo)
          FROM linea_estacion le JOIN linea l USING (id_linea)
         WHERE le.id_estacion = e.id_estacion)                        AS "Líneas",
       (SELECT count(*) FROM acceso a WHERE a.id_estacion = e.id_estacion) AS "Accesos",
       (SELECT count(*) FROM asignacion_guardia ag JOIN acceso a USING (id_acceso)
         WHERE a.id_estacion = e.id_estacion AND ag.fecha_fin IS NULL)  AS "Guardias",
       (SELECT count(*) FROM parqueo p WHERE p.id_estacion = e.id_estacion) AS "Parqueos",
       c.hostname AS "PC de trabajo"
  FROM estacion e
  JOIN municipalidad m      ON m.id_municipalidad = e.id_municipalidad
  LEFT JOIN computadora c   ON c.id_estacion = e.id_estacion
 ORDER BY e.codigo;
```

Tabla 46

Resultado del reporte R03 (primeras 3 de 35 filas)

Código	Estación	Municipalidad	Líneas	Accesos	Guardias	Parqueos	PC de trabajo
E-001	Centra Sur	Guatemala	L12, L13	4	8	1	PC-EST-001
E-002	Aguilar Batres	Guatemala	L12	1	1	0	PC-EST-002
E-003	Hincapié	Guatemala	L12	2	2	0	PC-EST-003

Nota. 35 filas en 7.3 ms.

R04. Buses asignados a cada línea

Detalle de los buses de cada línea con su parqueo y los pilotos vigentes por turno. *Reglas del enunciado: 4, 6, 7.*

Consulta R04

```
SELECT l.codigo AS "Línea", b.numero_unidad AS "Unidad", b.placa AS "Placa", b.tipo AS "Tipo",
       b.capacidad_pasajeros AS "Capacidad", b.estado AS "Estado", p.nombre AS "Parqueo",
       (SELECT string_agg(pi.nombres || ' ' || pi.apellidos || ' (' || lower(ap.turno) || ')', '; ' ORDER BY ap.turno)
          FROM asignacion_piloto_bus ap JOIN piloto pi USING (id_piloto)
         WHERE ap.id_bus = b.id_bus AND ap.fecha_fin IS NULL) AS "Pilotos"
  FROM bus b
  JOIN linea l   ON l.id_linea = b.id_linea
  JOIN parqueo p ON p.id_parqueo = b.id_parqueo
 ORDER BY l.codigo, b.numero_unidad;
```

Tabla 47

Resultado del reporte R04 (primeras 3 de 58 filas)

Línea	Unidad	Placa	Tipo	Capacidad	Estado	Parqueo	Pilotos
L12	TM-001	C-787IZM	ARTICULADO	160	OPERATIVO	Patio Centra Sur	Pedro Pedro Castillo …
L12	TM-002	C-416KAP	ARTICULADO	160	OPERATIVO	Patio Centra Sur	Byron Alex Cifuentes …
L12	TM-003	C-888ROE	ARTICULADO	160	OPERATIVO	Patio Centra Sur	Edwin Fredy López Ac …

Nota. 58 filas en 4.2 ms.

R05. Buses sin línea asignada

Buses de reserva: no están en ninguna línea, pero siempre tienen parqueo. *Reglas del enunciado: 4, 7.*

Consulta R05

```
SELECT b.numero_unidad AS "Unidad", b.placa AS "Placa", b.tipo AS "Tipo",
       b.capacidad_pasajeros AS "Capacidad", b.estado AS "Estado", p.nombre AS "Parqueo"
  FROM bus b
  JOIN parqueo p ON p.id_parqueo = b.id_parqueo
 WHERE b.id_linea IS NULL
 ORDER BY b.numero_unidad;
```

Tabla 48

Resultado del reporte R05 (primeras 3 de 13 filas)

Unidad	Placa	Tipo	Capacidad	Estado	Parqueo
TM-059	C-749NMW	ARTICULADO	160	OPERATIVO	Taller y parqueo central
TM-060	C-820GBT	ARTICULADO	160	OPERATIVO	Patio Centra Sur
TM-061	C-469UQT	ARTICULADO	160	MANTENIMIENTO	Patio Centra Sur

Nota. 13 filas en 1.2 ms.

R06. Accesos de cada línea

Todos los accesos de todas las estaciones de cada línea, con los guardias vigentes de cada uno. *Reglas del enunciado: 3, 8, 10.*

Consulta R06

```
SELECT l.codigo AS "Línea", le.orden AS "Orden", e.nombre AS "Estación", a.numero AS "Acceso",
       a.nombre AS "Nombre del acceso", a.tipo AS "Tipo", count(ag.id_asignacion) AS "Guardias vigentes"
  FROM linea l
  JOIN linea_estacion le ON le.id_linea = l.id_linea
  JOIN estacion e        ON e.id_estacion = le.id_estacion
  JOIN acceso a          ON a.id_estacion = e.id_estacion
  LEFT JOIN asignacion_guardia ag ON ag.id_acceso = a.id_acceso AND ag.fecha_fin IS NULL
 GROUP BY l.codigo, le.orden, e.nombre, a.numero, a.nombre, a.tipo
 ORDER BY l.codigo, le.orden, a.numero;
```

Tabla 49

Resultado del reporte R06 (primeras 3 de 92 filas)

Línea	Orden	Estación	Acceso	Nombre del acceso	Tipo	Guardias vigentes
L12	1	Centra Sur	1	Acceso Norte	PRINCIPAL	2
L12	1	Centra Sur	2	Acceso Sur	PRINCIPAL	2
L12	1	Centra Sur	3	Acceso Oriente	SECUNDARIO	2

Nota. 92 filas en 4.4 ms.

R07. Guardias de seguridad por acceso

Guardias asignados a cada acceso, con su turno. *Reglas del enunciado: 10.*

Consulta R07

```
SELECT e.nombre AS "Estación", a.numero AS "Acceso", g.nombres || ' ' || g.apellidos AS "Guardia",
       ag.turno AS "Turno", g.telefono AS "Teléfono", ag.fecha_inicio AS "Desde"
  FROM asignacion_guardia ag
  JOIN guardia g  ON g.id_guardia = ag.id_guardia
  JOIN acceso a   ON a.id_acceso = ag.id_acceso
  JOIN estacion e ON e.id_estacion = a.id_estacion
 WHERE ag.fecha_fin IS NULL
 ORDER BY e.nombre, a.numero, ag.turno;
```

Tabla 50

Resultado del reporte R07 (primeras 3 de 102 filas)

Estación	Acceso	Guardia	Turno	Teléfono	Desde
Aguilar Batres	1	Marvin Ricardo Ajú Cifuentes	MATUTINO	5411-6406	04/10/2024
Atanasio Tzul	1	Walter Juan Ixcoy Morales	MATUTINO	4992-7788	28/10/2025
Atanasio Tzul	2	Erick Luis Orellana Coc	MATUTINO	4665-6798	10/01/2026

Nota. 102 filas en 3.5 ms.

R08. Distancia entre estaciones contiguas

Distancia de cada tramo entre dos estaciones, en kilómetros, y las líneas que lo recorren. *Reglas del enunciado: 12.*

Consulta R08

```
SELECT ea.nombre AS "Estación A", eb.nombre AS "Estación B", t.distancia_km AS "Distancia (km)",
       (SELECT string_agg(DISTINCT l.codigo, ', ')
          FROM linea_estacion x JOIN linea_estacion y ON y.id_linea = x.id_linea AND y.orden = x.orden + 1
          JOIN linea l ON l.id_linea = x.id_linea
         WHERE LEAST(x.id_estacion, y.id_estacion) = t.id_estacion_a
           AND GREATEST(x.id_estacion, y.id_estacion) = t.id_estacion_b) AS "Líneas"
  FROM tramo t
  JOIN estacion ea ON ea.id_estacion = t.id_estacion_a
  JOIN estacion eb ON eb.id_estacion = t.id_estacion_b
 ORDER BY t.id_estacion_a, t.id_estacion_b;
```

Tabla 51

Resultado del reporte R08 (primeras 3 de 35 filas)

Estación A	Estación B	Distancia (km)	Líneas
Centra Sur	Aguilar Batres	1.48	L12
Centra Sur	El Trébol	1.03	L13
Centra Sur	San Miguel Petapa	1.65	L13

Nota. 35 filas en 5.5 ms.

R09. Estaciones de intercambio

Estaciones que pertenecen a más de una línea. *Reglas del enunciado: 2.*

Consulta R09

```
SELECT e.nombre AS "Estación", count(*) AS "Número de líneas",
       string_agg(l.codigo, ', ' ORDER BY l.codigo) AS "Líneas"
  FROM estacion e
  JOIN linea_estacion le ON le.id_estacion = e.id_estacion
  JOIN linea l           ON l.id_linea = le.id_linea
 GROUP BY e.id_estacion, e.nombre
HAVING count(*) > 1
 ORDER BY count(*) DESC, e.nombre;
```

Tabla 52

Resultado del reporte R09 (primeras 3 de 5 filas)

Estación	Número de líneas	Líneas
El Trébol	3	L12, L13, L6
Centra Norte	2	L12, L20
Centra Sur	2	L12, L13

Nota. 5 filas en 1.9 ms.

R10. Pilotos: residencia y comunicación

Datos personales, de residencia y de contacto de los pilotos (DPI enmascarado) y su mayor nivel de estudio. *Reglas del enunciado: 9.*

Consulta R10

```
SELECT piloto AS "Piloto", dpi_enmascarado AS "DPI", edad AS "Edad", licencia_tipo AS "Licencia",
       direccion_residencia AS "Dirección", zona AS "Zona", municipio_residencia AS "Municipio",
       telefono_movil AS "Teléfono móvil", correo AS "Correo", mayor_estudio AS "Mayor estudio",
       bus_actual AS "Bus actual"
  FROM vw_pilotos_ficha
 ORDER BY piloto;
```

Tabla 53

Resultado del reporte R10 (primeras 3 de 90 filas)

Piloto	DPI	Edad	Licencia	Dirección	Zona	Municipio	Teléfono móvil	Correo	Mayor estudio	Bus actual
Ana Fredy Xicay…	********* 1427	27	B	2a avenida 21-6…	6	Guatemala	3113-7157	ana.xicay55@cor…	Ciclo básico	TM-055
Ana Pedro Cabre…	********* 7351	51	B	6a avenida 30-3…	12	Villa Nueva	3261-3608	ana.cabrera31@c…	Bachiller en Ci…	TM-031
Ana Ricardo Vel…	********* 8839	33	C	12a avenida 1-7…	1	Santa Catarina …	3850-7164	ana.velasquez70…	Magisterio de E…	TM-012

Nota. 90 filas en 11.7 ms.

R11. Historial educativo de los pilotos

Cada estudio registrado de cada piloto, en orden cronológico. *Reglas del enunciado: 9.*

Consulta R11

```
SELECT pi.nombres || ' ' || pi.apellidos AS "Piloto", pe.nivel AS "Nivel", pe.titulo AS "Título",
       pe.institucion AS "Institución", pe.anio_inicio AS "Inicio", pe.anio_fin AS "Fin",
       CASE WHEN pe.concluido THEN 'Concluido' ELSE 'En curso' END AS "Estado"
  FROM piloto_estudio pe
  JOIN piloto pi ON pi.id_piloto = pe.id_piloto
 ORDER BY pi.apellidos, pi.nombres, pe.anio_inicio;
```

Tabla 54

Resultado del reporte R11 (primeras 3 de 349 filas)

Piloto	Nivel	Título	Institución	Inicio	Fin	Estado
Lucía Hugo Ac Chávez	PRIMARIA	Primaria completa	Escuela Nacional de V…	2004	2010	Concluido
Lucía Hugo Ac Chávez	BASICO	Ciclo básico	Instituto Nacional de…	2010	2013	Concluido
Lucía Hugo Ac Chávez	DIVERSIFICADO	Magisterio de Educaci…	Escuela Normal	2013	2016	Concluido

Nota. 349 filas en 7.9 ms.

R12. Parqueos y ocupación

Capacidad de cada parqueo, buses que alberga y estación a la que pertenece (si la tiene). *Reglas del enunciado: 6, 7.*

Consulta R12

```
SELECT p.codigo AS "Código", p.nombre AS "Parqueo", COALESCE(e.nombre, '(sin estación)') AS "Estación",
       p.capacidad_buses AS "Capacidad", count(b.id_bus) AS "Buses",
       p.capacidad_buses - count(b.id_bus) AS "Espacios libres"
```

```
  FROM parqueo p
  LEFT JOIN estacion e ON e.id_estacion = p.id_estacion
  LEFT JOIN bus b      ON b.id_parqueo = p.id_parqueo
GROUP BY p.id_parqueo, p.codigo, p.nombre, e.nombre, p.capacidad_buses
ORDER BY p.codigo;
```

Tabla 55

Resultado del reporte R12 (primeras 3 de 7 filas)

Código	Parqueo	Estación	Capacidad	Buses	Espacios libres
PQ-CH	Parqueo Terminal Chinautla	Terminal Chinautla	8	3	5
PQ-CN	Patio Centra Norte	Centra Norte	40	13	27
PQ-CS	Patio Centra Sur	Centra Sur	50	21	29

Nota. 7 filas en 1.6 ms.

R13. Alertas de capacidad de estaciones

Alertas generadas cuando esperan 150 % o más de la capacidad de un bus, y bus de refuerzo enviado. *Reglas del enunciado: 14.*

Consulta R13

```
SELECT id_alerta AS "Alerta", fecha AS "Fecha y hora", linea AS "Línea", estacion AS "Estación",
       pasajeros_esperando AS "Pasajeros esperando", umbral AS "Umbral", pct_capacidad_bus AS "% de la capacidad del
bus",
       estado AS "Estado", bus_refuerzo AS "Bus de refuerzo"
  FROM vw_alertas_capacidad
 ORDER BY fecha, id_alerta;
```

Tabla 56

Resultado del reporte R13 (primeras 3 de 26 filas)

Alerta	Fecha y hora	Línea	Estación	Pasajeros esperando	Umbral	% de la capacidad del bus	Estado	Bus de refuerzo
1	23/10 05:10	L12	Cuatro Grados	262	240	164	ATENDIDA	TM-001
2	23/10 05:24	L12	Reformador	264	240	165	ATENDIDA	TM-008
3	23/10 05:53	L18	Oakland	139	120	174	ATENDIDA	TM-049

Nota. 26 filas en 2.9 ms.

R14. Visitas con espera adicional

Visitas de los buses a estaciones donde iban con menos del 25 % de su capacidad y esperan 5 minutos más. *Reglas del enunciado: 1, 15.*

Consulta R14

```
SELECT linea AS "Línea", bus AS "Bus", sentido AS "Sentido", orden AS "Orden", estacion AS "Estación",
       hora_programada AS "Hora programada", pasajeros_a_bordo AS "Pasajeros", capacidad_pasajeros AS "Capacidad",
       ocupacion_pct AS "Ocupación %", espera_base_min + espera_extra_min AS "Espera total (min)",
       salida_estimada AS "Salida estimada"
  FROM vw_visitas_espera
 WHERE espera_extra_min > 0
 ORDER BY hora_programada, bus;
```

Tabla 57

Resultado del reporte R14 (primeras 3 de 18 filas)

Línea	Bus	Sentido	Orden	Estación	Hora programada	Pasajeros	Capacidad	Ocupación %	Espera total (min)	Salida estimada
L12	TM-001	IDA	1	Centra Sur	23/10 05:30	32	160	20.00	6	23/10 05:36
L6	TM-028	IDA	8	El Trébol	23/10 05:48	19	80	23.75	6	23/10 05:54
L12	TM-002	VUELTA	1	Centra Norte	23/10 06:00	35	160	21.88	6	23/10 06:06

Nota. 18 filas en 3.7 ms.

R15. PC, operador y enlaces de red por estación

Computadora de trabajo de cada estación, operadores asignados y enlaces de comunicación. *Reglas del enunciado: 16, 17.*

Consulta R15

```
SELECT estacion AS "Estación", hostname AS "PC", ip_vpn::TEXT AS "IP en la VPN", estado_pc AS "Estado de la PC",
       operador AS "Operador", turno AS "Turno", enlaces AS "Enlaces"
  FROM vw_operadores_estacion
 ORDER BY estacion, turno;
```

Tabla 58

Resultado del reporte R15 (primeras 3 de 44 filas)

Estación	PC	IP en la VPN	Estado de la PC	Operador	Turno	Enlaces
Aguilar Batres	PC-EST-002	10.20.2.10/32	EN_LINEA	Jorge Otto Tzoc Herná…	MATUTINO	PRINCIPAL: BANDA_ANCH…
Atanasio Tzul	PC-EST-013	10.20.13.10/32	EN_LINEA	Selvin Fredy Girón Ac	MATUTINO	PRINCIPAL: BANDA_ANCH…
Bethania	PC-EST-012	10.20.12.10/32	EN_LINEA	Mynor Brayan López Ci…	MATUTINO	PRINCIPAL: BANDA_ANCH…

Nota. 44 filas en 3.2 ms.

R16. Recaudo por método de pago

Operaciones y monto total por método de pago y tipo (recarga o cobro de pasaje). *Reglas del enunciado: 18.*

Consulta R16

```
SELECT mp.nombre AS "Método de pago", p.tipo AS "Tipo", count(*) AS "Operaciones", sum(p.monto) AS "Total (Q)"
  FROM pago p
  JOIN metodo_pago mp ON mp.id_metodo_pago = p.id_metodo_pago
 GROUP BY mp.nombre, p.tipo
 ORDER BY mp.nombre, p.tipo;
```

Tabla 59

Resultado del reporte R16 (primeras 3 de 8 filas)

Método de pago	Tipo	Operaciones	Total (Q)
Billetera móvil	COBRO_PASAJE	30	30.00
Billetera móvil	RECARGA	11	200.00
Efectivo en taquilla	COBRO_PASAJE	138	138.00

Nota. 8 filas en 2 ms.

R17. Controles de integridad

Cantidad de incumplimientos de cada regla; todos deben estar en cero. *Reglas del enunciado: 2, 5, 10, 12.*

Consulta R17

```
SELECT 'Accesos sin guardia vigente (regla 10)' AS "Control", count(*) AS "Incumplimientos" FROM vw_accesos_sin_guardia
UNION ALL
SELECT 'Estaciones sin línea (regla 2)', count(*) FROM estacion e
 WHERE NOT EXISTS (SELECT 1 FROM linea_estacion le WHERE le.id_estacion = e.id_estacion)
UNION ALL
SELECT 'Líneas activas fuera del rango de buses (regla 5)', count(*) FROM vw_linea_cumplimiento
 WHERE estado = 'ACTIVA' AND cumplimiento <> 'CUMPLE'
UNION ALL
SELECT 'Tramos contiguos sin distancia (regla 12)', count(*) FROM vw_tramos_faltantes
UNION ALL
SELECT 'Buses sin parqueo (regla 7)', count(*) FROM bus WHERE id_parqueo IS NULL;
```

Tabla 60

Resultado del reporte R17 (primeras 3 de 5 filas)

Control	Incumplimientos
Accesos sin guardia vigente (regl…	0
Estaciones sin línea (regla 2)	0
Líneas activas fuera del rango de…	0

Nota. 5 filas en 7.7 ms.

Capítulo XVI. Implementación de estándares de seguridad

La seguridad del sistema se diseñó con tres referencias: la lista OWASP Top 10 de riesgos de aplicaciones web (OWASP Foundation, 2021), la norma ISO/IEC 27001:2022 para la gestión de seguridad de la información (ISO, 2022) y el principio de mínimo privilegio. Esta sección describe qué se implementó en esta versión y qué queda pendiente para la fase de producción.

16.1 Seguridad según OWASP Top 10

Tabla 61

Riesgos del OWASP Top 10 (2021) y medidas del sistema

Riesgo	Medida implementada	Estado
A01 Pérdida de control de acceso	Tres roles de base de datos con privilegios mínimos; el rol de consulta no puede leer el DPI; el operador no puede borrar buses (pruebas T31 a T33).	**Implementado en la BD**
A02 Fallas criptográficas	Sitio servido por HTTPS; VPN WireGuard entre estaciones; DPI enmascarado en vistas; cifrado de disco en el servidor de producción.	**Implementado** / producción
A03 Inyección	Todas las consultas de la aplicación usan parámetros; la consola SQL libre solo actúa sobre la base local de demostración.	**Implementado**
A04 Diseño inseguro	Las reglas del servicio se aplican en el motor y no en la interfaz; modelo de amenazas en la Tabla 63.	**Implementado**
A05 Configuración de seguridad incorrecta	Sin credenciales en el repositorio; privilegios revocados sobre la bitácora; dependencias mínimas.	**Implementado**
A06 Componentes vulnerables	Pocas dependencias y análisis con `npm audit` sin vulnerabilidades conocidas al momento de la entrega.	**Verificado**
A07 Fallas de identificación y autenticación	No hay inicio de sesión en esta versión; se recomienda un proveedor OpenID Connect con doble factor (RF-13 y RF-20).	**Pendiente**
A08 Fallas de integridad de software y datos	Bitácora que no admite modificar ni borrar; dependencias fijadas por archivo de bloqueo; publicación automatizada.	**Implementado**
A09 Fallas de registro y monitoreo	Bitácora de cambios en la base y consola de eventos en la aplicación.	**Implementado**
A10 Falsificación de peticiones del lado del servidor	No aplica: la aplicación no tiene servidor que haga peticiones.	No aplica

16.2 Controles de ISO/IEC 27001:2022

Tabla 62

Controles del Anexo A de ISO/IEC 27001:2022 aplicados

Control	Descripción	Implementación en el sistema
5.15	Control de acceso	Roles `tm_consulta`, `tm_operador` y `tm_admin` con permisos por tabla y por columna.
5.34	Privacidad y protección de datos personales	DPI enmascarado, mínimo de columnas visibles para consulta, datos de demostración ficticios.
8.2	Derechos de acceso privilegiado	Solo el rol administrador cambia la estructura y las asignaciones de flota.
8.5	Autenticación segura	Pendiente en la aplicación (ver A07).
8.13	Copias de seguridad	En producción: volcado diario con `pg_dump`, retención de 30 días y prueba de restauración mensual (objetivo de punto de recuperación: 24 horas; de tiempo de recuperación: 4 horas).
8.15	Registro de actividad	Bitácora de auditoría con tabla, operación, usuario, fecha y detalle en formato JSON.
8.24	Uso de criptografía	TLS 1.3 para HTTPS; WireGuard (ChaCha20-Poly1305) para las VPN.
8.28	Codificación segura	Consultas con parámetros, validación con restricciones de la base y pruebas automáticas.

16.3 Seguridad dentro de la base de datos

- **Tres roles sin inicio de sesión directo** (`tm_consulta`, `tm_operador` y `tm_admin`) a los que se asigna a cada usuario real, de modo que los permisos se cambian en un solo lugar.
- **Permisos por columna:** el rol de consulta lee los pilotos, guardias y operadores sin la columna del DPI. La vista `vw_pilotos_ficha` solo muestra los últimos cuatro dígitos.
- **Bitácora inalterable:** un disparador impide modificar o borrar sus filas y se revocan los permisos de actualización a todos los roles.
- **Registro automático de cambios** en líneas, buses, estaciones por línea, tarjetas, guardias asignados y alertas, con la fila completa en JSON.
- **Restricciones como defensa:** formatos de DPI y color, estados válidos, saldos no negativos y llaves foráneas evitan datos incoherentes aunque otra herramienta escriba en la base.

16.4 Modelo de amenazas

Tabla 63

Amenazas principales (clasificación STRIDE) y controles

Amenaza	Ejemplo	Control
Suplantación de identidad	Alguien usa la PC de una estación sin ser el operador	Autenticación con doble factor (pendiente) y VPN con llaves por estación
Manipulación de datos	Cambiar el saldo de una tarjeta a mano	El saldo solo cambia por disparador; auditoría de cambios; permisos mínimos
Repudio	Negar que se asignó un bus a una línea	Bitácora inalterable con usuario y fecha
Divulgación de información	Copiar el DPI de los pilotos	Enmascarado y permisos por columna; cifrado en tránsito y en reposo
Denegación de servicio	Caída del enlace de una estación	Enlace de respaldo móvil y cola local con reenvío
Elevación de privilegios	Un operador intenta borrar buses	Privilegios revocados; pruebas T31 a T33

16.5 Seguridad de la información en el sistema unificado

La seguridad se concibe en profundidad. En el canal, toda comunicación entre nodos de borde y servicios emplea TLS 1.3 (Rescorla, 2018) con autenticación mutua mediante certificados X.509, de modo que tanto el servidor como cada dispositivo prueban su identidad. En reposo, los datos sensibles se protegen con AES-256. En la tarjeta, el módulo SAM impide la clonación y la alteración del saldo, y la lista negra se sincroniza en menos de 60 segundos para neutralizar tarjetas comprometidas.

En la capa de aplicación, el acceso a las funciones se rige por la matriz de roles de la Tabla 6, con doble factor de autenticación para las operaciones administrativas. La bitácora de eventos se firma digitalmente y no admite modificación ni borrado, lo que garantiza la trazabilidad exigida para la auditoría financiera. Por último, el diseño protege la privacidad de los usuarios: el análisis de video para aforo se ejecuta en el borde y solo emite conteos, sin conservar imágenes ni rasgos biométricos.

La meta de disponibilidad del 99.98 % equivale a menos de 105 minutos de indisponibilidad al año. Para alcanzarla se combinan tres mecanismos: replicación de los servicios críticos en un clúster orquestado con Kubernetes, replicación de la base de datos y del bus de eventos entre nodos, y autonomía de los dispositivos de borde. Esta última es la que protege el servicio frente a la causa más frecuente de interrupción, que es la pérdida del enlace de comunicaciones: los molinetes y las computadoras a bordo continúan operando con datos locales y reenvían los registros acumulados, de forma idempotente, al restablecerse la conexión.

Capítulo XVII. Métricas de software

17.1 Modelo de calidad

La evaluación de la calidad del software se fundamenta en el modelo ISO/IEC 25010 (ISO, 2011) y en el método Goal-Question-Metric (Basili et al., 1994). El proyecto define un catálogo de 42 métricas automatizadas, recopiladas a lo largo del ciclo de vida, y asociadas a las ocho características del modelo. La Tabla 64 ejemplifica la derivación de métricas a partir de objetivos para tres de los atributos más críticos.

Tabla 64

Ejemplos de derivación Goal-Question-Metric

Objetivo	Pregunta	Métrica
Garantizar un cobro ágil en molinetes.	¿Qué proporción de las validaciones se completa en menos de 300 ms?	Percentil 99 de la latencia de validación.
Mantener el servicio ante la pérdida de enlace.	¿Se conservan todas las transacciones acumuladas sin conexión?	Porcentaje de registros sincronizados tras la reconexión.
Facilitar la evolución del código.	¿Las unidades de código son lo bastante simples para modificarse sin riesgo?	Complejidad ciclomática por método.

Nota. Elaboración propia según el método GQM.

17.2 Métricas medidas en esta versión

Las métricas de la Tabla 65 se midieron sobre el código y la base de datos entregados. Las de complejidad se calcularon con la regla `complexity` de ESLint sobre todas las funciones de la aplicación; las de base de datos, consultando el catálogo de PostgreSQL; y las de pruebas y tiempos, ejecutando los scripts.

Tabla 65

Métricas medidas en la versión entregada

Métrica	Valor medido	Umbral objetivo	Resultado
Líneas de código de la aplicación (React y JavaScript)	3,164	—	Informativo
Líneas de SQL de estructuras, reglas, vistas y seguridad	930	—	Informativo
Tablas / vistas / funciones / disparadores	27 / 14 / 20 / 21	—	Informativo
Llaves foráneas / restricciones de verificación / índices	36 / 48 / 59	Integridad declarativa	Cumple
Funciones de la aplicación analizadas	379	—	Informativo
Complejidad ciclomática media por función	2.21	Menor a 10	**Cumple**
Funciones con complejidad mayor a 10	5 de 379 (1.3 %)	Ninguna	**No cumple**
Mayor complejidad (componente App)	31	Menor a 10	**No cumple**

Métrica	Valor medido	Umbral objetivo	Resultado
Condiciones del caso con prueba automática	17 de 18	18 de 18	Casi: la 17 es de diseño
Pruebas automáticas de reglas superadas	36 de 36	100 %	**Cumple**
Reportes con respuesta menor a 2 s	17 de 17 (máximo 12.9 ms)	100 %	**Cumple**
Carga completa de la base de datos (5 scripts)	3.7 s	Menos de 10 s	**Cumple**
Vulnerabilidades conocidas en dependencias (`npm audit`)	0	Cero	**Cumple**
Tamaño de la aplicación publicada	1.1 MB de código + 16 MB del motor SQL	—	Informativo
Conciliación de saldos de tarjetas	Recargas − cobros = saldos	Diferencia cero	**Cumple**

Nota. El motor SQL (PGlite) pesa unos 16 MB sin comprimir (3.6 MB comprimidos) y se descarga una sola vez por dispositivo.

17.3 Derivación Goal-Question-Metric del nivel de red

Tabla 66

Derivación Goal-Question-Metric de las métricas del nivel de red

Objetivo	Pregunta	Métrica
Que las reglas del caso se cumplan siempre.	¿Qué proporción de las reglas tiene una prueba automática superada?	Reglas con prueba superada / reglas totales
Que los reportes sean rápidos.	¿Cuánto tarda el reporte más lento?	Tiempo máximo de respuesta (ms)
Que el código se pueda modificar sin riesgo.	¿Qué funciones son demasiado complejas?	Funciones con complejidad mayor a 10
Que los saldos sean confiables.	¿Cuadran las recargas y los cobros con los saldos?	Diferencia de conciliación (Q)

17.4 Interpretación y acciones

El resultado es mayoritariamente favorable. La complejidad media de 2.21 es baja y las 36 pruebas de reglas se superan. La excepción es que 5 componentes de interfaz superan el umbral de 10: App (31), StationTab (24), NetworkTab (21), GateTab (21), DatabaseTab (16). Son componentes de React que concentran muchas condiciones de presentación. La acción propuesta para la primera iteración de mantenimiento es dividirlos en subcomponentes (por ejemplo, separar de `StationTab` los paneles de alertas, visitas y caja), lo que reduciría la complejidad de cada pieza por debajo de 10 sin cambiar el comportamiento.

Capítulo XVIII. Plan de pruebas

18.1 Objetivo y alcance

El plan de pruebas tiene por objeto garantizar que los módulos del sistema cumplan los requerimientos funcionales RF-01 a RF-20 y los no funcionales RNF-01 a RNF-08, con una disponibilidad del 99.98 %, tiempos de respuesta menores a 300 ms en la validación de tarjetas y alta tolerancia a fallos en operación fuera de línea. Abarca pruebas unitarias, de integración, funcionales, de rendimiento, de tolerancia a fallos, de seguridad y de aceptación de usuario sobre los cuatro grupos de componentes: flota y terminales, recaudación y estaciones, consola de operadores, y seguridad con Centro de Control C5.

Para el nivel de red, el plan agrega las dieciocho condiciones del caso: cada una debe tener, cuando sea posible, una prueba automática en la base de datos.

18.2 Estrategia y tipos de prueba

Tabla 67
Estrategia de pruebas por tipo

Tipo de prueba	Descripción	Herramientas y métodos
Funcionales y de aceptación (UAT)	Verificación del cumplimiento de las historias de usuario y de los flujos de negocio.	Ejecución manual guiada y automatización con Cypress o Selenium.
Rendimiento y carga	Evaluación de la latencia en transacciones NFC y de la concurrencia en hora pico, con carga escalonada desde una línea base de 5,000 transacciones por minuto hasta el pico de 2,500 validaciones por segundo (RNF-05).	JMeter o k6.
Tolerancia a fallos	Verificación de la operación en modo fuera de línea de los molinetes y de la sincronización posterior.	Desconexión física de la red Ethernet o celular durante transacciones.
Seguridad y penetración	Intentos de clonación de tarjetas, interceptación del tráfico MQTT y suplantación de identidad.	OWASP ZAP, Burp Suite y auditoría de certificados mTLS.

Nota. Las herramientas indicadas son las propuestas para la ejecución del plan.

18.3 Pruebas automáticas de las reglas del caso

El archivo `database/07_pruebas.sql` contiene 36 pruebas que se ejecutan con una sola orden y que también se pueden lanzar desde la pestaña **Base de datos** de la aplicación. Cada prueba ejecuta una operación en una subtransacción: si la regla debe rechazarla, comprueba el mensaje; si debe aceptarla, comprueba el efecto. Al terminar, la subtransacción se revierte, por lo que las pruebas no cambian los datos. Los resultados están en la Tabla 68.

Tabla 68

Resultados de las pruebas automáticas de reglas

ID	Regla	Descripción de la prueba	Resultado
T01	R5 máximo de buses	Asignar 4 buses más a la Línea 12 (27 + 4 = 31, el máximo es 30)	**Superada**
T02	R5 mínimo de buses	Quitar 13 buses a la Línea 12 activa (quedarían 14, el mínimo es 15)	**Superada**
T03	R5 activación	Activar la Línea 20 con 3 buses cuando necesita al menos 5	**Superada**
T04	R5 activación	Asignar 2 buses de reserva a la Línea 20 y activarla (5 buses para 5 estaciones)	**Superada**
T05	R4 bus sin línea	Un bus puede quedar sin línea asignada	**Superada**
T06	R7 bus sin parqueo	Dejar un bus sin parqueo	**Superada**
T07	R7 cambio de parqueo	Cambiar un bus de parqueo y conservar el historial	**Superada**
T08	R6 capacidad del parqueo	Mover 6 buses al parqueo de Chinautla (capacidad 8, ya tiene 3)	**Superada**
T09	R2 estación sin línea	Crear una estación que no pertenece a ninguna línea	**Superada**
T10	R2 varias líneas	Existen estaciones que pertenecen a varias líneas (El Trébol: L12, L6 y L13)	**Superada**
T11	R10 acceso sin guardia	Crear un acceso sin guardia de seguridad	**Superada**
T12	R10 último guardia	Retirar al único guardia de un acceso	**Superada**
T13	R11 municipalidad obligatoria	Crear una línea sin municipalidad	**Superada**
T14	R8 accesos de una línea	El reporte de accesos de la Línea 12 incluye todos los accesos de sus estaciones	**Superada**
T15	R12 distancia entre estaciones	La distancia entre Centra Sur y El Trébol en la Línea 12 es la suma de sus tramos	**Superada**
T16	R13 distancia de la línea	La Línea 12 mide 18.40 km	**Superada**
T17	R14 sin alerta	Lectura de 239 pasajeros en una estación de la Línea 12 (bus de 160, umbral 240): no genera alerta	**Superada**
T18	R14 alerta	Lectura de 240 pasajeros (150 % de la capacidad del bus): genera una alerta pendiente	**Superada**
T19	R14 atender alerta	Despachar un bus de refuerzo marca la alerta como atendida	**Superada**
T20	R15 espera extra	Bus con 39 de 160 pasajeros (24.38 %): espera 5 minutos más	**Superada**
T21	R15 límite del 25 %	Bus con 40 de 160 pasajeros (25 %): no espera más	**Superada**
T22	Capacidad del bus	Registrar 161 pasajeros en un bus de capacidad 160	**Superada**
T23	R4 bus de otra línea	Programar un viaje de la Línea 6 con un bus de la Línea 12	**Superada**
T24	R1 visitas en orden	Las visitas de un recorrido de ida siguen el orden de las estaciones de la línea	**Superada**
T25	R18 saldo insuficiente	Cobrar un pasaje de Q1.00 a una tarjeta regular con Q0.50	**Superada**
T26	R18 recarga	Recargar Q20.00 en efectivo suma al saldo de la tarjeta	**Superada**
T27	R18 cobro con tarjeta	El cobro con tarjeta estudiante descuenta Q0.50 del saldo	**Superada**

ID	Regla	Descripción de la prueba	Resultado
T28	R18 método de pago	Recargar una tarjeta usando como método la propia tarjeta de acceso	**Superada**
T29	Auditoría	Cambiar una línea deja un registro en la bitácora	**Superada**
T30	Auditoría inalterable	Modificar un registro de la bitácora	**Superada**
T31	Privilegios mínimos	El rol de consulta intenta leer el DPI de los pilotos	**Superada**
T32	Privilegios mínimos	El rol de consulta lee la ficha de pilotos con el DPI enmascarado	**Superada**
T33	Privilegios mínimos	El rol de operador intenta borrar un bus	**Superada**
T34	R9 datos del piloto	Registrar un piloto sin teléfono móvil (dato de comunicación obligatorio)	**Superada**
T35	R16 una PC por estación	Instalar una segunda computadora en una estación que ya tiene una	**Superada**
T36	R3 acceso de una estación	Crear un acceso que no pertenece a ninguna estación	**Superada**

Nota. Resultados de la ejecución sobre PostgreSQL 16 y PGlite (PostgreSQL 17).

18.4 Resultados de la ejecución

Tabla 69

Entornos en los que se ejecutó el plan automático

Entorno	Motor	Pruebas superadas
Contenedor Docker con PostgreSQL oficial	PostgreSQL 16	36 de 36
Node.js con PGlite	PostgreSQL 17 (WebAssembly)	36 de 36
Aplicación web en Microsoft Edge (pestaña Base de datos)	PostgreSQL 17 (WebAssembly)	36 de 36

Defectos encontrados durante las pruebas

Tabla 70

Defectos detectados y corregidos antes de la entrega

Cód.	Defecto	Cómo se detectó	Corrección
D-01	La prueba T09 esperaba un mensaje con texto distinto al que produce la regla de estación sin línea.	Primera ejecución del plan (32 de 33 superadas)	Se ajustó el texto esperado de la prueba
D-02	La dirección `#reportes/R04` abría el primer reporte y no el pedido.	Revisión de la aplicación en el navegador	La pantalla lee la dirección en cada visita

Nota. Defectos de la propia entrega; no confundir con los defectos D-01 a D-12 de los controles actuales del Capítulo VI.

18.5 Pruebas pendientes

- **Carga y rendimiento** con JMeter o k6 sobre un servidor PostgreSQL en la nube, con el volumen de las 35 estaciones.
- **Seguridad externa** con OWASP ZAP sobre la aplicación y revisión del túnel VPN.
- **Usabilidad con operadores** reales: tiempo para registrar una lectura (objetivo: menos de 30 segundos).
- **Operación con enlace caído** y reenvío de la cola local, que corresponde a la fase de producción.
- **Autenticación y doble factor** (RF-13 y RF-20), pendientes de implementar.

Capítulo XIX. Criterios de aceptación

19.1 Criterios del nivel de red

Los criterios de la Tabla 71 permiten que cada municipalidad verifique por sí misma que el sistema cumple las condiciones del caso. Todos se comprobaron con los datos de demostración, y la última columna remite a la evidencia.

Tabla 71

Criterios de aceptación del nivel de red

ID	Criterio	Condición	Evidencia	Estado
A-RED-01	Toda línea activa tiene entre tantos buses como estaciones y el doble; el sistema rechaza cualquier cambio que lo incumpla.	RN-05	T01 a T04; control de R17 en cero	Cumplido
A-RED-02	No existe ninguna estación sin línea; una estación puede estar en varias líneas.	RN-02	T09, T10; reporte R09	Cumplido
A-RED-03	Todo acceso tiene al menos un guardia vigente y no se puede retirar al último.	RN-10	T11, T12; control de R17	Cumplido
A-RED-04	Todo bus tiene parqueo, no se excede su capacidad y los cambios quedan en el historial.	RN-06, 07	T06 a T08; reporte R12	Cumplido
A-RED-05	El reporte de accesos de una línea incluye todos los accesos de sus estaciones.	RN-08	T14; reporte R06	Cumplido
A-RED-06	La distancia de una línea es la suma de sus tramos; la Línea 12 mide 18.40 km.	RN-12, 13	T15, T16; reportes R01, R02, R08	Cumplido
A-RED-07	Una lectura que llega a 150 % de la capacidad de un bus genera una alerta que se puede atender con un bus de refuerzo.	RN-14	T17 a T19; reporte R13	Cumplido
A-RED-08	Un bus con menos del 25 % de ocupación espera 5 minutos más; con 25 % exacto no espera.	RN-15	T20, T21; reporte R14	Cumplido
A-RED-09	Cada estación tiene una sola PC con al menos un operador asignado.	RN-16	T35; reporte R15	Cumplido
A-RED-10	El saldo de cada tarjeta coincide con sus recargas menos sus cobros; no se cobra sin saldo.	RN-18	T25 a T28; conciliación del Capítulo XIII	Cumplido
A-RED-11	Los reportes responden en menos de 2 segundos.	RNF-10	Tabla de reportes del Capítulo XV	Cumplido
A-RED-12	La bitácora registra los cambios y no se puede modificar ni borrar.	Seguridad	T29, T30	Cumplido

19.2 Criterios del sistema unificado

Tabla 72

Criterios de aceptación funcionales

ID	Criterio	Requerimiento asociado
A-FUN-01	El sistema registra de forma automática la entrada y la salida de las unidades en los predios Centra Norte y Centra Sur mediante sensores y RFID, sin intervención manual, en el 100 % de los eventos.	RF-01
A-FUN-02	El algoritmo de regulación identifica convoyes cuando la distancia entre unidades es inferior al umbral programado (1.1 km, equivalente al 6 % de la ruta).	RF-03
A-FUN-03	Los validadores aplican la tarifa del perfil de la tarjeta: regular Q 1.00, estudiante Q 0.50 y adulto mayor Q 0.25.	RF-09
A-FUN-04	Toda evasión por salto o forzado de molinete se reporta a la consola de auditoría en un máximo de 2 segundos.	RF-11
A-FUN-05	El botón de pánico transmite la ubicación GPS del autobús al Centro de Control C5 con prioridad sobre cualquier otro tráfico de red.	RF-17

Nota. Los criterios son condición necesaria para el dictamen de aceptación.

Tabla 73

Criterios de aceptación técnicos y no funcionales

ID	Criterio	Requerimiento asociado
A-TEC-01	El tiempo total desde la aproximación de la tarjeta NFC hasta la señal de apertura del molinete no supera los 300 milisegundos.	RNF-01
A-TEC-02	El sistema central demuestra una disponibilidad del 99.98 % durante una prueba de carga sostenida de 72 horas continuas, lo que equivale a una indisponibilidad máxima aproximada de 52 segundos en ese lapso.	RNF-03
A-TEC-03	Ante una pérdida total de la conectividad, los molinetes continúan validando transacciones locales y almacenan hasta 50,000 registros para su sincronización automática posterior.	RNF-04
A-TEC-04	Toda transmisión entre autobuses o estaciones y el servidor está cifrada de extremo a extremo con mTLS y AES-256.	RF-19, RNF-06

Nota. mTLS = TLS con autenticación mutua.

Los criterios A-FUN y A-TEC dependen de hardware real (lectores NFC, sensores, enlace en campo) y se verifican en el piloto de la Línea 12; no se pueden dar por cumplidos con la simulación.

19.3 Trazabilidad entre condiciones, pruebas y criterios

Tabla 74

Trazabilidad de las condiciones del caso

Condición	Pruebas automáticas	Reportes	Criterio
RN-01	T24	R02, R14	A-RED-08
RN-02	T09, T10	R09, R17	A-RED-02
RN-03	T36	R03, R06	A-RED-05
RN-04	T05, T23	R04, R05	A-RED-01
RN-05	T01 a T04	R01, R17	A-RED-01
RN-06	T08	R12	A-RED-04
RN-07	T06, T07	R05, R12	A-RED-04
RN-08	T14	R06	A-RED-05
RN-09	T34	R10, R11	A-RED-09 (datos del piloto), A-RED-12
RN-10	T11, T12	R06, R07, R17	A-RED-03
RN-11	T13	R01, R03	A-RED-02
RN-12	T15	R08	A-RED-06
RN-13	T16	R01, R02	A-RED-06
RN-14	T17 a T19	R13	A-RED-07
RN-15	T20, T21	R14	A-RED-08
RN-16	T35	R15	A-RED-09
RN-17	Decisión de diseño (Capítulo XIII)	R15	A-RED-09
RN-18	T25 a T28	R16	A-RED-10

Capítulo XX. Carta de aceptación del proyecto

Concluida la ejecución del plan de pruebas, la entidad receptora formaliza la recepción del sistema con una carta de aceptación. Se presenta el modelo de la carta, que solo debe firmarse cuando los resultados de las pruebas respalden cada una de sus declaraciones. Los firmantes son el representante de la entidad receptora y el consultor responsable.

CARTA DE ACEPTACIÓN FINAL DEL SISTEMA

Lugar y fecha: Ciudad de Guatemala, ____ de _____ de 20____.

Atención: William Alejandro Morales Mendez, consultor responsable del proyecto.

Entidad receptora: Municipalidad de _____, en representación de las municipalidades participantes.

Por medio de la presente, la entidad receptora hace constar que ha recibido y revisado los entregables técnicos y funcionales del *Sistema Unificado de Gestión de Líneas, Estaciones, Flota y Recaudación del Transmetro de Guatemala*, descritos en este documento.

1. Declaración de conformidad. Una vez ejecutado el plan de pruebas, se verifica que el sistema cumple las dieciocho condiciones del caso, con las 36 pruebas automáticas de reglas superadas y los criterios A-RED-01 a A-RED-12 cumplidos; que los diecisiete reportes de estaciones, líneas, buses y demás se muestran en la aplicación con su consulta; y que la propuesta económica y los estudios de factibilidad fueron revisados.

2. Entregables recibidos. (a) Documento unificado en PDF; (b) presentación con resumen ejecutivo; (c) presentación y guion del video de explicación; (d) repositorio con el código, el script de la base de datos y los diagramas; (e) aplicación web publicada.

3. Observaciones. Se registran como pendientes para la fase de producción: autenticación con doble factor, base de datos compartida en la nube, cola local con reenvío en caso de caída del enlace y pruebas de carga con equipo real. _____

4. Dictamen final. Con base en los resultados de las pruebas técnicas y en la demostración funcional, se otorga el estado de **ACEPTADO** para el despliegue en el corredor piloto de la Línea 12.

Nancy Guadalupe Portillo Pinto
Representante de la entidad receptora
Revisión y aceptación

William Alejandro Morales Mendez
Consultor responsable del proyecto
william700morales@gmail.com · 5987-1377

Capítulo XXI. Conclusiones y recomendaciones

21.1 Conclusiones

En primer lugar, el caso se pudo traducir de forma completa a un modelo relacional: las dieciocho condiciones del enunciado quedaron expresadas como tablas, restricciones, funciones y disparadores de PostgreSQL, y diecisiete de ellas tienen una prueba automática que las comprueba (la restante, la conectividad, es una decisión de diseño). Las 36 pruebas se superan en PostgreSQL 16 y en PGlite, lo que da confianza en que los scripts se pueden llevar a un servidor sin cambios.

En segundo lugar, aplicar las reglas en la base de datos, y no en la interfaz, cambia la naturaleza del control: un planificador no puede dejar una línea sin buses suficientes, ni un acceso sin guardia, ni un bus sin parqueo, aunque lo intente, y cada intento rechazado queda explicado con un mensaje claro. Esto corrige nueve de las doce deficiencias detectadas en los controles actuales.

En tercer lugar, la propuesta económica es favorable pero ajustada. Con una inversión de Q 700,500.00 y una tasa de descuento de 10 %, el proyecto obtiene un VPN de Q 73,921.67, una TIR de 13.94 % y una recuperación en tres años y seis meses. Si el proyecto tuviera que asumir además las computadoras y la conectividad de las estaciones, el VPN sería negativo, por lo que la recomendación es financiar ese equipamiento como infraestructura ordinaria de las estaciones y medir los ahorros reales en el piloto.

En cuarto lugar, la conectividad entre estaciones se resolvió con una VPN WireGuard sobre internet de banda ancha y un enlace móvil de respaldo, la opción con mejor puntaje en la matriz de decisión y de costo moderado, y los métodos de pago quedaron definidos con una conciliación de saldos que cuadra.

Por último, el análisis de métricas muestra una complejidad media baja, pero cinco componentes de interfaz superan el umbral de 10 y deben dividirse. Quedan además pendientes la autenticación con doble factor, la base de datos compartida en la nube y las pruebas de carga con hardware real.

21.2 Recomendaciones

1. Validar con cada municipalidad las interpretaciones de las condiciones 14 y 15 (sección 7.2) y las tarifas por perfil, antes de la fase de desarrollo final.
2. Sustituir los datos de demostración por los datos reales de líneas, estaciones, accesos, buses, parqueos, pilotos y guardias, con las plantillas de carga y las validaciones de la base.
3. Pasar a un PostgreSQL gestionado en la nube con acceso por VPN y agregar autenticación con doble factor antes de operar con usuarios reales.
4. Financiar las computadoras y los enlaces de las estaciones como infraestructura ordinaria y medir en el piloto de la Línea 12 si los ahorros alcanzan los Q 310,000 anuales estimados.
5. Dividir los componentes de interfaz con complejidad mayor a 10 y repetir la medición en cada iteración.
6. Completar la matriz de pruebas con carga, seguridad externa y usabilidad con operadores reales.

21.3 Líneas de trabajo futuro

Como trabajo futuro se propone integrar un modelo de predicción de la demanda por franja horaria para alimentar la secuenciación dinámica de despachos, incorporar la aplicación móvil del pasajero con información de ocupación en tiempo real, evaluar la interoperabilidad de la tarjeta con otros modos de transporte de la ciudad y explorar el uso de aprendizaje automático sobre la bitácora de evasión para anticipar patrones de fraude en los molinetes.

Además, se propone integrar el nivel de red con los módulos del Centro de Control C5, de modo que las simulaciones de flota, pasajeros y operadores lean y escriban en la misma base de datos, y agregar una cola local en cada PC de estación para operar con el enlace caído.

Referencias

Basili, V. R., Caldiera, G., & Rombach, H. D. (1994). The goal question metric approach. En J. J. Marciniak (Ed.), *Encyclopedia of software engineering* (Vol. 1, pp. 528-532). John Wiley & Sons.

Congreso de la República de Guatemala. (1992). *Ley de Contrataciones del Estado* (Decreto Número 57-92). Diario de Centro América.

Congreso de la República de Guatemala. (2008). *Ley de Acceso a la Información Pública* (Decreto Número 57-2008). Diario de Centro América.

Daganzo, C. F. (2009). A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons. *Transportation Research Part B: Methodological, 43*(10), 913-921. https://doi.org/10.1016/j.trb.2009.04.002

Donenfeld, J. A. (2017). WireGuard: Next generation kernel network tunnel. *Proceedings of the 2017 Network and Distributed System Security Symposium*. https://doi.org/10.14722/ndss.2017.23160

EDP Guatemala. (2019). *EDP Guatemala* [Informe sobre protección de datos personales]. Prensa Libre. https://www.prensalibre.com/wp-content/uploads/2019/12/EDP_Guatemala.pdf

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Emisoras Unidas. (2021, 26 de mayo). *Municipalidad de Guatemala colocará 220 mil tarjetas de Transmetro*. https://emisorasunidas.com/2021/05/26/guatemala-transmetro-tarjetas-prepago/

Emisoras Unidas. (2022a, 2 de febrero). *Transmetro 15 aniversario febrero 2022*. https://emisorasunidas.com/2022/02/02/transmetro-15-aniversario-febrero-2022/

Emisoras Unidas. (2022b, 17 de marzo). *Precio del pasaje en el Transmetro se mantendrá en Q1, según alcalde*. https://emisorasunidas.com/2022/03/17/precio-pasaje-transmetro-combustibles/

International Organization for Standardization. (2011). *Systems and software engineering: Systems and software quality requirements and evaluation (SQuaRE): System and software quality models* (ISO/IEC 25010:2011).

International Organization for Standardization. (2018). *Cards and security devices for personal identification: Contactless proximity objects. Part 1: Physical characteristics* (ISO/IEC 14443-1:2018).

International Organization for Standardization. (2022). *Information security, cybersecurity and privacy protection: Information security management systems: Requirements* (ISO/IEC 27001:2022).

Kent, S., & Seo, K. (2005). *Security architecture for the Internet Protocol* (RFC 4301). Internet Engineering Task Force. https://doi.org/10.17487/RFC4301

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308-320. https://doi.org/10.1109/TSE.1976.233837

Newell, G. F., & Potts, R. B. (1964). Maintaining a bus schedule. En *Proceedings of the 2nd Australian Road Research Board Conference* (Vol. 2, pp. 388-393). Australian Road Research Board.

OWASP Foundation. (2021). *OWASP Top Ten*. https://owasp.org/Top10/

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation: CREATE TRIGGER*. https://www.postgresql.org/docs/16/sql-createtrigger.html

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill Education.

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://doi.org/10.17487/RFC8446

Sapag Chain, N., & Sapag Chain, R. (2014). *Preparación y evaluación de proyectos* (6.ª ed.). McGraw-Hill.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/

Sommerville, I. (2016). *Ingeniería de software* (10.ª ed.). Pearson Education.

Wikipedia. (s. f.). *Transmetro*. Recuperado el 23 de octubre de 2026, de https://en.wikipedia.org/wiki/Transmetro

Apéndice A. Glosario de siglas y términos

Tabla 75

Siglas y términos utilizados en el documento

Sigla o término	Significado
AES-256	Cifrado simétrico avanzado con llave de 256 bits.
BRT	Autobús de tránsito rápido (bus rapid transit).
C5	Centro de Control, Comando, Comunicaciones, Cómputo y Calidad.
CAPEX	Gastos de capital o inversión inicial.
Disparador (trigger)	Función que PostgreSQL ejecuta automáticamente ante un cambio en una tabla.
DPI	Documento Personal de Identificación.
ER	Entidad-relación, modelo que describe los datos y sus relaciones.
ETA	Tiempo estimado de llegada (estimated time of arrival).
GPS	Sistema de posicionamiento global.
GQM	Goal-Question-Metric, método de derivación de métricas.
HMI	Interfaz hombre-máquina.
IndexedDB	Almacén de datos del navegador donde se guarda la base de demostración.
LTE	Tecnología de datos móviles usada como enlace de respaldo.
MQTT	Protocolo de mensajería liviano para telemetría.
mTLS	TLS con autenticación mutua entre cliente y servidor.
NFC	Comunicación de campo cercano.
OPEX	Gastos operativos recurrentes.
PGlite	PostgreSQL compilado a WebAssembly que se ejecuta dentro del navegador.
RBAC	Control de acceso basado en roles.
RFID	Identificación por radiofrecuencia.
RN	Condición numerada del enunciado del caso.
SAM	Módulo de acceso seguro alojado en el lector.
Scrum	Marco de trabajo ágil con iteraciones cortas.
TIR	Tasa interna de retorno.
UML	Lenguaje unificado de modelado.
VPN	Valor presente neto.

Sigla o término	Significado
VPN	Red privada virtual: túnel cifrado entre sedes.
WireGuard	Protocolo de VPN moderno y de configuración sencilla.

Apéndice B. Script SQL completo de las estructuras

Texto de `01_schema.sql` (tablas) y `02_funciones_triggers.sql` (reglas). Las vistas (`03_vistas.sql`), la seguridad (`04_seguridad.sql`), los datos de demostración (`05_seed.sql`, 2,307 líneas), los reportes y las pruebas están completos en el repositorio.

database/01_schema.sql

```
--
================================================================
-- TRANSMETRO GUATEMALA - Sistema de control de líneas, estaciones y
flota
-- Archivo 01: estructuras (tablas, llaves y restricciones)
-- Motor: PostgreSQL 16 (probado también en PGlite, que es PostgreSQL
en WebAssembly)
-- Ejecutar los archivos en orden: 01, 02, 03, 04, 05 (y 06/07 para
reportes y pruebas)
-- Las horas se guardan como hora local de Guatemala (UTC-6), sin
zona horaria.
--
================================================================
DROP SCHEMA IF EXISTS transmetro CASCADE;
CREATE SCHEMA transmetro;
SET search_path TO transmetro, public;
-- ----------------------------------------------------------------
---
-- Parámetros configurables (reglas 5, 14 y 15 del enunciado)
-- ----------------------------------------------------------------
---
CREATE TABLE parametro_sistema (
    clave        VARCHAR(40)  PRIMARY KEY,
    valor        NUMERIC(10,2) NOT NULL,
    descripcion  VARCHAR(200) NOT NULL
);
-- ----------------------------------------------------------------
---
-- Municipalidades (regla 11)
-- ----------------------------------------------------------------
---
CREATE TABLE municipalidad (
    id_municipalidad INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    nombre           VARCHAR(80)  NOT NULL UNIQUE,
    departamento     VARCHAR(60)  NOT NULL,
    direccion        VARCHAR(150),
    telefono         VARCHAR(20)
);
-- ----------------------------------------------------------------
---
-- Líneas y estaciones (reglas 1, 2, 11)
-- ----------------------------------------------------------------
---
CREATE TABLE linea (
    id_linea         INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    codigo           VARCHAR(10)  NOT NULL UNIQUE,
    nombre           VARCHAR(100) NOT NULL,
    color_hex        CHAR(7)      NOT NULL DEFAULT '#1F8A4C' CHECK
(color_hex ~ '^#[0-9A-Fa-f]{6}$'),
    id_municipalidad INT          NOT NULL REFERENCES
municipalidad(id_municipalidad),
    estado           VARCHAR(12)  NOT NULL DEFAULT 'PLANIFICADA'
                     CHECK (estado IN
('PLANIFICADA','ACTIVA','SUSPENDIDA')),
    hora_inicio      TIME         NOT NULL DEFAULT '05:00',
    hora_fin         TIME         NOT NULL DEFAULT '21:00'
);
CREATE TABLE estacion (
    id_estacion      INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    codigo           VARCHAR(10)  NOT NULL UNIQUE,
    nombre           VARCHAR(100) NOT NULL UNIQUE,
    direccion        VARCHAR(150),
    id_municipalidad INT          NOT NULL REFERENCES
municipalidad(id_municipalidad),
    latitud          NUMERIC(9,6) NOT NULL CHECK (latitud BETWEEN 13
AND 18),
    longitud         NUMERIC(9,6) NOT NULL CHECK (longitud BETWEEN
-93 AND -88),
    capacidad_anden  INT          NOT NULL CHECK (capacidad_anden >
0),
    es_terminal      BOOLEAN      NOT NULL DEFAULT FALSE
);
-- Una estación puede pertenecer a varias líneas; el orden es propio
de cada línea
CREATE TABLE linea_estacion (
    id_linea    INT      NOT NULL REFERENCES linea(id_linea) ON
DELETE CASCADE,
    id_estacion INT      NOT NULL REFERENCES estacion(id_estacion),
    orden       SMALLINT NOT NULL CHECK (orden > 0),
    PRIMARY KEY (id_linea, id_estacion),
    CONSTRAINT uq_linea_orden UNIQUE (id_linea, orden) DEFERRABLE
INITIALLY DEFERRED
);
-- Distancia entre dos estaciones contiguas (regla 12). Se guarda una
sola vez por par,
-- con id_estacion_a < id_estacion_b, y sirve para todas las líneas
que las unan.
CREATE TABLE tramo (
    id_estacion_a INT          NOT NULL REFERENCES
estacion(id_estacion),
    id_estacion_b INT          NOT NULL REFERENCES
estacion(id_estacion),
    distancia_km  NUMERIC(6,2) NOT NULL CHECK (distancia_km > 0),
    PRIMARY KEY (id_estacion_a, id_estacion_b),
    CHECK (id_estacion_a < id_estacion_b)
);
-- ----------------------------------------------------------------
---
-- Accesos (regla 3) y parqueos (regla 6)
-- ----------------------------------------------------------------
---
CREATE TABLE acceso (
    id_acceso   INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_estacion INT         NOT NULL REFERENCES estacion(id_estacion)
ON DELETE CASCADE,
    numero      SMALLINT    NOT NULL CHECK (numero > 0),
    nombre      VARCHAR(80) NOT NULL,
    tipo        VARCHAR(12) NOT NULL DEFAULT 'PRINCIPAL'
                CHECK (tipo IN
('PRINCIPAL','SECUNDARIO','ACCESIBLE')),
    UNIQUE (id_estacion, numero)
);
CREATE TABLE parqueo (
    id_parqueo      INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    codigo          VARCHAR(10)  NOT NULL UNIQUE,
    nombre          VARCHAR(100) NOT NULL,
    capacidad_buses INT          NOT NULL CHECK (capacidad_buses >
0),
    id_estacion     INT          REFERENCES estacion(id_estacion),  -
- solo algunos parqueos están en una estación
    direccion       VARCHAR(150)
);
-- ----------------------------------------------------------------
---
-- Buses (reglas 4, 5, 6, 7)
-- ----------------------------------------------------------------
---
CREATE TABLE bus (
    id_bus              INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    numero_unidad       VARCHAR(10) NOT NULL UNIQUE,
    placa               VARCHAR(10) NOT NULL UNIQUE,
    tipo                VARCHAR(12) NOT NULL DEFAULT 'ARTICULADO'
                        CHECK (tipo IN
('ARTICULADO','CONVENCIONAL')),
    capacidad_pasajeros INT         NOT NULL CHECK
(capacidad_pasajeros > 0),
    anio                SMALLINT    NOT NULL CHECK (anio BETWEEN 2000
AND 2100),
    estado              VARCHAR(13) NOT NULL DEFAULT 'OPERATIVO'
                        CHECK (estado IN
('OPERATIVO','MANTENIMIENTO')),
    id_linea            INT         REFERENCES linea(id_linea),
-- NULL = sin línea (regla 4)
    id_parqueo          INT         NOT NULL REFERENCES
parqueo(id_parqueo) -- nunca sin parqueo (regla 7)
);
CREATE TABLE bus_parqueo_historial (
    id_historial BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_bus       INT       NOT NULL REFERENCES bus(id_bus) ON DELETE
CASCADE,
    id_parqueo   INT       NOT NULL REFERENCES parqueo(id_parqueo),
    desde        TIMESTAMP NOT NULL DEFAULT now(),
    hasta        TIMESTAMP,
    CHECK (hasta IS NULL OR hasta >= desde)
);
-- ----------------------------------------------------------------
---
```

```
---
-- Pilotos (regla 9): historial educativo, residencia y comunicación
-- ------------------------------------------------------------------
---
CREATE TABLE piloto (
    id_piloto            INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    dpi                  CHAR(13)     NOT NULL UNIQUE CHECK (dpi ~
'^[0-9]{13}$'),
    nombres              VARCHAR(80)  NOT NULL,
    apellidos            VARCHAR(80)  NOT NULL,
    fecha_nacimiento     DATE         NOT NULL CHECK
(fecha_nacimiento < DATE '2008-01-01'),
    licencia_numero      VARCHAR(15)  NOT NULL UNIQUE,
    licencia_tipo        CHAR(1)      NOT NULL CHECK (licencia_tipo
IN ('A','B','C')),
    fecha_contratacion   DATE         NOT NULL,
    activo               BOOLEAN      NOT NULL DEFAULT TRUE,
    -- residencia
    direccion_residencia VARCHAR(150) NOT NULL,
    zona                 SMALLINT     CHECK (zona BETWEEN 1 AND 25),
    id_municipalidad_residencia INT   NOT NULL REFERENCES
municipalidad(id_municipalidad),
    -- comunicación
    telefono_movil       VARCHAR(12)  NOT NULL,
    telefono_residencial VARCHAR(12),
    correo               VARCHAR(100) NOT NULL CHECK (correo LIKE
'%_@_%._%'),
    contacto_emergencia_nombre   VARCHAR(100) NOT NULL,
    contacto_emergencia_telefono VARCHAR(12)  NOT NULL
);
CREATE TABLE piloto_estudio (
    id_estudio  INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_piloto   INT          NOT NULL REFERENCES piloto(id_piloto) ON
DELETE CASCADE,
    nivel       VARCHAR(15)  NOT NULL
                CHECK (nivel IN
('PRIMARIA','BASICO','DIVERSIFICADO','TECNICO','UNIVERSITARIO','POSTG
RADO','CAPACITACION')),
    titulo      VARCHAR(120) NOT NULL,
    institucion VARCHAR(120) NOT NULL,
    anio_inicio SMALLINT     NOT NULL CHECK (anio_inicio BETWEEN 1950
AND 2100),
    anio_fin    SMALLINT,
    concluido   BOOLEAN      NOT NULL DEFAULT TRUE,
    CHECK (anio_fin IS NULL OR anio_fin >= anio_inicio)
);
CREATE TABLE asignacion_piloto_bus (
    id_asignacion INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_bus        INT         NOT NULL REFERENCES bus(id_bus),
    id_piloto     INT         NOT NULL REFERENCES piloto(id_piloto),
    turno         VARCHAR(10) NOT NULL CHECK (turno IN
('MATUTINO','VESPERTINO','NOCTURNO')),
    fecha_inicio  DATE        NOT NULL,
    fecha_fin     DATE,
    CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
);
-- un piloto solo tiene una asignación vigente, y un bus un piloto
vigente por turno
CREATE UNIQUE INDEX uq_asig_piloto_vigente ON asignacion_piloto_bus
(id_piloto) WHERE fecha_fin IS NULL;
CREATE UNIQUE INDEX uq_asig_bus_turno_vigente ON
asignacion_piloto_bus (id_bus, turno) WHERE fecha_fin IS NULL;
-- ------------------------------------------------------------------
---
-- Guardias de seguridad (regla 10)
-- ------------------------------------------------------------------
---
CREATE TABLE guardia (
    id_guardia         INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    dpi                CHAR(13)    NOT NULL UNIQUE CHECK (dpi ~ '^[0-
9]{13}$'),
    nombres            VARCHAR(80) NOT NULL,
    apellidos          VARCHAR(80) NOT NULL,
    telefono           VARCHAR(12) NOT NULL,
    fecha_contratacion DATE        NOT NULL,
    activo             BOOLEAN     NOT NULL DEFAULT TRUE
);
CREATE TABLE asignacion_guardia (
    id_asignacion INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_guardia    INT         NOT NULL REFERENCES
guardia(id_guardia),
    id_acceso     INT         NOT NULL REFERENCES acceso(id_acceso)
ON DELETE CASCADE,
    turno         VARCHAR(10) NOT NULL CHECK (turno IN
('MATUTINO','VESPERTINO','NOCTURNO')),
    fecha_inicio  DATE        NOT NULL,
    fecha_fin     DATE,
    CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
);
CREATE UNIQUE INDEX uq_asig_guardia_vigente ON asignacion_guardia
(id_guardia) WHERE fecha_fin IS NULL;
-- ------------------------------------------------------------------
---
-- PC de trabajo por estación y operador (regla 16) y conectividad
(regla 17)
-- ------------------------------------------------------------------
---
CREATE TABLE computadora (
    id_computadora INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_estacion    INT         NOT NULL UNIQUE REFERENCES
estacion(id_estacion), -- una PC por estación
    hostname       VARCHAR(40) NOT NULL UNIQUE,
    ip_vpn         INET        NOT NULL UNIQUE,
    sistema_operativo VARCHAR(40) NOT NULL DEFAULT 'Windows 11 Pro',
    estado         VARCHAR(12) NOT NULL DEFAULT 'EN_LINEA' CHECK
(estado IN ('EN_LINEA','SIN_ENLACE','MANTENIMIENTO'))
);
CREATE TABLE operador (
    id_operador    INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    dpi            CHAR(13)    NOT NULL UNIQUE CHECK (dpi ~ '^[0-9]
{13}$'),
    nombres        VARCHAR(80) NOT NULL,
    apellidos      VARCHAR(80) NOT NULL,
    usuario        VARCHAR(30) NOT NULL UNIQUE,
    correo         VARCHAR(100) NOT NULL,
    telefono       VARCHAR(12) NOT NULL,
    turno          VARCHAR(10) NOT NULL CHECK (turno IN
('MATUTINO','VESPERTINO','NOCTURNO')),
    id_computadora INT         NOT NULL REFERENCES
computadora(id_computadora),
    activo         BOOLEAN     NOT NULL DEFAULT TRUE
);
CREATE TABLE enlace_red (
    id_enlace        INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_estacion      INT         NOT NULL REFERENCES
estacion(id_estacion) ON DELETE CASCADE,
    rol              VARCHAR(9)  NOT NULL CHECK (rol IN
('PRINCIPAL','RESPALDO')),
    medio            VARCHAR(12) NOT NULL CHECK (medio IN
('FIBRA','BANDA_ANCHA','LTE')),
    proveedor        VARCHAR(60) NOT NULL,
    ancho_banda_mbps INT         NOT NULL CHECK (ancho_banda_mbps >
0),
    protocolo_vpn    VARCHAR(12) NOT NULL DEFAULT 'WIREGUARD' CHECK
(protocolo_vpn IN ('WIREGUARD','IPSEC')),
    estado           VARCHAR(10) NOT NULL DEFAULT 'ACTIVO' CHECK
(estado IN ('ACTIVO','CAIDO')),
    UNIQUE (id_estacion, rol)
);
-- ------------------------------------------------------------------
---
-- Métodos de pago y tarjeta de acceso (regla 18)
-- ------------------------------------------------------------------
---
CREATE TABLE tipo_tarjeta (
    id_tipo_tarjeta INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    nombre          VARCHAR(30)  NOT NULL UNIQUE,
    tarifa          NUMERIC(6,2) NOT NULL CHECK (tarifa >= 0)
);
CREATE TABLE tarjeta_acceso (
    id_tarjeta      INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    numero          CHAR(10)     NOT NULL UNIQUE,
    id_tipo_tarjeta INT          NOT NULL REFERENCES
tipo_tarjeta(id_tipo_tarjeta),
    saldo           NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (saldo >=
0),
    estado          VARCHAR(10)  NOT NULL DEFAULT 'ACTIVA' CHECK
(estado IN ('ACTIVA','BLOQUEADA','VENCIDA')),
    fecha_emision   DATE         NOT NULL DEFAULT current_date
);
CREATE TABLE metodo_pago (
    id_metodo_pago   INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    nombre           VARCHAR(50) NOT NULL UNIQUE,
    usa_saldo_tarjeta BOOLEAN    NOT NULL DEFAULT FALSE,  --
descuenta del saldo de la tarjeta de acceso
    permite_recarga  BOOLEAN     NOT NULL DEFAULT FALSE   -- sirve
para recargar una tarjeta
);
CREATE TABLE pago (
    id_pago        BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_tarjeta     INT          REFERENCES
tarjeta_acceso(id_tarjeta),
    id_metodo_pago INT          NOT NULL REFERENCES
metodo_pago(id_metodo_pago),
    id_estacion    INT          NOT NULL REFERENCES
estacion(id_estacion),
    tipo           VARCHAR(13)  NOT NULL CHECK (tipo IN
('RECARGA','COBRO_PASAJE')),
    monto          NUMERIC(8,2) NOT NULL CHECK (monto >= 0),
    fecha          TIMESTAMP    NOT NULL DEFAULT now()
);
-- ------------------------------------------------------------------
---
-- Recorridos, visitas a estaciones, lecturas de pasajeros y alertas
(reglas 1, 14, 15)
-- ------------------------------------------------------------------
---
CREATE TABLE viaje (
    id_viaje           INT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_linea           INT         NOT NULL REFERENCES
linea(id_linea),
    id_bus             INT         NOT NULL REFERENCES bus(id_bus),
```

```
    id_piloto          INT         NOT NULL REFERENCES
piloto(id_piloto),
    sentido            VARCHAR(6)  NOT NULL CHECK (sentido IN
('IDA','VUELTA')),
    fecha_hora_salida  TIMESTAMP   NOT NULL,
    estado             VARCHAR(10) NOT NULL DEFAULT 'PROGRAMADO'
CHECK (estado IN ('PROGRAMADO','EN_CURSO','FINALIZADO'))
);
CREATE TABLE visita (
    id_visita         BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
    id_viaje          INT          NOT NULL REFERENCES
viaje(id_viaje) ON DELETE CASCADE,
    id_estacion       INT          NOT NULL REFERENCES
estacion(id_estacion),
    orden             SMALLINT     NOT NULL CHECK (orden > 0),
    hora_programada   TIMESTAMP    NOT NULL,
    hora_llegada_real TIMESTAMP,
    pasajeros_a_bordo INT          CHECK (pasajeros_a_bordo >= 0),
    ocupacion_pct     NUMERIC(5,2),             -- lo calcula el
trigger
    espera_extra_min  SMALLINT     NOT NULL DEFAULT 0,  -- regla 15,
la calcula el trigger
    UNIQUE (id_viaje, orden)
);
CREATE TABLE lectura_estacion (
    id_lectura         BIGINT GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,
    id_estacion        INT       NOT NULL REFERENCES
estacion(id_estacion),
    id_linea           INT       NOT NULL REFERENCES linea(id_linea),
    fecha              TIMESTAMP NOT NULL DEFAULT now(),
    pasajeros_esperando INT      NOT NULL CHECK (pasajeros_esperando
>= 0)
);
CREATE TABLE alerta (
    id_alerta           BIGINT GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY,
    tipo                VARCHAR(20) NOT NULL DEFAULT 'AFORO_ESTACION'
CHECK (tipo IN ('AFORO_ESTACION')),
    id_estacion         INT         NOT NULL REFERENCES
estacion(id_estacion),
    id_linea            INT         NOT NULL REFERENCES
linea(id_linea),
    id_lectura          BIGINT      REFERENCES
lectura_estacion(id_lectura),
    fecha               TIMESTAMP   NOT NULL DEFAULT now(),
    pasajeros_esperando INT         NOT NULL,
    umbral              INT         NOT NULL,
    mensaje             VARCHAR(250) NOT NULL,
    atendida            BOOLEAN     NOT NULL DEFAULT FALSE,
    id_bus_refuerzo     INT         REFERENCES bus(id_bus),
    fecha_atencion      TIMESTAMP
);
-- ---------------------------------------------------------------------
---
-- Bitácora de auditoría (no se puede modificar ni borrar)
-- ---------------------------------------------------------------------
---
CREATE TABLE bitacora (
    id_bitacora BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    fecha       TIMESTAMP   NOT NULL DEFAULT now(),
    usuario_bd  VARCHAR(60) NOT NULL DEFAULT current_user,
    tabla       VARCHAR(40) NOT NULL,
    operacion   VARCHAR(6)  NOT NULL,
    id_registro VARCHAR(40),
    detalle     JSONB
);
-- Índices de apoyo a las consultas de los reportes
CREATE INDEX ix_bus_linea       ON bus (id_linea);
CREATE INDEX ix_bus_parqueo     ON bus (id_parqueo);
CREATE INDEX ix_acceso_estacion ON acceso (id_estacion);
CREATE INDEX ix_le_estacion     ON linea_estacion (id_estacion);
CREATE INDEX ix_visita_estacion ON visita (id_estacion);
CREATE INDEX ix_lectura_est     ON lectura_estacion (id_estacion,
fecha);
CREATE INDEX ix_pago_fecha      ON pago (fecha);
```

database/02_funciones_triggers.sql

```
--
=====================================================================
-- Archivo 02: funciones y disparadores (triggers) que aplican las
reglas del enunciado
--
=====================================================================
SET search_path TO transmetro, public;
-- Lectura de parámetros ------------------------------------------
----
CREATE OR REPLACE FUNCTION fn_parametro(p_clave TEXT) RETURNS NUMERIC
LANGUAGE sql STABLE AS $$
    SELECT valor FROM parametro_sistema WHERE clave = p_clave
$$;
-- Distancia entre dos estaciones contiguas, sin importar el orden
del par (regla 12)
CREATE OR REPLACE FUNCTION fn_distancia_tramo(p_a INT, p_b INT)
RETURNS NUMERIC
LANGUAGE sql STABLE AS $$
    SELECT distancia_km FROM tramo
     WHERE id_estacion_a = LEAST(p_a, p_b) AND id_estacion_b =
GREATEST(p_a, p_b)
$$;
-- Distancia entre dos estaciones de la misma línea: suma de los
tramos intermedios (regla 12)
CREATE OR REPLACE FUNCTION fn_distancia_en_linea(p_linea INT, p_a
INT, p_b INT) RETURNS NUMERIC
LANGUAGE sql STABLE AS $$
    WITH ord AS (
        SELECT id_estacion, orden,
               LAG(id_estacion) OVER (ORDER BY orden) AS anterior
          FROM linea_estacion WHERE id_linea = p_linea
    ), limites AS (
        SELECT MIN(orden) AS o1, MAX(orden) AS o2
          FROM ord WHERE id_estacion IN (p_a, p_b)
    )
    SELECT SUM(fn_distancia_tramo(o.anterior, o.id_estacion))
      FROM ord o, limites l
     WHERE o.orden > l.o1 AND o.orden <= l.o2 AND o.anterior IS NOT
NULL
$$;
-- Distancia total de una línea (regla 13)
CREATE OR REPLACE FUNCTION fn_distancia_linea(p_linea INT) RETURNS
NUMERIC
LANGUAGE sql STABLE AS $$
    SELECT COALESCE(SUM(fn_distancia_tramo(anterior, id_estacion)),
0)
      FROM (SELECT id_estacion,
                   LAG(id_estacion) OVER (ORDER BY orden) AS anterior
              FROM linea_estacion WHERE id_linea = p_linea) t
     WHERE anterior IS NOT NULL
$$;
-- ---------------------------------------------------------------------
---
-- Regla 5: buses por línea entre N y 2N (N = número de estaciones)
-- El máximo se exige siempre; el mínimo, cuando la línea está
ACTIVA.
-- Se valida al final de la transacción para poder mover varios buses
a la vez.
-- ---------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION fn_validar_flota_linea(p_id_linea INT)
RETURNS VOID
LANGUAGE plpgsql AS $$
DECLARE
    v_estado TEXT; v_cod TEXT; v_est INT; v_bus INT; v_min INT; v_max
INT;
BEGIN
    SELECT estado, codigo INTO v_estado, v_cod FROM linea WHERE
id_linea = p_id_linea;
    IF NOT FOUND THEN RETURN; END IF;
    SELECT count(*) INTO v_est FROM linea_estacion WHERE id_linea =
p_id_linea;
    SELECT count(*) INTO v_bus FROM bus WHERE id_linea = p_id_linea;
    v_min := ceil(v_est * fn_parametro('FLOTA_MIN_POR_ESTACION'));
    v_max := floor(v_est * fn_parametro('FLOTA_MAX_POR_ESTACION'));
    IF v_bus > v_max THEN
        RAISE EXCEPTION 'Línea %: tiene % buses y el máximo permitido
es % (el doble de sus % estaciones)',
            v_cod, v_bus, v_max, v_est USING ERRCODE = 'P0001';
    END IF;
    IF v_estado = 'ACTIVA' AND v_bus < v_min THEN
        RAISE EXCEPTION 'Línea %: una línea ACTIVA necesita al menos
% buses (uno por estación) y tiene %',
            v_cod, v_min, v_bus USING ERRCODE = 'P0001';
    END IF;
END $$;
CREATE OR REPLACE FUNCTION trg_bus_flota() RETURNS TRIGGER LANGUAGE
plpgsql AS $$
BEGIN
    IF TG_OP IN ('INSERT','UPDATE') AND NEW.id_linea IS NOT NULL THEN
        PERFORM fn_validar_flota_linea(NEW.id_linea);
    END IF;
    IF TG_OP IN ('UPDATE','DELETE') AND OLD.id_linea IS NOT NULL
       AND (TG_OP = 'DELETE' OR OLD.id_linea IS DISTINCT FROM
NEW.id_linea) THEN
        PERFORM fn_validar_flota_linea(OLD.id_linea);
    END IF;
    RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER ct_bus_flota
    AFTER INSERT OR UPDATE OF id_linea OR DELETE ON bus
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_bus_flota();
CREATE OR REPLACE FUNCTION trg_linea_flota() RETURNS TRIGGER LANGUAGE
plpgsql AS $$
BEGIN
```

```
    IF TG_TABLE_NAME = 'linea' THEN
        PERFORM fn_validar_flota_linea(NEW.id_linea);
    ELSIF TG_OP = 'DELETE' THEN
        PERFORM fn_validar_flota_linea(OLD.id_linea);
    ELSE
        PERFORM fn_validar_flota_linea(NEW.id_linea);
    END IF;
    RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER ct_linea_estado
    AFTER UPDATE OF estado ON linea
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_linea_flota();
CREATE CONSTRAINT TRIGGER ct_linea_estacion_flota
    AFTER INSERT OR DELETE ON linea_estacion
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_linea_flota();
-- ------------------------------------------------------------------
---
-- Regla 2: toda estación pertenece al menos a una línea
-- ------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_estacion_con_linea() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_id INT; v_nom TEXT;
BEGIN
    v_id := CASE WHEN TG_TABLE_NAME = 'estacion' THEN NEW.id_estacion
ELSE OLD.id_estacion END;
    SELECT nombre INTO v_nom FROM estacion WHERE id_estacion = v_id;
    IF FOUND AND NOT EXISTS (SELECT 1 FROM linea_estacion WHERE
id_estacion = v_id) THEN
        RAISE EXCEPTION 'La estación % debe pertenecer al menos a una
línea', v_nom USING ERRCODE = 'P0001';
    END IF;
    RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER ct_estacion_con_linea
    AFTER INSERT ON estacion
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_estacion_con_linea();
CREATE CONSTRAINT TRIGGER ct_linea_estacion_borrado
    AFTER DELETE ON linea_estacion
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_estacion_con_linea();
-- ------------------------------------------------------------------
---
-- Regla 10: al menos un guardia vigente por acceso
-- ------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_acceso_con_guardia() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_ids INT[]; v_id INT; v_nom TEXT;
BEGIN
    IF TG_TABLE_NAME = 'acceso' THEN
        v_ids := ARRAY[NEW.id_acceso];
    ELSIF TG_OP = 'DELETE' THEN
        v_ids := ARRAY[OLD.id_acceso];
    ELSE
        v_ids := ARRAY[OLD.id_acceso, NEW.id_acceso];
    END IF;
    FOREACH v_id IN ARRAY v_ids LOOP
        SELECT e.nombre || ', acceso ' || a.numero INTO v_nom
          FROM acceso a JOIN estacion e USING (id_estacion) WHERE
a.id_acceso = v_id;
        IF FOUND AND NOT EXISTS (SELECT 1 FROM asignacion_guardia
                                 WHERE id_acceso = v_id AND
fecha_fin IS NULL) THEN
            RAISE EXCEPTION 'El acceso % (%) debe tener al menos un
guardia de seguridad vigente', v_id, v_nom
                USING ERRCODE = 'P0001';
        END IF;
    END LOOP;
    RETURN NULL;
END $$;
CREATE CONSTRAINT TRIGGER ct_acceso_con_guardia
    AFTER INSERT ON acceso
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_acceso_con_guardia();
CREATE CONSTRAINT TRIGGER ct_guardia_retiro
    AFTER UPDATE OR DELETE ON asignacion_guardia
    DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION
trg_acceso_con_guardia();
-- ------------------------------------------------------------------
---
-- Reglas 6 y 7: cada bus tiene parqueo; el parqueo no puede exceder
su capacidad
-- ------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_bus_parqueo_capacidad() RETURNS
TRIGGER LANGUAGE plpgsql AS $$
DECLARE v_cap INT; v_uso INT; v_nom TEXT;
BEGIN
    SELECT capacidad_buses, nombre INTO v_cap, v_nom FROM parqueo
WHERE id_parqueo = NEW.id_parqueo;
    SELECT count(*) INTO v_uso FROM bus WHERE id_parqueo =
NEW.id_parqueo AND id_bus <> NEW.id_bus;
    IF v_uso >= v_cap THEN
        RAISE EXCEPTION 'El parqueo % está lleno (capacidad %
buses)', v_nom, v_cap USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER tg_bus_parqueo_capacidad
    BEFORE INSERT OR UPDATE OF id_parqueo ON bus
    FOR EACH ROW EXECUTE FUNCTION trg_bus_parqueo_capacidad();
-- Historial de cambios de parqueo
CREATE OR REPLACE FUNCTION trg_bus_parqueo_historial() RETURNS
TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO bus_parqueo_historial (id_bus, id_parqueo) VALUES
(NEW.id_bus, NEW.id_parqueo);
    ELSIF NEW.id_parqueo IS DISTINCT FROM OLD.id_parqueo THEN
        UPDATE bus_parqueo_historial SET hasta = now() WHERE id_bus =
NEW.id_bus AND hasta IS NULL;
        INSERT INTO bus_parqueo_historial (id_bus, id_parqueo) VALUES
(NEW.id_bus, NEW.id_parqueo);
    END IF;
    RETURN NULL;
END $$;
CREATE TRIGGER tg_bus_parqueo_historial
    AFTER INSERT OR UPDATE OF id_parqueo ON bus
    FOR EACH ROW EXECUTE FUNCTION trg_bus_parqueo_historial();
-- ------------------------------------------------------------------
---
-- Recorridos: el bus debe estar asignado a la línea del viaje (regla
4)
-- ------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_viaje_valida() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_linea INT;
BEGIN
    SELECT id_linea INTO v_linea FROM bus WHERE id_bus = NEW.id_bus;
    IF v_linea IS DISTINCT FROM NEW.id_linea THEN
        RAISE EXCEPTION 'El bus % no está asignado a la línea del
viaje', NEW.id_bus USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER tg_viaje_valida
    BEFORE INSERT OR UPDATE OF id_bus, id_linea ON viaje
    FOR EACH ROW EXECUTE FUNCTION trg_viaje_valida();
-- Genera las visitas planificadas de un viaje, en el orden de la
línea (regla 1)
CREATE OR REPLACE FUNCTION fn_generar_visitas(p_id_viaje INT) RETURNS
INT
LANGUAGE plpgsql AS $$
DECLARE
    v viaje%ROWTYPE; r RECORD; t TIMESTAMP; prev_est INT; d NUMERIC;
n INT := 0;
BEGIN
    SELECT * INTO v FROM viaje WHERE id_viaje = p_id_viaje;
    t := v.fecha_hora_salida;
    FOR r IN SELECT id_estacion FROM linea_estacion WHERE id_linea =
v.id_linea
             ORDER BY CASE WHEN v.sentido = 'IDA' THEN orden END
ASC,
                      CASE WHEN v.sentido = 'VUELTA' THEN orden END
DESC
    LOOP
        IF prev_est IS NOT NULL THEN
            d := COALESCE(fn_distancia_tramo(prev_est,
r.id_estacion), 1.0);
            t := t + (d / fn_parametro('VELOCIDAD_PROMEDIO_KMH') *
60) * INTERVAL '1 minute';
        END IF;
        n := n + 1;
        INSERT INTO visita (id_viaje, id_estacion, orden,
hora_programada)
        VALUES (p_id_viaje, r.id_estacion, n, date_trunc('minute',
t));
        t := t + fn_parametro('ESPERA_BASE_MIN') * INTERVAL '1
minute';
        prev_est := r.id_estacion;
    END LOOP;
    RETURN n;
END $$;
-- Regla 15: si el bus no llena el 25 % de su capacidad espera 5
minutos más en la estación
CREATE OR REPLACE FUNCTION trg_visita_ocupacion() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_cap INT; v_linea INT;
BEGIN
    SELECT b.capacidad_pasajeros, v.id_linea INTO v_cap, v_linea
      FROM viaje v JOIN bus b ON b.id_bus = v.id_bus WHERE v.id_viaje
= NEW.id_viaje;
    IF NOT EXISTS (SELECT 1 FROM linea_estacion WHERE id_linea =
v_linea AND id_estacion = NEW.id_estacion) THEN
        RAISE EXCEPTION 'La estación % no pertenece a la línea del
viaje', NEW.id_estacion USING ERRCODE = 'P0001';
    END IF;
    IF NEW.pasajeros_a_bordo IS NULL THEN
        NEW.ocupacion_pct := NULL;
        NEW.espera_extra_min := 0;
    ELSE
        IF NEW.pasajeros_a_bordo > v_cap THEN
            RAISE EXCEPTION 'El bus solo admite % pasajeros y se
```

```
registraron %', v_cap, NEW.pasajeros_a_bordo
                USING ERRCODE = 'P0001';
        END IF;
        NEW.ocupacion_pct := round(100.0 * NEW.pasajeros_a_bordo /
v_cap, 2);
        NEW.espera_extra_min := CASE WHEN NEW.ocupacion_pct <
fn_parametro('OCUPACION_MIN_PCT')
                                     THEN
fn_parametro('ESPERA_EXTRA_MIN')::INT ELSE 0 END;
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER tg_visita_ocupacion
    BEFORE INSERT OR UPDATE ON visita
    FOR EACH ROW EXECUTE FUNCTION trg_visita_ocupacion();
-- ----------------------------------------------------------------------
---
-- Regla 14: alerta cuando la estación acumula 50 % más que la
capacidad de un bus
-- ----------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_lectura_valida() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM linea_estacion WHERE id_linea =
NEW.id_linea AND id_estacion = NEW.id_estacion) THEN
        RAISE EXCEPTION 'La estación % no pertenece a la línea %',
NEW.id_estacion, NEW.id_linea USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END $$;
-- La alerta se crea después de insertar la lectura, para poder
enlazar su identificador
CREATE OR REPLACE FUNCTION trg_lectura_alerta_after() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
DECLARE v_cap INT; v_umbral INT; v_est TEXT; v_lin TEXT;
BEGIN
    SELECT MIN(capacidad_pasajeros) INTO v_cap FROM bus WHERE
id_linea = NEW.id_linea;
    IF v_cap IS NULL THEN RETURN NULL; END IF;
    v_umbral := ceil(v_cap * fn_parametro('ALERTA_AFORO_PCT') /
100.0);
    IF NEW.pasajeros_esperando >= v_umbral THEN
        SELECT nombre INTO v_est FROM estacion WHERE id_estacion =
NEW.id_estacion;
        SELECT codigo INTO v_lin FROM linea WHERE id_linea =
NEW.id_linea;
        INSERT INTO alerta (id_estacion, id_linea, id_lectura, fecha,
pasajeros_esperando, umbral, mensaje)
        VALUES (NEW.id_estacion, NEW.id_linea, NEW.id_lectura,
NEW.fecha, NEW.pasajeros_esperando, v_umbral,
                format('Estación %s: %s pasajeros esperando (umbral
%s = %s%% de la capacidad de un bus). Enviar otro bus de la línea
%s.',
                       v_est, NEW.pasajeros_esperando, v_umbral,
fn_parametro('ALERTA_AFORO_PCT')::INT, v_lin));
    END IF;
    RETURN NULL;
END $$;
CREATE TRIGGER tg_lectura_valida BEFORE INSERT ON lectura_estacion
    FOR EACH ROW EXECUTE FUNCTION trg_lectura_valida();
CREATE TRIGGER tg_lectura_alerta AFTER INSERT ON lectura_estacion
    FOR EACH ROW EXECUTE FUNCTION trg_lectura_alerta_after();
-- Despacho del bus de refuerzo que atiende una alerta
CREATE OR REPLACE FUNCTION fn_atender_alerta(p_id_alerta BIGINT,
p_id_bus INT) RETURNS VOID
LANGUAGE plpgsql AS $$
DECLARE v_linea INT; v_bus_linea INT; v_ya BOOLEAN;
BEGIN
    SELECT id_linea, atendida INTO v_linea, v_ya FROM alerta WHERE
id_alerta = p_id_alerta;
    IF NOT FOUND THEN RAISE EXCEPTION 'La alerta % no existe',
p_id_alerta USING ERRCODE = 'P0001'; END IF;
    IF v_ya THEN RAISE EXCEPTION 'La alerta % ya fue atendida',
p_id_alerta USING ERRCODE = 'P0001'; END IF;
    SELECT id_linea INTO v_bus_linea FROM bus WHERE id_bus =
p_id_bus;
    IF NOT FOUND THEN RAISE EXCEPTION 'El bus % no existe', p_id_bus
USING ERRCODE = 'P0001'; END IF;
    IF v_bus_linea IS NOT NULL AND v_bus_linea <> v_linea THEN
        RAISE EXCEPTION 'El bus % pertenece a otra línea', p_id_bus
USING ERRCODE = 'P0001';
    END IF;
    UPDATE alerta SET atendida = TRUE, id_bus_refuerzo = p_id_bus,
fecha_atencion = now()
     WHERE id_alerta = p_id_alerta;
END $$;
-- ----------------------------------------------------------------------
---
-- Regla 18: pagos con tarjeta de acceso y otros métodos
-- ----------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_pago_aplica() RETURNS TRIGGER LANGUAGE
plpgsql AS $$
DECLARE m metodo_pago%ROWTYPE; t tarjeta_acceso%ROWTYPE; v_tarifa
NUMERIC;
BEGIN
    SELECT * INTO m FROM metodo_pago WHERE id_metodo_pago =
NEW.id_metodo_pago;
    IF NEW.tipo = 'RECARGA' THEN
        IF NEW.id_tarjeta IS NULL THEN
            RAISE EXCEPTION 'Una recarga necesita una tarjeta de
acceso' USING ERRCODE = 'P0001';
        END IF;
        IF NOT m.permite_recarga THEN
            RAISE EXCEPTION 'El método "%" no sirve para recargar',
m.nombre USING ERRCODE = 'P0001';
        END IF;
        IF NEW.monto <= 0 THEN
            RAISE EXCEPTION 'El monto de la recarga debe ser mayor
que cero' USING ERRCODE = 'P0001';
        END IF;
        UPDATE tarjeta_acceso SET saldo = saldo + NEW.monto
         WHERE id_tarjeta = NEW.id_tarjeta AND estado = 'ACTIVA';
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La tarjeta no existe o no está activa'
USING ERRCODE = 'P0001';
        END IF;
    ELSIF m.usa_saldo_tarjeta THEN
        IF NEW.id_tarjeta IS NULL THEN
            RAISE EXCEPTION 'El cobro con tarjeta de acceso necesita
el número de tarjeta' USING ERRCODE = 'P0001';
        END IF;
        SELECT t2.* INTO t FROM tarjeta_acceso t2 WHERE t2.id_tarjeta
= NEW.id_tarjeta;
        SELECT tarifa INTO v_tarifa FROM tipo_tarjeta WHERE
id_tipo_tarjeta = t.id_tipo_tarjeta;
        IF t.estado <> 'ACTIVA' THEN
            RAISE EXCEPTION 'La tarjeta % está %', t.numero, t.estado
USING ERRCODE = 'P0001';
        END IF;
        IF t.saldo < v_tarifa THEN
            RAISE EXCEPTION 'Saldo insuficiente: la tarjeta % tiene
Q% y la tarifa es Q%', t.numero, t.saldo, v_tarifa
                USING ERRCODE = 'P0001';
        END IF;
        UPDATE tarjeta_acceso SET saldo = saldo - v_tarifa WHERE
id_tarjeta = NEW.id_tarjeta;
        NEW.monto := v_tarifa;
    END IF;
    RETURN NEW;
END $$;
CREATE TRIGGER tg_pago_aplica BEFORE INSERT ON pago
    FOR EACH ROW EXECUTE FUNCTION trg_pago_aplica();
-- ----------------------------------------------------------------------
---
-- Auditoría: bitácora que solo admite inserciones
-- ----------------------------------------------------------------------
---
CREATE OR REPLACE FUNCTION trg_auditar() RETURNS TRIGGER LANGUAGE
plpgsql AS $$
DECLARE v_fila JSONB;
BEGIN
    v_fila := CASE WHEN TG_OP = 'DELETE' THEN to_jsonb(OLD) ELSE
to_jsonb(NEW) END;
    INSERT INTO bitacora (tabla, operacion, id_registro, detalle)
    VALUES (TG_TABLE_NAME, TG_OP, v_fila ->> TG_ARGV[0], v_fila);
    RETURN NULL;
END $$;
CREATE TRIGGER aud_linea       AFTER INSERT OR UPDATE OR DELETE ON
linea
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_linea');
CREATE TRIGGER aud_bus         AFTER INSERT OR UPDATE OR DELETE ON
bus
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_bus');
CREATE TRIGGER aud_linea_est   AFTER INSERT OR UPDATE OR DELETE ON
linea_estacion
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_linea');
CREATE TRIGGER aud_tarjeta     AFTER UPDATE OR DELETE ON
tarjeta_acceso
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_tarjeta');
CREATE TRIGGER aud_guardia_asig AFTER INSERT OR UPDATE OR DELETE ON
asignacion_guardia
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_asignacion');
CREATE TRIGGER aud_alerta      AFTER UPDATE ON alerta
    FOR EACH ROW EXECUTE FUNCTION trg_auditar('id_alerta');
CREATE OR REPLACE FUNCTION trg_bitacora_inmutable() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'La bitácora de auditoría no se puede modificar
ni borrar' USING ERRCODE = 'P0001';
END $$;
CREATE TRIGGER tg_bitacora_inmutable BEFORE UPDATE OR DELETE ON
bitacora
    FOR EACH ROW EXECUTE FUNCTION trg_bitacora_inmutable();
```

Apéndice C. Diccionario de datos

Columnas de las 27 tablas con su tipo (las marcadas con * admiten valores nulos). El detalle de llaves y restricciones está en `01_schema.sql`.

Tabla 76
Diccionario de datos

Tabla	Columnas (tipo)
`acceso`	id_acceso (int), id_estacion (int), numero (smallint), nombre (varchar(80)), tipo (varchar(12))
`alerta`	id_alerta (bigint), tipo (varchar(20)), id_estacion (int), id_linea (int), id_lectura* (bigint), fecha (timestamp), pasajeros_esperando (int), umbral (int), mensaje (varchar(250)), atendida (boolean), id_bus_refuerzo* (int), fecha_atencion* (timestamp)
`asignacion_guardia`	id_asignacion (int), id_guardia (int), id_acceso (int), turno (varchar(10)), fecha_inicio (date), fecha_fin* (date)
`asignacion_piloto_bus`	id_asignacion (int), id_bus (int), id_piloto (int), turno (varchar(10)), fecha_inicio (date), fecha_fin* (date)
`bitacora`	id_bitacora (bigint), fecha (timestamp), usuario_bd (varchar(60)), tabla (varchar(40)), operacion (varchar(6)), id_registro* (varchar(40)), detalle* (jsonb)
`bus`	id_bus (int), numero_unidad (varchar(10)), placa (varchar(10)), tipo (varchar(12)), capacidad_pasajeros (int), anio (smallint), estado (varchar(13)), id_linea* (int), id_parqueo (int)
`bus_parqueo_historial`	id_historial (bigint), id_bus (int), id_parqueo (int), desde (timestamp), hasta* (timestamp)
`computadora`	id_computadora (int), id_estacion (int), hostname (varchar(40)), ip_vpn (inet), sistema_operativo (varchar(40)), estado (varchar(12))
`enlace_red`	id_enlace (int), id_estacion (int), rol (varchar(9)), medio (varchar(12)), proveedor (varchar(60)), ancho_banda_mbps (int), protocolo_vpn (varchar(12)), estado (varchar(10))
`estacion`	id_estacion (int), codigo (varchar(10)), nombre (varchar(100)), direccion* (varchar(150)), id_municipalidad (int), latitud (numeric(9,6)), longitud (numeric(9,6)), capacidad_anden (int), es_terminal (boolean)
`guardia`	id_guardia (int), dpi (char(13)), nombres (varchar(80)), apellidos (varchar(80)), telefono (varchar(12)), fecha_contratacion (date), activo (boolean)
`lectura_estacion`	id_lectura (bigint), id_estacion (int), id_linea (int), fecha (timestamp), pasajeros_esperando (int)
`linea`	id_linea (int), codigo (varchar(10)), nombre (varchar(100)), color_hex (char(7)), id_municipalidad (int), estado (varchar(12)), hora_inicio (time without time zone), hora_fin (time without time zone)
`linea_estacion`	id_linea (int), id_estacion (int), orden (smallint)
`metodo_pago`	id_metodo_pago (int), nombre (varchar(50)), usa_saldo_tarjeta (boolean), permite_recarga (boolean)
`municipalidad`	id_municipalidad (int), nombre (varchar(80)), departamento (varchar(60)), direccion* (varchar(150)), telefono* (varchar(20))
`operador`	id_operador (int), dpi (char(13)), nombres (varchar(80)), apellidos (varchar(80)), usuario (varchar(30)), correo (varchar(100)), telefono (varchar(12)), turno (varchar(10)), id_computadora (int), activo (boolean)

Tabla	Columnas (tipo)
pago	id_pago (bigint), id_tarjeta* (int), id_metodo_pago (int), id_estacion (int), tipo (varchar(13)), monto (numeric(8,2)), fecha (timestamp)
parametro_sistema	clave (varchar(40)), valor (numeric(10,2)), descripcion (varchar(200))
parqueo	id_parqueo (int), codigo (varchar(10)), nombre (varchar(100)), capacidad_buses (int), id_estacion* (int), direccion* (varchar(150))
piloto	id_piloto (int), dpi (char(13)), nombres (varchar(80)), apellidos (varchar(80)), fecha_nacimiento (date), licencia_numero (varchar(15)), licencia_tipo (char(1)), fecha_contratacion (date), activo (boolean), direccion_residencia (varchar(150)), zona* (smallint), id_municipalidad_residencia (int), telefono_movil (varchar(12)), telefono_residencial* (varchar(12)), correo (varchar(100)), contacto_emergencia_nombre (varchar(100)), contacto_emergencia_telefono (varchar(12))
piloto_estudio	id_estudio (int), id_piloto (int), nivel (varchar(15)), titulo (varchar(120)), institucion (varchar(120)), anio_inicio (smallint), anio_fin* (smallint), concluido (boolean)
tarjeta_acceso	id_tarjeta (int), numero (char(10)), id_tipo_tarjeta (int), saldo (numeric(8,2)), estado (varchar(10)), fecha_emision (date)
tipo_tarjeta	id_tipo_tarjeta (int), nombre (varchar(30)), tarifa (numeric(6,2))
tramo	id_estacion_a (int), id_estacion_b (int), distancia_km (numeric(6,2))
viaje	id_viaje (int), id_linea (int), id_bus (int), id_piloto (int), sentido (varchar(6)), fecha_hora_salida (timestamp), estado (varchar(10))
visita	id_visita (bigint), id_viaje (int), id_estacion (int), orden (smallint), hora_programada (timestamp), hora_llegada_real* (timestamp), pasajeros_a_bordo* (int), ocupacion_pct* (numeric(5,2)), espera_extra_min (smallint)

Apéndice D. Enlaces a los repositorios y puesta en marcha

Tabla 77
Enlaces del proyecto

Elemento	Enlace
Repositorio con todo el proyecto (código, base de datos, diagramas, documentos)	https://github.com/william700morales-cloud/transmetro-sistema
Aplicación web publicada	https://william700morales-cloud.github.io/transmetro-sistema/
Script de base de datos	https://github.com/william700morales-cloud/transmetro-sistema/tree/main/database
Diagramas UML y ER (fuentes)	https://github.com/william700morales-cloud/transmetro-sistema/tree/main/docs/diagramas
Entregables (PDF, presentaciones, guion)	https://github.com/william700morales-cloud/transmetro-sistema/tree/main/docs/entregables

Nota. El enunciado pide almacenar toda la información en un repositorio; por eso un solo repositorio contiene todo.

D.1 Ejecutar la aplicación en un equipo

```
git clone https://github.com/william700morales-cloud/transmetro-sistema.git
cd transmetro-sistema
npm install
npm run dev            # abre http://localhost:5173
npm run build          # genera la carpeta dist para publicar
```

D.2 Publicar la aplicación en GitHub Pages

1. Crear en GitHub un repositorio público llamado `transmetro-sistema` y subir el contenido de la carpeta del proyecto.
2. En *Settings* → *Pages*, elegir como origen *GitHub Actions*.
3. Cada cambio en la rama principal ejecuta el archivo `.github/workflows/publicar.yml`, que instala las dependencias, compila y publica la carpeta `dist`.
4. La aplicación queda disponible en la dirección de la Tabla 77.

D.3 Crear la base de datos en un servidor PostgreSQL

```
createdb transmetro
psql -d transmetro -f database/01_schema.sql
psql -d transmetro -f database/02_funciones_triggers.sql
psql -d transmetro -f database/03_vistas.sql
psql -d transmetro -f database/04_seguridad.sql
psql -d transmetro -f database/05_seed.sql
psql -d transmetro -f database/07_pruebas.sql
```